

A Project Report

on

**IMPLEMENTATION OF SMOKE DETECTION SYSTEM
USING OPTICAL SMOKE SENSOR & MSP430FRxxxx**

by

Mr. Krishiv S Rana

ABSTRACT

Fire and smoke detection is one of the most critical aspects of industrial and building safety systems. In industrial environments, early and accurate detection of smoke is essential to prevent fire hazards and ensure worker safety. This project implements a high-sensitivity, low-power optical smoke detection system using the Optical Smoke Sensor integrated photometric sensor and the MSP430FRxxxx ultra-low-power microcontroller, with the goal of achieving reliable smoke detection with accurate alarm triggering and no false alarms.

The Optical Smoke Sensor employs a dual-wavelength sensing technique using a Blue LED at 470 nm and an Infrared LED at 850 nm. The blue LED at 470 nm is more sensitive to fine, sub-micron smoke particles that are typically generated during early-stage fire and combustion events. The infrared LED at 850 nm responds more strongly to larger particles such as dust, fog, and steam. Both LEDs are pulsed in independent time slots and the backscattered light from airborne particles is measured by two integrated photodetectors. By comparing the response of both channels, the system can identify the nature of the particulate matter — a strong blue channel response with a relatively lower IR response indicates true fire smoke, while a dominant IR response with a lower blue response indicates larger nuisance particles such as dust or fog. This dual-wavelength approach provides superior sensitivity and selectivity, enabling early-stage smoke detection while effectively eliminating false alarms caused by non-fire particulates. The MSP430FRxxxx microcontroller was specifically chosen for this application due to its ultra-low-power architecture, making it highly suitable for industrial deployment where battery-powered or energy-efficient operation is essential. The microcontroller interfaces with the Optical Smoke Sensor using the I2C protocol at 100 kHz and performs all on-chip signal processing including 32-bit raw ADC data acquisition, 8-sample moving average filtering, Power Transfer Ratio (PTR) computation using device-specific eFUSE calibration values, adaptive baseline tracking, and obscuration level calculation.

Processed sensor data is transmitted to a host PC using a custom Frame Layer protocol implemented over UART at 9600 baud through an RS485 physical layer. Each 21-byte response frame carries the frame count, raw ADC values, PTR values, obscuration levels, alarm status, and a 16-bit CRC checksum for data integrity verification. The host sends a 4-byte request frame to poll the sensor every second. A Python script on the host PC validates the received

frame using CRC16, extracts PTR values, and displays them on a live real-time graph using the matplotlib library.

The system also implements an I2C bus fault detection and 9-clock bus recovery mechanism to ensure robust and uninterrupted operation in electrically noisy industrial environments. Hardware CRC16 computation using the MSP430FRxxxx's built-in CRC module and precomputed PTR scale factors minimize the computational load on the microcontroller.

Testing results confirmed that the system performs as expected. The smoke detection sensitivity was found to be well suited for industrial environments. The alarm triggered correctly when the blue-channel obscuration exceeded the defined threshold of 2.0, with no false alarms observed during testing. The system successfully met the defined threshold requirements, demonstrating reliable and accurate smoke detection suitable for industrial alarm systems.

Table of Contents

Abstract.....	vi
List of Figures.....	xv
List of Tables	xvi
1 Project Definition and Specifications.....	1
1.1 Introduction	1
1.2 Problem Definition.....	1
1.3 Objectives.....	2
1.4 System Block Diagram	2
1.5 Applications of Smoke Detection System	3
1.6 Obscuration	4
2 Optical Smoke Sensor.....	6
2.1 Introduction	6
2.2 Pin Configuration and Description	6
2.3 Absolute Maximum Ratings and Operating Conditions	6
2.4 Internal Architecture and Functional Block Diagram.....	7
2.5 Dual Time Slot Operation	10
2.6 Dual Wavelength Smoke Detection Principle.....	11
2.7 LED Driver Operation.....	12
2.8 Normal Mode Operation and Data Flow	13
2.9 Using a Smoke Chamber with the Optical Smoke Sensor	13
2.10 Current Consumption.....	14
3 MSP430FRxxxx Microcontroller.....	17
3.1 Introduction	17
3.2 Internal Architecture	18
3.3 16-bit RISC CPU Architecture	19
3.4 FRAM Memory Architecture	19
3.5 Clock System.....	22
3.6 eUSCI Modules	24

3.7	Hardware CRC Module	26
3.8	Low Power Modes.....	27
4	I2C Protocol and Sensor Interfacing.....	29
4.1	Introduction to I2C Protocol	29
4.2	I2C Bus Physical Layer	30
4.3	I2C Protocol Fundamentals	31
4.4	Complete I2C Write and Read Sequences.....	35
4.5	I2C Initialization on MSP430FRxxxx.....	37
4.6	Optical Smoke Sensor Sensor Interfacing Summary	38
5	UART Protocol and Frame Layer.....	40
5.1	Introduction to UART	40
5.2	UART Fundamentals	40
5.3	RS485 Physical Layer	43
5.4	Frame Layer Protocol Design	45
5.5	CRC16 Error Detection.....	50
6	Signal Calculations and eFUSE Calibration.....	53
6.1	Introduction.....	53
6.2	LED Peak Current Calculation	54
6.3	Photodetector Current calculation	57
6.4	LED Efficiency Calculation.....	58
6.5	Power Transfer Ratio Calculation.....	61
6.6	Efuse Calibration.....	63
7.	Algorithms.....	65
7.1	Introduction	65
7.2	Moving Average Filter	66
7.3	Baseline Calibration.....	69
7.4	Adaptive Baseline Tracking.....	72
7.5	Obscuration Calculation	74
7.6	Alarm Decision Logic.....	76
8.	Implementation & Test Results.....	75
8.1	Firmware Architecture.....	76
8.2	Main Program Flow.....	77

8.3 UART ISR Operation	79
8.4 Python Host Script.....	80
8.5 Introduction	86
8.6 Test Setup	86
8.7 UART Echo-Back Communication Test	87
8.8 Smoke Detection Test.....	88
8.9 Alarm Trigger and Reset Test.....	90
8.10 Performance Parameters Summary	91
9 Problems Encountered and Solutions	93
Conclusion	95
References.....	96
Appendix	
A Abbreviations.....	97
B C Firmware Source Code Flow	100
C Python Host Script Flow.....	103

List of Figures

1.1	System Block Diagram — Implementation of Smoke Detection System.....	3
2.1	Optical Smoke Sensor Internal Architecture Block Diagram	7
2.2	Dual-Wavelength Backscattering Principle with Time Slot A (Blue) and Time Slot B (IR).....	10
3.1	MSP430FRxxxx Internal Block Diagram.....	18
3.2	MSP430FRxxxx Memory Map with Address Ranges.....	21
3.3	MSP430FRxxxx Clock System Architecture — DCO, LFXT, HFXT Sources and MCLK, SMCLK, ACLK Outputs	22
4.1	I ² C START and STOP Conditions (SDA falls/rises while SCL is HIGH)	32
4.2	I ² C Data Bit Transfer Waveform with ACK and NACK	33
4.3	I ² C 7-Bit Address Frame Structure.....	34
4.4	I ² C Write and Read Sequences for Register Access (16-bit address, 16-bit data)	36
5.1	UART 8N1 Frame Waveform — Idle, Start Bit, 8 Data Bits (LSB First), Stop Bit, Idle.....	42
5.2	RS485 Half-Duplex DE//RE Control Timing for One Complete Polling Cycle .	45
5.3	Frame Layer Protocol — Response Frame (21 bytes).....	46
5.4	Request Frame (4 bytes)	47
7.1	Post-Processing Pipeline: PTR (raw) to Alarm Decision	65
7.2	Moving Average Window (N =8): New sample enters from right, oldest leaves from left, output is the mean of the window contents.....	67
7.3	Circular Buffer Structure for 8-Sample Moving Average: avg_index advances one position each cycle, wrapping from position 7 back to position 0	68

7.4	Effect of 8-Sample Moving Average on PTR: Raw PTR (grey) shows sample-to-sample noise; filtered output (blue) tracks the true mean while suppressing noise peaks.....	69
7.5	Baseline Concept: PTR _{total} is the sum of the chamber background (PTR _{baseline}) and the smoke signal (PTR _{smoke}). Only PTR _{smoke} is used for obscuration calculation.....	70
7.6	Conditional Baseline Update Gate: Baseline tracking is suspended when obscuration exceeds the threshold (shaded region), preventing the baseline from rising during a smoke event.....	74
7.7	Alarm Decision Tree: Three conditions evaluated in priority order for each polling cycle	77
8.1	Foreground/background architecture of the firmware.....	79
8.2	Main Program flow.....	86
8.3	Uart ISR Operation	87
8.4	Python Functional Blocks.....	88
8.5	Frame Reception and parsing	89
8.6	Live Monitoring Graph	90
8.7	Test Setup: Two-Box Arrangement with Smoke Generation Chamber (Box 1) and Sensor Measurement Chamber (Box 2).....	92
8.8	Live PTR Graph During Smoke Detection Test: Blue and IR PTR values rise.....	95
8.9	Live Obscuration Graph During Smoke Detection Test: Blue channel crosses the 2.0 ft ⁻¹ threshold triggering the alarm	97
8.10	Terminal Frame Output During Smoke Event: Alarm byte transitioning from 0x00 to 0x01 as obscuration crosses the threshold	98

List of Tables

2.1	Optical Smoke Sensor Pin Description	15
2.2	TIA Gain Settings, ADC Resolution and Saturation (3 μ s LED Pulse Width, 4 μ s AFE Integration, Register 0x39 and 0x3B = 0x22F0)	15
3.1	FRAM vs Flash vs SRAM Comparison	20
3.2	MSP430FRxxxx Low-Power Modes.....	28
4.1	I ² C Reserved Addresses (7-bit).....	34
4.2	I ² C Connection Between MSP430FRxxxx and Optical Smoke Sensor	38
5.1	RS485 vs RS232 vs TTL UART Comparison	43
5.2	RequestFrame Field Description	47
5.3	Response Frame Field Description (21 bytes, Big Endian).....	49
6.1	LED Peak Current Calculation Summary.....	57
6.2	ADC Resolution Q (nA/LSB) for 3 μ s Pulse Width.....	57
6.3	eFUSE Calibration Equation Parameters by Channel and Module ID.....	64
8.1	Firmware Source File Section Map	80
8.2	Clean Air Baseline Measurements.....	93
8.3	System Performance Parameters	96
9.1	Summary of Problems Encountered and Solutions Applied	97
A.1	List of Abbreviations	98

CHAPTER 1

PROJECT DEFINITION AND SPECIFICATIONS

1.1 Introduction

In modern industrial and building safety systems, fire and smoke detection is one of the most critical requirements. Smoke is the earliest indicator of a fire event, appearing well before flames or significant heat are generated. Early and accurate detection of smoke can prevent loss of life and property by providing sufficient time for evacuation and fire suppression.

This project implements a complete optical smoke detection system using the Optical Smoke Sensor integrated photometric sensor from Analog Devices and the MSP430FRxxxx ultra-low-power microcontroller from Texas Instruments. The system is designed for industrial deployment where reliable, low-power, and communication-capable smoke detection is required. The sensor communicates with the microcontroller using the I2C protocol, and the processed data is transmitted to a host PC using a custom Frame Layer protocol over UART and RS485.

1.2 Problem Definition

Smoke detection systems deployed in industrial environments must satisfy several requirements that are difficult to meet simultaneously. The system must be sensitive enough to detect smoke at early stages before a fire develops, yet selective enough to avoid false alarms caused by non-fire particulates such as dust, steam, and cooking aerosols. Additionally, for industrial and battery-powered deployments, the system must consume minimal power while maintaining continuous monitoring.

This project addresses the following specific problems:

- The need for a miniaturized, fully integrated Optical Smoke Sensor with dual-wavelength detection capability to improve sensitivity and selectivity.
- The requirement for reliable digital communication between the sensor and a microcontroller using the I2C protocol, with fault detection and automatic bus recovery.
- The need for a structured Frame Layer protocol with CRC-based error detection for UART communication over RS485 links used in industrial environments.

- The necessity of accurate signal processing including PTR computation, eFUSE calibration, baseline correction, and obscuration measurement on a resource-constrained microcontroller.
- The requirement for a real-time monitoring and visualization interface on a host PC for observation and analysis of sensor data.

1.3 Objectives

The primary objectives of this project are as follows:

- To interface the Optical Smoke Sensor with the MSP430FRxxxx microcontroller using the I2C protocol at 100 kHz.
- To configure all Optical Smoke Sensor registers for optimal dual-wavelength smoke detection using the Blue LED at 470 nm and the Infrared LED at 850 nm.
- To implement a complete signal processing pipeline on the MSP430FRxxxx including 32-bit raw ADC data acquisition, 8-sample moving average filtering, Power Transfer Ratio (PTR) computation using eFUSE calibration values, adaptive baseline tracking, and obscuration level calculation.
- To design and implement a custom Frame Layer protocol over UART at 9600 baud with CRC16 error detection for reliable data transmission via RS485.
- To implement I2C bus fault detection and a 9-clock bus recovery mechanism for robust operation in electrically noisy industrial environments.
- To develop a Python-based host application for continuous polling, CRC validation, and real-time graphical display of PTR values using the matplotlib library.
- To test and validate the complete system under clean air and smoke conditions to verify alarm triggering and absence of false alarms.

1.4 System Block Diagram

The complete smoke detection system consists of three functional layers: the sensing layer, the processing and communication layer, and the host monitoring layer. Figure 1.1 shows the overall system block diagram.

The Optical Smoke Sensor forms the sensing layer. It integrates two LEDs and two photo detectors along with a complete analog front end and ADC into a single package. The sensor is configured and read by the MSP430FRxxxx microcontroller using the I2C protocol.

The MSP430FRxxxx acts as the central processing unit. It performs sensor initialization, raw data acquisition, signal processing, frame construction, and UART transmission. Communication with the host PC is achieved through the UART peripheral at 9600 baud, using an RS485 transceiver for the physical communication layer.

The host PC runs a Python script that polls the MSP430FRxxxx for sensor data every second, validates received frames using CRC16, extracts PTR values, and displays them in a real-time graph.

The complete signal flow is:

Smoke Particles → Optical Smoke Sensor I2C → MSP430FRxxxx UART/RS485 → Python Script →
PTR Graph

1.5 Applications of Smoke Detection System

1.5.1 Industrial Safety

In industrial environments such as factories, warehouses, and manufacturing plants, early smoke detection is critical to prevent fire-related accidents. The low-power architecture of the MSP430FRxxxx makes this system suitable for deployment as a battery-powered industrial smoke detector node. The RS485 communication interface allows multiple sensor nodes to be networked on a single twisted-pair bus, enabling centralized monitoring of large industrial spaces.

1.5.2 Building Fire Safety

Commercial and residential buildings require smoke detectors that comply with fire safety standards such as ANSI/UL 217, ANSI/UL 268, and EN 14604. The dual-wavelength detection capability of the Optical Smoke Sensor provides improved sensitivity and nuisance rejection compared to single-wavelength systems, making it suitable for building fire safety applications where false alarms are a significant concern.

1.5.3 Automotive Systems

The optical smoke detection principle can be applied in automotive safety systems, particularly for detecting smoke from electrical faults or battery thermal runaway events in electric vehicles. The compact size of the Optical Smoke Sensor and the low-power operation of the MSP430FRxxxx make this combination well suited for in-vehicle applications.

1.5.4 IoT and Embedded Sensor Nodes

The system architecture is directly applicable to IoT smoke sensor nodes. The MSP430FRxxxx's ultra-low-power modes allow the device to operate for extended periods on battery power, waking up only to perform measurements and transmit data. The structured Frame Layer protocol with CRC verification ensures data integrity in wireless or wired IoT communication networks.

1.6 Obscuration

Obscuration is the standard unit of measurement used in smoke detection systems to quantify the amount of smoke present in the air. It is defined as the reduction in the intensity of a light beam passing through a volume of smoke-laden air, and is expressed in units of ft^{-1} or $\%/ft$. The relationship between the obscuration level β and the Power Transfer Ratio (PTR) measured by the Optical Smoke Sensor is given by:

$$\beta = \frac{\text{PTR}}{\gamma} \quad (1.1)$$

where γ is a wavelength-dependent scaling factor:

- $\gamma = 0.64$ for the Blue LED channel (470 nm)
- $\gamma = 0.24$ for the Infrared LED channel (850 nm)

In this project, the blue channel obscuration is used as the primary alarm indicator. When the calculated obscuration value exceeds the defined threshold of 2.0, the system sets the alarm flag in the response frame. The blue channel is preferred for alarm triggering because the 470 nm blue LED is more sensitive to fine, sub-micron smoke particles that are characteristic of early-stage fire events.

The obscuration values transmitted in the response frame are scaled by a factor of 100 and stored as 16-bit unsigned integers, allowing a resolution of 0.01 in the obscuration value while using integer arithmetic in the frame.

CHAPTER 2

OPTICAL SMOKE SENSOR

2.1 Introduction

The Optical Smoke Sensor is a complete integrated optical module for smoke detection, manufactured by Analog Devices, Inc. It is a complete photometric system that uses optical dual-wavelength technology to detect airborne smoke particles. The module integrates a highly efficient photometric front end, two light emitting diodes (LEDs), and two photodiodes (PDs), all housed in a custom package that prevents light from going directly from the LED to the photodiode without first entering the smoke detection chamber.

The front end of the application specific integrated circuit (ASIC) consists of a control block, a 14-bit analog-to-digital converter (ADC) with a 20-bit burst accumulator, and three flexible, independently configurable LED drivers. An on-board sample-to-sample accumulator further enables up to 27 bits per data read. The control circuitry includes flexible LED signaling and synchronous detection. The analog front end (AFE) features best-in-class rejection of signal offset and corruption due to modulated interference commonly caused by ambient light. Data output and functional configuration occur over a 1.8 V I²C interface or Serial Peripheral Interface (SPI) port. In this project, the I²C interface is used exclusively with a fixed slave address of 0x64.

2.2 Pin Configuration and Description

The Optical Smoke Sensor is available in a 24-pin LGA package. Table 2.1 describes all the functional pins of the device.

2.3 Absolute Maximum Ratings and Operating Conditions

The Optical Smoke Sensor operates with VDD at 1.8 V (range 1.7–1.9 V), VLED1 (Blue LED) at 3.0–5.5 V, and VLED3 (IR LED) at 3.3–8.0 V. LED peak current ranges from 12 mA to 370 mA. Both LEDs and the digital/analog supplies must not exceed their absolute maximum ratings to prevent permanent damage. The operating temperature range is –40 °C to +85 °C.

2.4 Internal Architecture and Functional Block Diagram

Figure 2.1 shows the internal architecture of the Optical Smoke Sensor.

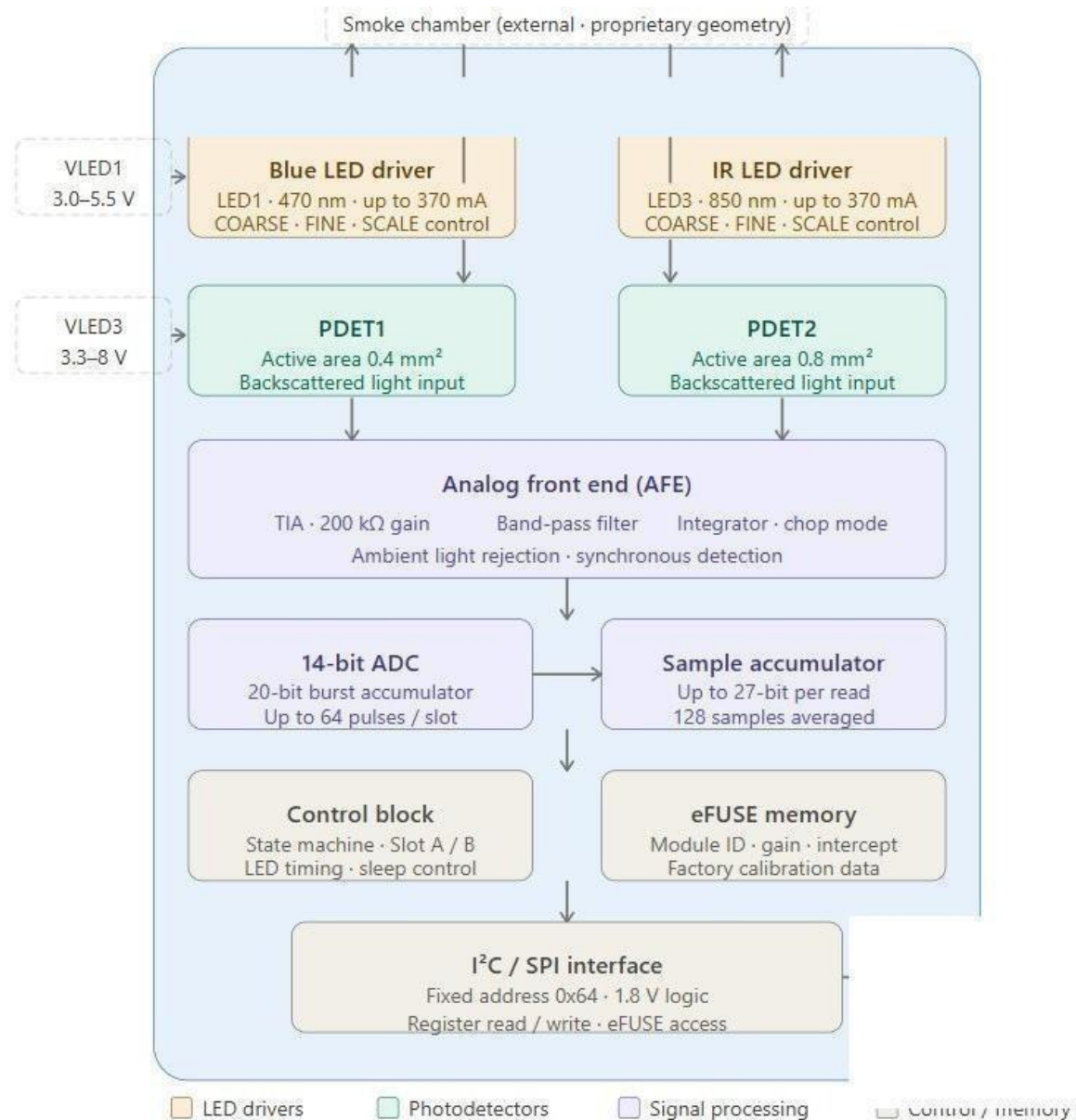


Figure 2.1: Optical Smoke Sensor Internal Architecture Block Diagram

The Optical Smoke Sensor contains the following integrated functional blocks:

2.4.1 LED Drivers

The Optical Smoke Sensor integrates three independent LED drivers (LED1, LED2, LED3), each capable of delivering up to 370 mA peak current. In this device, LED1 drives the Blue LED at 470 nm and LED3 drives the Infrared LED at 850 nm. LED2 is available for connection to an external LED if additional wavelength sensing is required.

Each LED driver is independently configurable through its own set of current control registers. The current is set through three separate control fields: a coarse current setting, a fine current adjustment, and a current scale factor. This three-level control gives the user very fine resolution over the LED drive current across the full range from 12 mA to 370 mA. The LED drivers are pulsed in synchronization with the AFE to implement the synchronous detection scheme that rejects ambient light interference.

2.4.2 Photodetectors

The Optical Smoke Sensor contains two internal photodetectors named PDET1 and PDET2. PDET1 has an active area of 0.4 mm² and PDET2 has an active area of 0.8 mm². The two photodetectors can be used individually or combined for a larger effective active area.

The photodetectors are positioned in the package geometry such that they receive backscattered light from the smoke chamber. The package itself prevents any direct optical path from the LEDs to the photodetectors — all light received by the photodetectors must first enter and scatter within the smoke detection chamber. This is the fundamental physical mechanism that makes the Optical Smoke Sensor specific to backscattering smoke detection rather than forward-scattering detection used by older discrete designs.

2.4.3 Analog Front End (AFE)

The AFE is the core signal conditioning block of the Optical Smoke Sensor. It consists of:

Transimpedance Amplifier (TIA): The TIA converts the small photocurrent from the photodetectors (in the nanoampere range) into a measurable voltage. The TIA feedback resistance determines the gain and therefore the ADC resolution. Four gain settings are available: 25 k Ω , 50 k Ω , 100 k Ω , and 200 k Ω . The selection of TIA gain is a tradeoff between sensitivity (higher gain for smaller signals) and dynamic range (lower gain for larger signals before saturation). Table 2.2 shows the ADC resolution and saturation level for each TIA setting at a 4 μ s LED pulse width.

In this project, 200 k Ω TIA gain is configured (Register 0x55 = 0x0000), providing the highest resolution of 0.41 nA/LSB at 4 μ s pulse width, which is appropriate for smoke detection where the photodetector signals in clean air are in the low nanoampere range.

Band-Pass Filter (BPF): The AFE includes a band-pass filter that removes DC offset from the TIA output, effectively rejecting the constant ambient light contribution to the photodetector current. Only the pulsed LED signal component passes through the filter, providing excellent ambient light rejection.

Integrator: The integrator accumulates the filtered signal during the LED pulse window, improving the charge collection efficiency and SNR.

2.4.4 14-bit ADC and Accumulators

The 14-bit ADC converts the integrator output to a digital code. The Optical Smoke Sensor provides two levels of digital accumulation beyond the 14-bit ADC:

Burst Accumulator (20-bit): Multiple LED pulses within a single time slot are accumulated in the 20-bit burst accumulator. This hardware averaging improves the effective resolution and SNR beyond 14 bits without increasing the sampling interval.

Sample-to-Sample Accumulator (up to 27-bit): Multiple complete sample periods can be accumulated in an additional accumulator, extending the effective bit depth up to 27 bits per data read. This is configured through the decimation and averaging register (0x0011).

The combination of these two accumulators gives the Optical Smoke Sensor a very high dynamic range suitable for detecting trace amounts of smoke while remaining immune to noise.

2.4.5 Control Block and State Machine

The control block manages the timing and sequencing of the entire measurement cycle. It controls the LED pulsing sequence, the AFE timing windows, the ADC conversion, and the data movement to the FIFO output registers. The state machine implements the following states in each measurement cycle:

- **Sleep state:** The device waits between measurement cycles in a low-power state. The duration of the sleep state is determined by the configured sampling frequency.
- **Time Slot A:** The LED assigned to Time Slot A (Blue LED in this project) is pulsed the configured number of times. Each pulse activates the AFE, which samples the photodetector output. The accumulated result is stored in the Slot A data registers.
- **Time Slot B:** The LED assigned to Time Slot B (IR LED in this project) is pulsed the configured number of times. The accumulated result is stored in the Slot B data registers.
- **Processing time:** The device performs post-slot processing. The datasheet specifies 68 μ s for Slot A processing and 20 μ s for Slot B processing.

2.5 Dual Time Slot Operation

The Optical Smoke Sensor implements a dual time slot operating scheme that is the foundation of its dual-wavelength smoke detection capability. Figure 2.2 shows the conceptual timing of the two time slots.

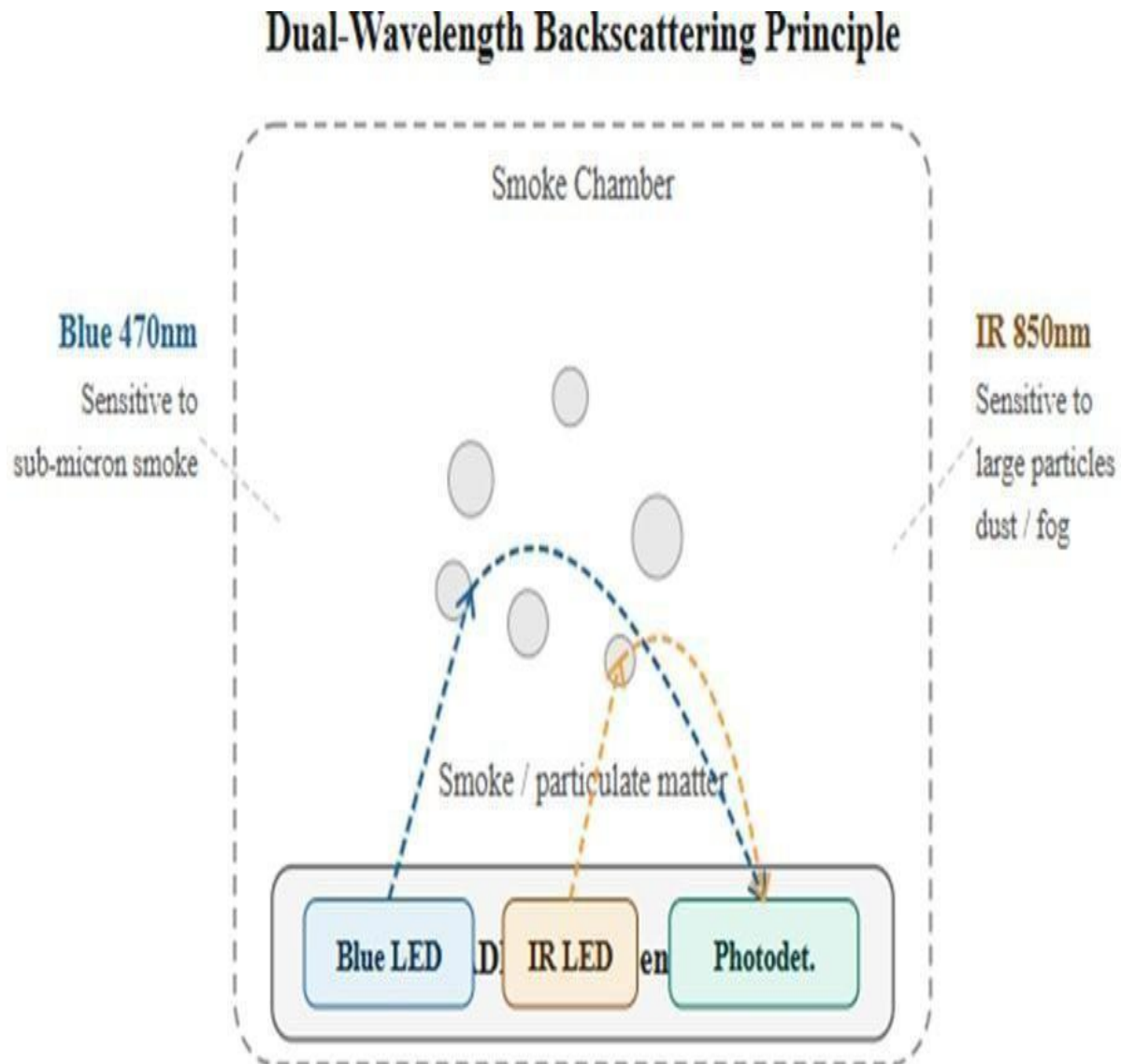


Figure 2.2: Dual-Wavelength Backscattering Principle with Time Slot A (Blue) and Time Slot B (IR)

The two time slots, Time Slot A and Time Slot B, are carried out sequentially within each sampling period. Each time slot has a completely separate and independently configurable datapath that covers the LED driver settings, AFE configuration, pulse count, pulse period, and the resulting accumulated data output. This means the Blue LED and the IR LED can be driven at completely different currents, with different pulse widths and different pulse counts,

with different TIA gain settings, and the resulting data from each channel is stored in separate output registers.

Time Slot A — Blue Channel: The Blue LED (LEDX1, 470 nm) is activated. The configuration is controlled by registers 0x14 through 0x1C for the slot control settings and registers 0x22–0x25 for the LED current. The AFE gain for this slot is set through registers 0x39 and 0x55. The output data is stored in registers 0x70 (low word) and 0x74 (high word), forming a 32-bit accumulated value.

Time Slot B — IR Channel: The IR LED (LEDX3, 850 nm) is activated. The configuration is controlled by registers 0x1D through 0x25, and the output data is stored in registers 0x78 (low word) and 0x7C (high word).

2.6 Dual Wavelength Smoke Detection Principle

The Optical Smoke Sensor detects smoke through the principle of optical backscattering. This section explains this principle in detail from first principles.

2.6.1 Optical Backscattering

When a beam of light travels through air, the photons in the beam interact with any particles present in the air. This interaction causes a portion of the light to be scattered in directions other than the original direction of propagation. The scattered light that returns in the direction of the source is called backscattered light.

The Optical Smoke Sensor uses backscattering rather than the forward-scattering approach used by many traditional optical smoke detectors. In forward scattering, the light source and photodetector are placed on opposite sides of the sensing chamber, and smoke is detected by the reduction in transmitted light intensity. In backscattering, the source and detector are on the same side, and smoke is detected by the increase in light scattered back toward the detector. The backscattering approach used by the Optical Smoke Sensor provides several advantages: it requires only one side of the smoke chamber for the optical components, it produces a positive response to smoke (increasing signal), and the integrated package geometry of the Optical Smoke Sensor naturally implements the backscattering arrangement.

2.6.2 Mie Scattering and Particle Size Dependence

The intensity of light scattered by a particle depends strongly on the ratio of the particle size to the wavelength of the light. This relationship is described by Mie scattering theory. The key practical consequence for smoke detection is:

Blue LED at 470 nm: Light at 470 nm has a shorter wavelength that is comparable in size to the sub-micron particles produced during combustion. Soot particles from fires typically have diameters in the range of $0.01\ \mu\text{m}$ to $1\ \mu\text{m}$. These particles are highly effective at scattering the 470 nm blue light. Therefore, fire smoke produces a strong and immediate response on the blue channel even at low smoke concentrations.

IR LED at 850 nm: Light at 850 nm has a longer wavelength that interacts more effectively with larger particles, typically greater than $1\ \mu\text{m}$. Nuisance aerosols such as water droplets from steam, cooking oil droplets, and dust particles tend to be larger than the sub-micron particles of fire smoke. These larger particles scatter 850 nm infrared light more strongly relative to the 470 nm blue light.

By measuring backscatter at both wavelengths simultaneously, the Optical Smoke Sensor system can identify the type of particulate matter present in the smoke chamber based on the ratio of blue to IR response. This is the physical basis of the dual-wavelength discrimination capability that allows the system to detect fire smoke while rejecting nuisance events that would trigger false alarms in a single-wavelength system.

2.7 LED Driver Operation

The LED drivers in the Optical Smoke Sensor are implemented as programmable current sinks. The LED supply voltage (V_{LED1} for Blue, V_{LED3} for IR) is connected externally to the anode side of the LED. The LED driver pin ($LED1/DNC$ or $LED3/DNC$) connects to the cathode and acts as a current sink. When driving the internal integrated LEDs, the $LED1/DNC$ and $LED3/DNC$ pins must not be connected to any external circuitry.

The LED current is controlled through three levels of adjustment that provide the full current range from 12 mA to 370 mA with fine resolution at any current level:

Coarse Adjustment: A 4-bit field in the LED control register (bits [3:0]) sets the primary current magnitude in steps of approximately 19.8 mA. This determines the major operating point.

Fine Adjustment: A 5-bit field in register 0x0025 provides high-precision adjustment within each coarse step. The fine adjustment uses a separate register shared between the Blue and IR channels with different bit field positions.

Scale Factor: A single bit in the LED control register (bit [13]) provides a global scaling factor. When set to 0, it reduces the current to approximately 10% of the configured coarse-fine level, which is useful for low-power testing. When set to 1, the full configured coarse-fine

current is delivered.

2.8 Normal Mode Operation and Data Flow

In Normal Mode, the Optical Smoke Sensor continuously executes the dual time slot measurement cycle. The data flow from optical input to digital output proceeds as follows:

1. The LED for the active time slot is pulsed the configured number of times (N_{pulses}).
2. For each LED pulse, the optical return signal is received by the photodetectors and converted to a photocurrent.
3. The photocurrent passes through the AFE (TIA + BPF + Integrator). The BPF removes the ambient light component, leaving only the pulsed signal contribution.
4. The integrator output is sampled by the 14-bit ADC.
5. The 14-bit ADC output is added to the 20-bit burst accumulator. After all N_{pulses} are processed, the accumulated result represents the sum of all pulses.
6. If sample-to-sample averaging is enabled through Register 0x0011, multiple complete sample periods are further accumulated in the secondary accumulator.
7. The final accumulated value is stored in the FIFO output registers. For Time Slot A (Blue), the output is in registers 0x70 (low 16-bit word) and 0x74 (high 16-bit word). For Time Slot B (IR), the output is in registers 0x78 and 0x7C.
8. The firmware reads the output by first enabling the data-hold bit (writing 0x0006 to register 0x005F) to prevent mid-read sample updates, reading the two 16-bit words, and then releasing the data-hold by writing 0x0000 to register 0x005F.

2.9 Using a Smoke Chamber with the Optical Smoke Sensor

Most practical smoke detector deployments mount the Optical Smoke Sensor inside a dedicated smoke chamber. The smoke chamber serves three important functions:

Ambient Light Rejection: The chamber provides a dark enclosure around the Optical Smoke Sensor, preventing external ambient light from entering the optical sensing area. Even though the Optical Smoke Sensor has excellent electronic ambient light rejection, the optical isolation provided by a well-designed chamber further improves performance.

Optical Background Signal: When the Optical Smoke Sensor is mounted inside a smoke

chamber, the chamber walls themselves produce a constant background scattering signal. The LED light reflects off the inner surfaces of the chamber, producing a background photocurrent even in clean air. This background contributes to the total measured PTR as $PTR_{CHAMBER}$, such that:

$$PTR_{TOTAL} = PTR_{CHAMBER} + PTR_{SMOKE} \quad (2.1)$$

The chamber background signal must be characterized and subtracted from measurements to obtain the smoke-only PTR_{SMOKE} used for alarm decisions.

Insect and Contamination Protection: The chamber prevents insects, spiders, and other particulates such as dust from permanently settling on the optical components and causing sustained false alarms.

Chamber Geometry: The Optical Smoke Sensor uses a backscattering optical arrangement, which is fundamentally incompatible with conventional smoke chamber designs built for forward-scattering detectors. The Analog Devices proprietary smoke chamber is specifically designed to maximize the SNR for the Optical Smoke Sensor's backscattering geometry. The chamber dimensions are 36 mm across the flanges with an internal sensing area of 109.36 mm².

2.10 Current Consumption

The average current consumption of the Optical Smoke Sensor depends on the sampling frequency, the number of LED pulses, and the LED peak current. The VDD standby current is only 0.3 μA , making the device suitable for battery-powered applications.

These extremely low average current values are achieved because the Optical Smoke Sensor spends most of each sampling period in the sleep state, with the analog circuits active only during the brief LED pulse windows. This ultra-low power characteristic is one of the primary reasons the Optical Smoke Sensor is chosen for industrial and battery-powered smoke detection applications when paired with the low-power MSP430FRxxxx microcontroller.

Table 2.1 shows the optical smoke sensor pin description.

Table 2.1: Optical Smoke Sensor Pin Description

Pin Name	Type	Description
SDA	I/O	I ² C serial data line. Open-drain, requires external pull-up resistor.
SCL	Input	I ² C serial clock line. Open-drain, requires external pull-up resistor.
CS	Input	SPI chip select, active low. Not used in I ² C mode.
SCLK	Input	SPI clock input.
MOSI	Input	SPI master out slave in.
MISO	Output	SPI master in slave out.
GPIO0	I/O	General purpose I/O. Configurable as interrupt output or timing signal.
GPIO1	I/O	General purpose I/O. Configurable as interrupt output or timing signal.
VDD1	Power	Digital supply, 1.8 V.
VDD2	Power	Analog supply, 1.8 V.
VLED1	Power	Blue LED supply voltage, 3.0 V to 5.5 V.
VLED3	Power	IR LED supply voltage, 3.3 V to 8 V (typically 6 V).
LED1/DNC	Output	Blue LED cathode connection. Do not connect when using internal blue LED.
LED3/DNC	Output	IR LED cathode connection. Do not connect when using internal IR LED.
PDET1	Input	Photodetector 1 input (active area 0.4 mm ²).
PDET2	Input	Photodetector 2 input (active area 0.8 mm ²).
EXT_IN1	Input	External input 1, for additional sensors such as CO or temperature.
EXT_IN2	Input	External input 2, for additional sensors.
PDC	Output	Photodiode cathode voltage control.
TIA_VREF	Output	TIA reference voltage output.
AGND	Ground	Analog ground.
DGND	Ground	Digital ground.
LGND	Ground	LED driver ground.

Table 2.2: TIA Gain Settings, ADC Resolution and Saturation (3 μ s LED Pulse Width, 4 μ s AFE Integration, Register 0x39 and 0x3B = 0x22F0)

TIA Gain	Q (nA/LSB)	ADC Saturation (μ A)	TIA Linear Range (μ A)
25 k Ω	3.27	26.8	42.8
50 k Ω	1.64	13.4	21.4
100 k Ω	0.82	6.7	10.7
200 k Ω	0.41	3.35	5.4

CHAPTER 3

MSP430FRXXXX MICROCONTROLLER

3.1 Introduction

The MSP430FRxxxx is a 16-bit mixed-signal microcontroller from Texas Instruments, part of the MSP430FRxxxx9xx family. It belongs to the MSP430 ultra-low-power (ULP) FRAM platform, which combines uniquely embedded Ferroelectric RAM (FRAM) with a holistic ultra-low-power system architecture. The device is designed for applications where minimising energy consumption is a primary requirement while maintaining sufficient processing capability for real-time embedded control and communication tasks.

In this project, the MSP430FRxxxx serves as the central processing unit of the smoke detection system. It is responsible for initialising and configuring the Optical Smoke Sensor sensor over I2C, acquiring raw 32-bit ADC data from the sensor, executing the complete signal processing pipeline including PTR computation and obscuration calculation, constructing the 21-byte response frame, and transmitting it over UART through an RS485 transceiver to the host PC.

The MSP430FRxxxx was selected for this application for the following reasons:

- Its ultra-low-power architecture makes it ideal for industrial and battery-powered smoke detection deployments where long operating life is essential.
- It provides dedicated hardware I2C and UART peripherals through the eUSCI modules, enabling reliable and accurate serial communication without software overhead.
- The built-in hardware CRC-CCITT module eliminates the need for software CRC computation, reducing CPU load during frame transmission.
- The 64 KB of FRAM provides fast, reliable, and essentially unlimited-endurance non-volatile storage for both program code and runtime variables in a unified memory architecture.
- The wide supply voltage range (1.8 V to 3.6 V) is compatible with the 1.8 V VDDIO requirement of the Optical Smoke Sensor I2C interface.

3.2 Internal Architecture

Figure 3.1 shows the complete internal block diagram of the MSP430FRxxxx, illustrating all major functional blocks and their connections through the 16-bit bus fabric.

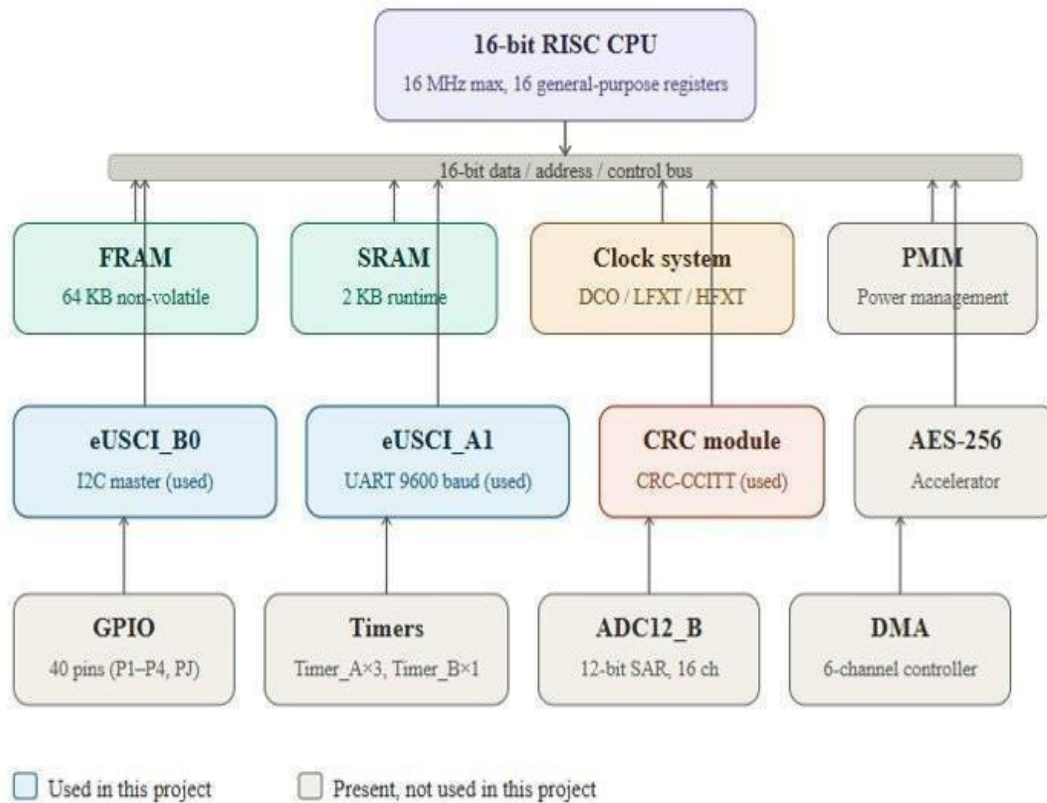


Figure 3.1: MSP430FRxxxx Internal Block Diagram

The MSP430FRxxxx follows the Von Neumann architecture where program code, data, and peripheral registers all share the same address space and are accessed through a unified 16-bit bus. All peripherals are memory-mapped and can be accessed using any MSP430 instruction that can address memory, giving the programmer maximum flexibility in how peripherals are controlled.

The blocks highlighted in blue in Figure 3.1 — eUSCI_B0, eUSCI_A1, and the CRC module — are the three peripheral modules actively used in this project. All other modules are present in the device but are not utilised in the current firmware implementation.

3.3 16-bit RISC CPU Architecture

The MSP430 CPU is a 16-bit reduced instruction set computer (RISC) processor. Its design philosophy emphasises simplicity and code efficiency, which contributes to the ultra-low power characteristic of the MSP430 family.

Register Set: The CPU contains 16 general-purpose 16-bit registers (R0 through R15).

Four of these have dedicated functions:

- **R0 (PC):** Program Counter — holds the address of the next instruction
- **R1 (SP):** Stack Pointer — points to the top of the software stack
- **R2 (SR):** Status Register — holds CPU status flags (Zero, Carry, Negative, Overflow) and power mode control bits
- **R3 (CG):** Constant Generator — provides frequently used constants (0, 1, 2, 4, 8, −1) without requiring memory access

The remaining 12 registers (R4 through R15) are fully general purpose and are used by the compiler for local variables, function arguments, and intermediate computation results.

Instruction Set: The MSP430 instruction set contains 27 core instructions that operate on bytes (8-bit) or words (16-bit). Most instructions execute in a single clock cycle for register-to-register operations, rising to 2–6 cycles for memory access operations. This single-cycle execution capability is the key reason the MSP430 achieves approximately 100 $\mu\text{A}/\text{MHz}$ in active mode — the CPU spends as few clock cycles as possible on each computation.

Orthogonal Architecture: The MSP430 instruction set is orthogonal, meaning that most instructions can be applied to any combination of source and destination (register, immediate, memory). This flexibility means the compiler can generate compact, efficient code without needing special-case instructions for specific data movements.

3.4 FRAM Memory Architecture

The most distinctive feature of the MSP430FRxxxx is its use of FRAM (Ferroelectric Random Access Memory) as the primary non-volatile memory. FRAM is a memory technology that stores data through the polarisation state of a ferroelectric material, combining the non-volatility of Flash with characteristics that closely resemble SRAM in terms of speed and endurance.

3.4.1 FRAM vs Flash vs SRAM Comparison

Table 3.1 compares the three memory technologies relevant to embedded microcontroller design.

Table 3.1: FRAM vs Flash vs SRAM Comparison

Property	FRAM	Flash	SRAM
Non-volatile	Yes	Yes	No
Write speed	125 ns/word	ms (erase required)	ns (SRAM speed)
Endurance	$> 10^{13}$ cycles	$\approx 10^4$ cycles	Unlimited
Write power	Very low	High (erase voltage)	Low
Erase required	No	Yes(blockerase)	No
Density	Medium	High	Medium
Cost	Medium	Low	Low

3.4.2 Unified Memory Architecture

The MSP430FRxxxx uses a unified memory architecture where program code, constant data, and variable data all reside in the same 64 KB FRAM address space. This is fundamentally different from Harvard architecture devices where program memory and data memory are physically separated. The unified FRAM architecture provides two key advantages:

Flexible Memory Partitioning: The developer can choose at compile time how much of the 64 KB is allocated to program code versus data. There is no fixed split — the linker can allocate any proportion to either use. This is particularly useful for data-logging applications where large amounts of sensor data need to be stored alongside the firmware.

Self-Modifying Code and Data Persistence: Since code and data share the same memory, variables declared with the `PERSISTENT` pragma are stored in FRAM and retain their values across device resets without requiring a separate EEPROM.

3.4.3 FRAM Memory Map

Figure 3.2 shows the complete memory map of the MSP430FRxxxx with actual address ranges.

The memory map from the linker file (`lnk_MSP430FRxxxx.cmd`) contains the following segments:

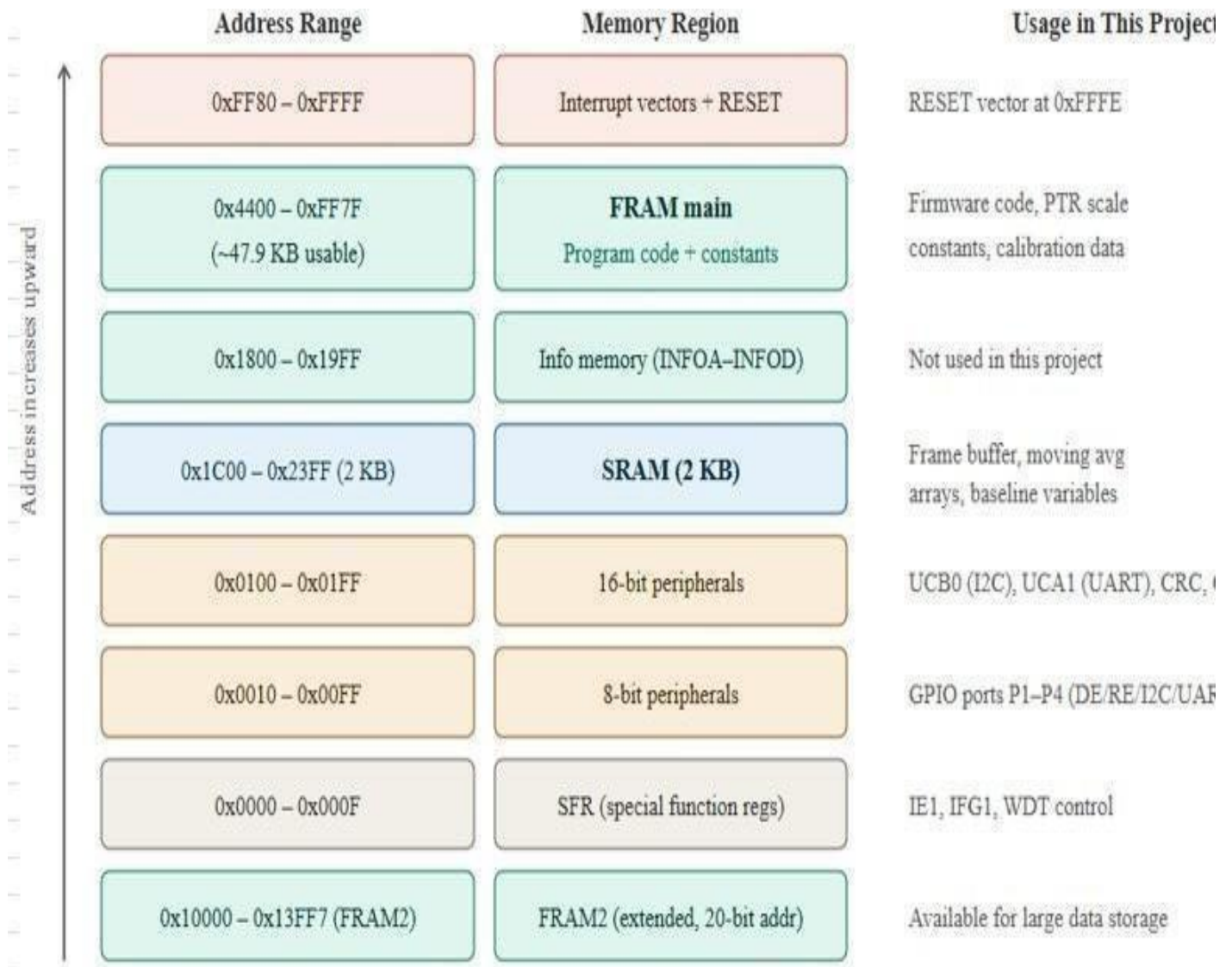


Figure 3.2: MSP430FRxxxx Memory Map with Address Ranges

- SFR: 0x0000 – 0x000F (Special Function Registers)
- PERIPHERALS_8BIT: 0x0010 – 0x00FF (GPIO and 8-bit peripherals)
- PERIPHERALS_16BIT: 0x0100 – 0x01FF (eUSCI, CRC, Clock System, DMA)
- RAM: 0x1C00 – 0x23FF (2 KB SRAM)
- INFOA–INFOD: 0x1800 – 0x19FF (Information memory in FRAM)
- FRAM: 0x4400 – 0xFF7F (Main FRAM — program and data)
- Interrupt vectors: 0xFF80 – 0xFFFF
- FRAM2: 0x10000 – 0x13FF7 (Extended FRAM, 20-bit addressing)

3.4.4 FRAM Wait States

At MCLK frequencies above 8 MHz, the FRAM requires one or more wait states for reliable access. A wait state inserts additional idle clock cycles between when the address is presented and when data is available. At exactly 8 MHz — the frequency used in this project — one wait state is required. This is configured using:

```
FRCTL0 = FRCTLPW | NWAITS_1;
```

The FRCTLPW is the FRAM control password that must be written with any change to the FRCTL0 register. NWAITS_1 sets one wait state. Without this wait state, the FRAM may produce incorrect data at 8 MHz, which could lead to unpredictable firmware behaviour.

3.4.5 FRAM Usage in This Project

In this project, the 64 KB FRAM stores:

- The complete firmware executable code
- All constant tables and initialisation data
- The `SensorFrame_t` frame buffer (`frame_t[1]`)
- The moving average circular buffers (`blue_buffer[8]`, `ir_buffer[8]`)
- The precomputed PTR scale constants (`g_ptr_scale_blue`, `g_ptr_scale_ir`)
- The baseline PTR values (`ptr_blue_baseline`, `ptr_ir_baseline`)

The 2 KB SRAM is used for the runtime stack, local function variables, and the volatile receive buffer (`rx_buffer[4]`) used by the UART ISR.

3.5 Clock System

The MSP430FRxxxx clock system generates the clocks required by the CPU and all peripheral modules from several available sources. Figure 3.3 shows the complete clock system architecture.

Figure 3.3: MSP430FRxxxx Clock System Architecture — DCO, LFXT, HFXT Sources and MCLK, SMCLK, ACLK Outputs

3.5.1 Clock Sources

DCO (Digitally Controlled Oscillator): The internal DCO is a fully digital, software-configurable oscillator that requires no external components. The DCO in the MSP430FRxxxx supports discrete frequency steps including 1 MHz, 4 MHz, 8 MHz, 12 MHz, and 16 MHz. The DCO starts instantly without any stabilisation time, making it ideal for applications that frequently transition between low-power modes and active computation.

LFXT (Low-Frequency Crystal Oscillator): Supports an external 32.768 kHz crystal connected to the XIN and XOUT pins. This provides an accurate low-frequency clock for the ACLK domain, typically used for real-time clock functions and low-power timer operations.

HFXT (High-Frequency Crystal Oscillator): Supports an external high-frequency crystal up to 24 MHz. Used when higher clock accuracy than the DCO is required or when specific frequency standards must be met.

3.5.2 Derived Clocks

The Clock System (CS) module combines the available sources through multiplexers and dividers to produce three derived clocks:

MCLK (Master Clock): The primary clock for the CPU and the instruction execution engine. Sourced from SELM bits in CSCTL2.

SMCLK (Sub-Main Clock): The peripheral clock for high-speed modules including the eUSCI (I2C and UART) modules and the CRC module. Sourced from SELS bits in CSCTL2.

ACLK (Auxiliary Clock): A low-frequency clock for low-power peripherals such as real-time clock timers. Sourced from SELA bits in CSCTL2.

3.5.3 Clock Configuration in This Project

In this project, the DCO is configured to 8 MHz using the `init_clock_8MHz()` function.

The complete register write sequence is:

```
CSCTL0_H = CSKEY >> 8;           // Unlock CS registers
CSCTL1   = DCORSEL | DCOFSEL_3; // DCO = 8 MHz
CSCTL2   = SELS__DCOCLK | SELM__DCOCLK; // Both from DCO
CSCTL3   = DIVS__1 | DIVM__1;    // No division
CSCTL0_H = 0;                   // Lock CS registers
```

This gives MCLK = SMCLK = 8 MHz. The SMCLK is then used as the clock source for:

- **eUSCI_B0 (I2C):** Divides SMCLK by UCB0BRW = 80 to produce the I2C clock: 8 MHz / 80 = 100 kHz
- **eUSCI_A1 (UART):** Uses the oversampling baud rate generator with BR0 = 52 and UCOS16 enabled to produce 9600 bps from 8 MHz SMCLK

3.6 eUSCI Modules

The Enhanced Universal Serial Communication Interface (eUSCI) is the peripheral module family in the MSP430FRxxxx that provides hardware support for synchronous and asynchronous serial communication. The MSP430FRxxxx contains four eUSCI modules: two eUSCI_A and two eUSCI_B.

3.6.1 eUSCI_A Modules — UART and SPI

The eUSCI_A modules (UCA0 and UCA1) each support UART mode and SPI mode. In UART mode, the module provides:

- Full-duplex asynchronous serial communication
- Configurable baud rate using a 16-bit integer divider (UCAxBRW) and a fractional modulation stage (UCAxMCTLW)
- Optional oversampling mode (UCOS16) for more accurate baud rate generation at high ratios of SMCLK to baud rate
- Configurable data bits (7 or 8), parity (none, even, odd), and stop bits (1 or 2)
- Hardware receive and transmit buffers (UCAxRXBUF and UCAxTXBUF)
- Receive interrupt (UCRXIFG) and transmit interrupt (UCTXIFG) flags
- IrDA encoding/decoding support

In this project, **eUSCI_A1 (UCA1)** is configured in UART mode for communication with the host PC via the RS485 transceiver. The baud rate calculation for 9600 bps from 8 MHz SMCLK using oversampling mode is:

$$N = \frac{f_{SMCLK}}{f_{baud}} = \frac{8,000,000}{9600} = 833.33 \quad (3.1)$$

With UCOS16 = 1 (oversampling enabled):

$$\text{UCBRx} = \text{INT} \frac{N}{16} = \text{INT}(52.08) = 52 \quad (3.2)$$

The fractional part is accommodated by the UCBRF and UCBRS modulation fields in the UCAxMCTLW register. In this project, UCBRF = 1 and UCBRS = 0x49 are used, which are the values given in the TI baud rate calculation table for 9600 bps from 8 MHz.

The UART is configured in the `init_uart_9600()` function:

```
UART_CTLW0 = UCSWRST;           // Hold in reset
UART_CTLW0 |= UCSSEL__SMCLK;    // SMCLK source
UART_BR0    = 52;                // Baud rate divisor
UART_BR1    = 0;

UART_MCTLW  = UCOS16 | UCBRF_1 | (0x49 << 8);
UART_CTLW0 &= ~UCSWRST;        // Release reset
UART_IE     |= UCRXIE;          // Enable RX interrupt
```

3.6.2 eUSCI_B Modules — I2C and SPI

The eUSCI_B modules (UCB0 and UCB1) each support I2C mode and SPI mode. In I2C mode, the module provides:

- Master and slave operation
- 7-bit and 10-bit device addressing
- Standard Mode (100 kHz), Fast Mode (400 kHz), and Fast Mode Plus (1 MHz)
- Automatic START, STOP, repeated START, ACK, and NACK generation in master mode
- Hardware detection of arbitration loss and bus errors
- Slave address register (UCBxI2CSA) for storing the target slave address
- Receive and transmit interrupt flags (UCRXIFG0, UCTXIFG0)

In this project, **eUSCI_B0 (UCB0)** is configured as I2C master for communication with the Optical Smoke Sensor sensor at address 0x64. The I2C clock frequency is:

$$f_{I2C} = \frac{f_{SMCLK}}{UCB0BRW \ 80} = \frac{8 \text{ MHz}}{80} = 100 \text{ kHz} \quad (3.3)$$

The I2C is configured in the `I2C_Init()` function:

```
UCB0CTLW0 |= UCSWRST;           // Hold in reset
UCB0CTLW0 = UCMST | UCMODE_3 | UCSYNC
                | UCSSEL_SMCLK;   // I2C master, SMCLK
UCB0BRW      = 80;               // 8MHz/80 = 100kHz
UCB0I2CSA    = 0x64;             // Slave address
P1SEL1      |= BIT6 | BIT7;      // P1.6=SDA, P1.7=SCL
P1SEL0      &= ~(BIT6 | BIT7);
UCB0CTLW0 &= ~UCSWRST;          // Release reset
```

3.7 Hardware CRC Module

The MSP430FRxxxx contains a dedicated hardware CRC (Cyclic Redundancy Check) module that implements the CRC-CCITT polynomial. This module performs all CRC computation in hardware without consuming additional CPU cycles beyond writing each input byte to the module's data register.

3.7.1 CRC-CCITT Polynomial

The CRC-CCITT uses the generator polynomial:

$$G(x) = x^{16} + x^{12} + x^5 + 1 \quad (3.4)$$

This is represented as the 16-bit hexadecimal value 0x1021. The CRC-CCITT is specifically designed to detect:

- All single-bit errors
- All double-bit errors (for any message length up to 32,767 bits)
- All error bursts of 16 bits or fewer
- Most error bursts longer than 16 bits

These error detection properties make the CRC-CCITT highly suitable for detecting transmission errors in the UART Frame Layer protocol used in this project.

3.7.2 CRC Module Operation

The hardware CRC module uses two key registers:

- **CRCINIRE**: CRC initialisation and result register. Writing to this register sets the initial CRC value. Reading from it after processing all data bytes gives the final CRC result.
- **CRCDIRB_L**: CRC data input register (byte access). Each byte of the data payload is written here. The hardware automatically updates the running CRC after each byte.

In this project, the `crcl6_hw_bytes()` function uses the hardware module:

```
uint16_t crcl6_hw_bytes(uint8_t *data, uint16_t len)
{
    uint16_t i;
    CRCINIRE = 0xFFFF;           // Initialize CRC
    for (i = 0; i < len; i++)
        CRCDIRB_L = data[i];     // Feed each byte
    return CRCINIRE;             // Return result
}
```

The hardware processes each byte in a single clock cycle, enabling CRC computation of the 19-byte frame payload in 19 clock cycles, taking only 2.375 μ s at 8 MHz. The Python host script implements the identical CRC-CCITT algorithm in software for frame verification.

3.8 Low Power Modes

The MSP430FRxxxx provides seven operating power modes: Active Mode and six Low-Power Modes (LPM0 through LPM4.5). Each mode progressively shuts down more internal circuitry to reduce current consumption. Table 3.2 shows low power modes

Entering Low-Power Modes: Low-power modes are entered by setting specific bits in the Status Register (SR) using the `__bis_SR_register()` intrinsic or by using the low-power mode macros such as `__low_power_mode_3()`.

Waking from Low-Power Modes: The device wakes from LPM0 through LPM4 when an enabled interrupt fires. For LPM3.5 and LPM4.5, special wakeup sequences are required. After

Table 3.2: MSP430FRxxxx Low-Power Modes

Mode	CPU	Typical Current	Active Clocks / Notes
Active	ON	$\approx 100 \mu\text{A}/\text{MHz}$	MCLK, SMCLK, ACLK all active
LPM0	OFF	–	SMCLK and ACLK active, MCLK off
LPM1	OFF	–	SMCLK and ACLK active, DCO off if unused
LPM2	OFF	–	ACLK active, SMCLK and DCO off
LPM3	OFF	$0.4 \mu\text{A}$	ACLK only (VLO or XT1), all others off
LPM3.5	OFF	$0.25 \mu\text{A}$	RTC only. RAM content not retained
LPM4.5	OFF	$0.02 \mu\text{A}$	All clocks off. Wake on external interrupt only

waking from a low-power mode, the CPU resumes execution from the instruction following the low-power mode entry.

Peripherals in Low-Power Modes: Most peripherals can remain operational in LPM3 if they are clocked by ACLK rather than SMCLK. The UART receive interrupt can wake the device from LPM0 since SMCLK remains active.

Usage in This Project: In the current firmware implementation, the microcontroller operates in Active Mode throughout. The main while loop continuously checks the `request_ready` flag set by the UART ISR. Future optimisation could implement LPM0 sleep between polling cycles, with the UART receive interrupt configured to wake the device when a request arrives, significantly reducing average current consumption.

CHAPTER 4

I2C PROTOCOL AND SENSOR INTERFACING

4.1 Introduction to I2C Protocol

The Inter-Integrated Circuit (I²C) protocol is a synchronous, serial, multi-master, multi-slave communication bus developed by Philips Semiconductor (now NXP Semiconductors) in 1982. It was originally designed to allow microcontrollers to communicate with low-speed peripheral devices such as EEPROMs, ADCs, sensors, and display controllers using only two wires.

Today, I²C has become a world standard that is implemented in thousands of integrated circuits and licensed to hundreds of semiconductor companies. The official specification is published by NXP as document UM10204, “I²C-bus Specification and User Manual,” with the most recent version being Revision 7.0 (October 2021).

The fundamental advantages of I²C that make it ideal for the sensor interface in this project are:

- Only two signal lines are required for communication with any number of devices on the same bus, greatly simplifying PCB layout and reducing pin count.
- Each device has a unique address, allowing the master to address any specific slave without dedicated chip-select lines.
- The protocol is self-clocked and synchronous — the master generates the clock and the slave synchronises to it, eliminating baud rate mismatch issues.
- The open-drain bus topology with pull-up resistors provides a safe wired-AND connection, preventing bus contention even when multiple masters are connected.
- The protocol includes built-in acknowledgement (ACK/NACK) for every byte transmitted, providing simple error detection at the protocol level.

In this project, the MSP430FRxxxx operates as the sole I²C master and the Optical Smoke Sensor optical smoke sensor operates as the I²C slave with a fixed address of 0x64.

4.2 I2C Bus Physical Layer

4.2.1 Signal Lines

The I²C bus uses exactly two bidirectional signal lines:

SDA (Serial Data Line): Carries the data bits, address bytes, and acknowledgement bits in both directions. Both master and slave can drive SDA LOW.

SCL (Serial Clock Line): Carries the clock signal generated by the master. In normal operation only the master drives SCL, though slaves can hold SCL LOW to implement clock stretching (see Section 4.3.5).

Both lines use open-drain (or open-collector) output stages. This means a device can only actively pull a line LOW by connecting it to ground — it cannot actively drive the line HIGH. The lines are pulled HIGH by external pull-up resistors connected to the supply voltage. When no device is actively pulling a line LOW, the pull-up resistor holds it HIGH.

4.2.2 Pull-up Resistors

The pull-up resistors on SDA and SCL are critical to I²C operation. They must be sized correctly to satisfy two constraints:

Maximum pull-up resistance ($R_{p,max}$): The pull-up resistor must be small enough to pull the bus line HIGH within the required rise time t_r . For Standard Mode (100 kHz), the maximum rise time is 1000 ns. For Fast Mode (400 kHz), it is 300 ns. The maximum resistance is:

$$R_{p,max} = \frac{t_r}{0.8473 \times C_{bus}} \quad (4.1)$$

where C_{bus} is the total capacitance on the bus line including all device pins and PCB traces.

Minimum pull-up resistance ($R_{p,min}$): The pull-up resistor must be large enough to limit the sink current when any device pulls the bus LOW. The minimum resistance is:

$$R_{p,min} = \frac{V_{DD} - V_{OL(max)}}{I_{OL}} \quad (4.2)$$

where $V_{OL(max)}$ is the maximum LOW output voltage (0.4 V for Standard/Fast Mode) and I_{OL} is the maximum sink current (3 mA typically).

In this project, 4.7 k Ω pull-up resistors are used on SDA and SCL, connected to 1.8 V (the VDDIO level of the Optical Smoke Sensor). This value is appropriate for Standard Mode at 100 kHz with typical bus capacitances up to 100 pF.

4.2.3 Bus Logic Levels

The I²C specification defines logic levels as fractions of the supply voltage V_{DD} rather than absolute voltages:

- Logic HIGH: $V_{IH} \geq 0.7 \times V_{DD}$
- Logic LOW: $V_{IL} \leq 0.3 \times V_{DD}$
- Noise margin HIGH: $V_{nH} \geq 0.2 \times V_{DD}$
- Noise margin LOW: $V_{nL} \geq 0.1 \times V_{DD}$

At 1.8 V supply (Optical Smoke Sensor VDDIO):

- $V_{IH} \geq 1.26 \text{ V}$
- $V_{IL} \leq 0.54 \text{ V}$

The MSP430FRxxxx GPIO pins are compatible with 1.8 V I²C logic, making direct connection of the I²C lines between the two devices possible without any level shifter.

4.3 I²C Protocol Fundamentals

4.3.1 START and STOP Conditions

Two special conditions that are not data are defined by the I²C specification: the START condition and the STOP condition. These conditions are the only times when the SDA line is allowed to change state while SCL is HIGH.

START Condition (S): A HIGH to LOW transition on SDA while SCL is HIGH. The START condition signals all slaves on the bus that a new transaction is beginning. Every I²C transaction begins with a START condition generated by the master.

STOP Condition (P): A LOW to HIGH transition on SDA while SCL is HIGH. The STOP condition signals the end of the current transaction and releases the bus. After the STOP condition, the bus is considered free.

Repeated START (Sr): A new START condition generated without a preceding STOP condition. The repeated START is used when the master needs to switch between write and read directions without releasing the bus (for example, sending a register address in write mode and then reading the register data in read mode). The Optical Smoke Sensor register read operation uses a repeated START.

Figure 4.1 shows the SDA and SCL waveforms for the START and STOP conditions.

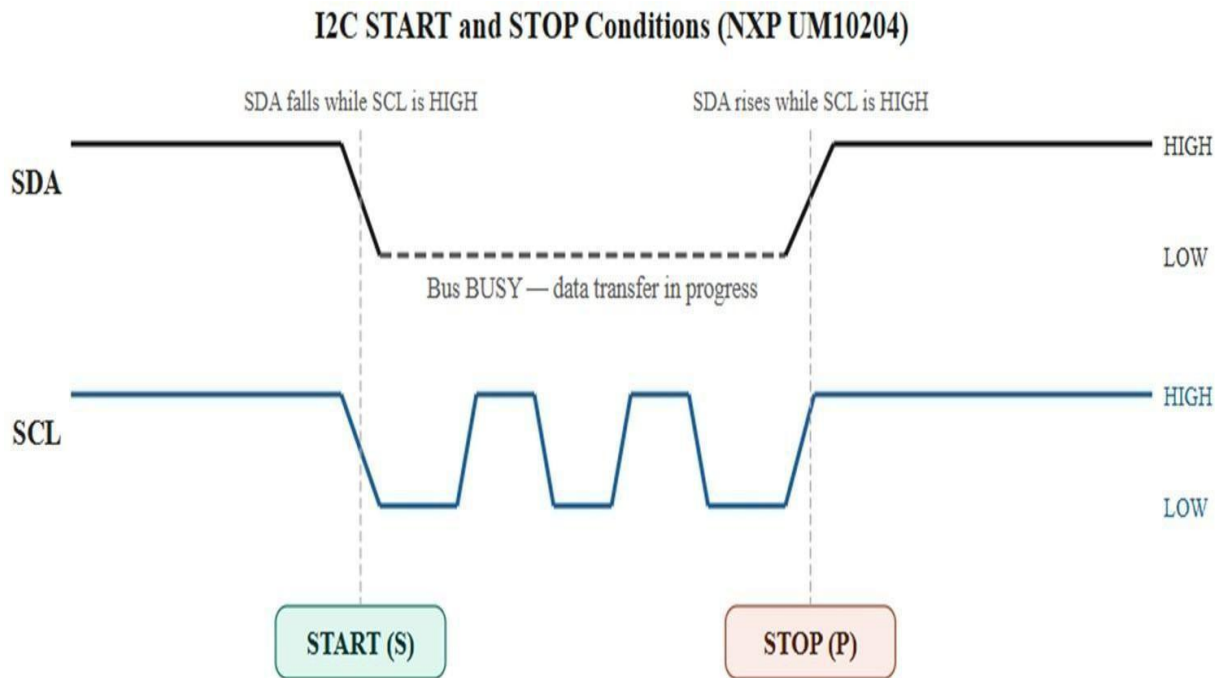


Figure 4.1: I²C START and STOP Conditions (SDA falls/rises while SCL is HIGH)

4.3.2 Data Transfer and Byte Format

Every byte placed on the SDA line must be 8 bits long. Data is transferred MSB (Most Significant Bit) first. The number of bytes in a transfer is unrestricted.

The fundamental rule of I²C data transfer is:

SDA must only change state when SCL is LOW. SDA must remain stable when SCL is HIGH.

A change in SDA when SCL is HIGH is interpreted as a START or STOP condition, not as data. This is the physical mechanism that distinguishes the START/STOP control conditions from data bits.

Figure 4.2 shows the SDA and SCL waveforms for data bit transfer, ACK, and NACK.

4.3.3 Acknowledge (ACK) and Not-Acknowledge (NACK)

After the master transmits each byte, it releases SDA HIGH and generates one additional clock pulse (the 9th clock pulse of each byte). During this 9th clock pulse, the receiver responds:

ACK (Acknowledge): The receiver pulls SDA LOW during the 9th clock pulse. This signals to the master that the byte was received successfully and the transfer should continue.

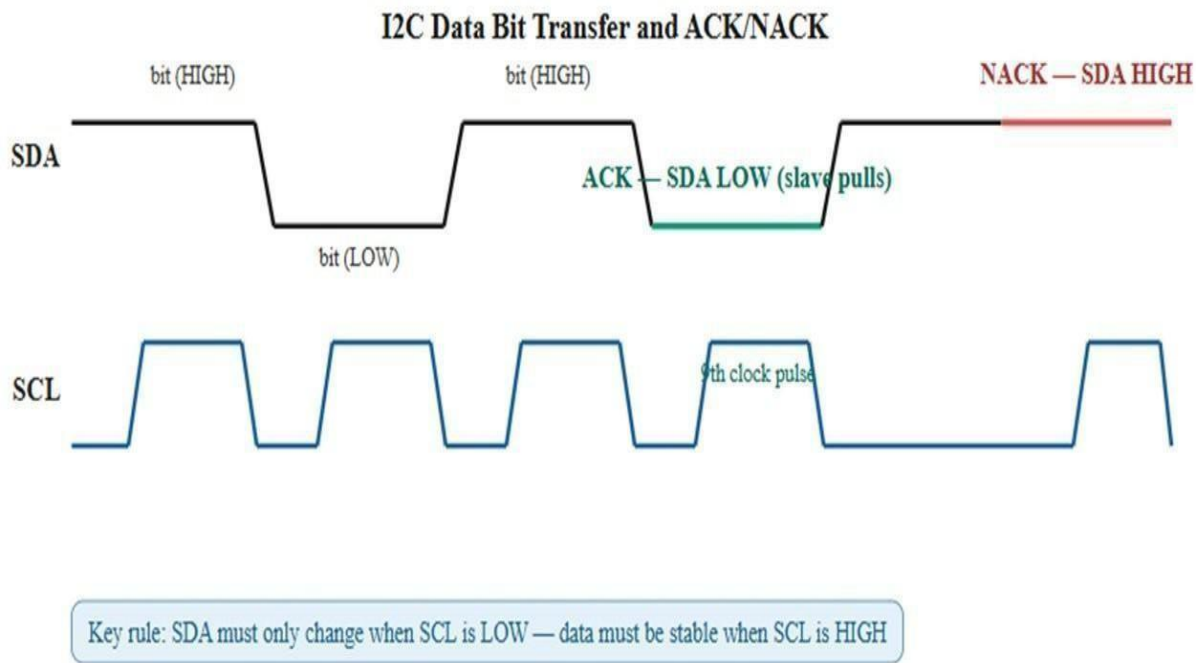


Figure 4.2: I2C Data Bit Transfer Waveform with ACK and NACK

NACK (Not-Acknowledge): The receiver allows SDA to remain HIGH (does not pull it LOW) during the 9th clock pulse. This signals to the master that the byte was not received or that the receiver wants to stop the transfer.

According to the NXP I2C specification, there are five conditions that cause a NACK:

1. No device with the transmitted address is present on the bus.
2. The receiver is not ready to receive or transmit data.
3. The receiver receives data or commands it does not understand.
4. The receiver cannot accept more data bytes.
5. A master-receiver signals to a slave-transmitter that the last byte of data has been transferred (intentional NACK to terminate the read).

In the Optical Smoke Sensor firmware, every I2C write and read operation checks for timeout conditions that indicate an absent ACK. When a timeout occurs, the `i2c_error_flag` is set and the bus recovery procedure is initiated.

4.3.4 7-bit Addressing

After the START condition, the master sends a 7-bit device address followed by a 1-bit Read/Write direction indicator. This forms the first 8-bit byte of every I2C transaction, called

the address byte.

Figure 4.3 shows the structure of the 7-bit address frame.

Figure 4.3 shows the structure of the 7-bit address frame.

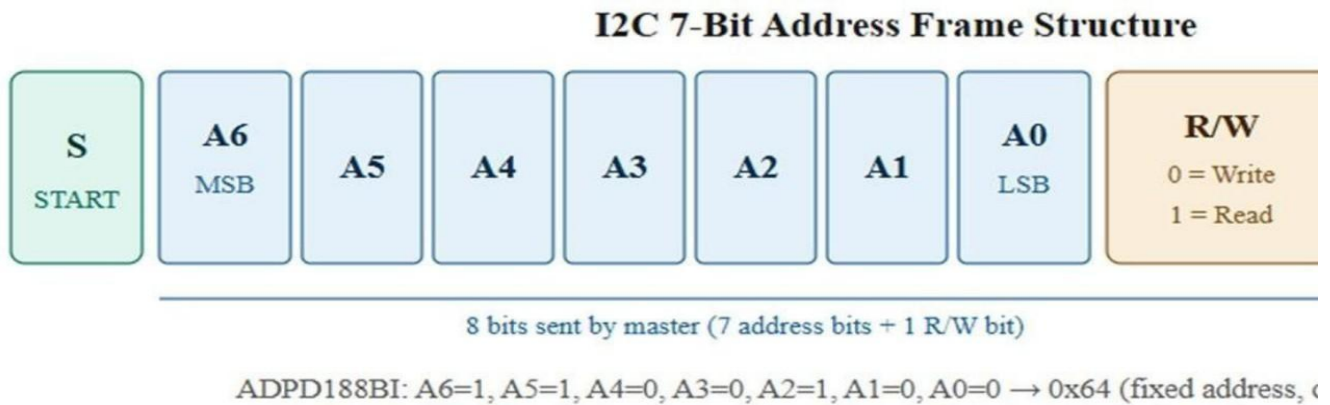


Figure 4.3: I²C 7-Bit Address Frame Structure

Figure 4.3: I²C 7-Bit Address Frame Structure

The 7-bit address allows addressing of up to 128 devices (0x00 through 0x7F), of which several addresses are reserved:

Table 4.1: I²C Reserved Addresses (7-bit)

Address	Reserved Purpose
0x00	General call address
0x01	CBUS address
0x02	Reserved for different bus format
0x03	Reserved for future use
0x04–0x07	High-speed master code
0x78–0x7B	10-bit slave addressing
0x7C–0x7F	Reserved for future use

The **Read/Write (R/W) bit** follows the 7-bit address:

- R/W = 0 (Write): The master will transmit data to the slave after the address byte
- R/W = 1 (Read): The master will receive data from the slave after the address byte

Optical Smoke Sensor Address: The Optical Smoke Sensor has a fixed I²C address of 0x64 (binary: 1100100). Therefore:

- Write address byte: $0x64 \times 2 + 0 = 0xC8$ (sent as 7-bit address 1100100 + W bit 0)
- Read address byte: $0x64 \times 2 + 1 = 0xC9$ (sent as 7-bit address 1100100 + R bit 1)

In the MSP430FRxxxx eUSCI_B0 hardware, the 7-bit slave address 0x64 is loaded into the UCB0I2CSA register. The hardware automatically appends the R/W bit based on whether UCTR (transmitter) is set or cleared.

4.3.5 Clock Stretching

Clock stretching is a mechanism that allows an I²C slave to slow down the communication if it needs more time to process data. When a slave needs to hold the master's clock, it pulls SCL LOW after the master has released it HIGH, effectively stretching the LOW period of the clock.

The master must not generate the next SCL HIGH pulse while the slave is holding SCL LOW. Instead, the master detects that SCL is not HIGH despite releasing it, and waits until SCL is released by the slave before continuing.

The Optical Smoke Sensor may use clock stretching during register reads when it needs additional time to prepare data. The MSP430FRxxxx eUSCI_B0 hardware handles clock stretching automatically by monitoring the SCL line during the START condition wait cycle.

4.4 Complete I2C Write and Read Sequences

Figure 4.4 shows the complete transaction diagrams for I²C register write and register read operations as implemented for the Optical Smoke Sensor, which uses 16-bit register addresses and 16-bit data values.

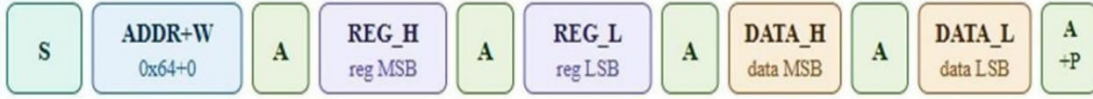
4.4.1 Register Write Sequence

The write transaction for the Optical Smoke Sensor consists of the following steps in order:

1. Master generates START condition (S)
2. Master sends address byte: 0x64 + W (0xC8) — 7-bit slave address + Write bit
3. Slave responds with ACK
4. Master sends register address high byte (REG_H)
5. Slave responds with ACK
6. Master sends register address low byte (REG_L)
7. Slave responds with ACK

I2C Write and Read Sequence (Register Access for ADPD188BI)

Write Sequence:



Read Sequence Phase 1 — Send register address:



Read Sequence Phase 2 — Repeated START, switch to read, receive data:



Master transmits
 Slave transmits
 ACK (slave pulls SDA LOW)
 NACK (master, last byte)

Figure 4.4: I²C Write and Read Sequences for Register Access (16-bit address, 16-bit data)

Figure 4.4: I²C Write and Read Sequences for Register Access (16-bit address, 16-bit data)

8. Master sends data high byte (DATA_H)
9. Slave responds with ACK
10. Master sends data low byte (DATA_L)
11. Slave responds with ACK
12. Master generates STOP condition (P)

Total bytes transmitted: 5 data bytes (address, REG_H, REG_L, DATA_H, DATA_L), each followed by an ACK pulse. Total clock cycles: $5 \times 9 = 45$ clock cycles plus START and STOP overhead, giving approximately $470 \mu\text{s}$ per register write at 100 kHz.

In the firmware, the `smoke_sensor_Writereg()` function implements this sequence using the MSP430FRxxxx eUSCI_B0 hardware. The UCTXSTT bit initiates the START condition and the UCTXSTP bit generates the STOP condition after the last byte is confirmed sent.

4.4.2 Register Read Sequence

The Optical Smoke Sensor register read uses a combined write-then-read transaction with a Repeated START. This is necessary because the Optical Smoke Sensor does not have an auto-

increment register pointer — the master must explicitly specify the register address it wants to read before performing the read.

The read transaction consists of two phases:

Phase 1 — Send register address (write):

1. Master generates START condition (S)
2. Master sends 0x64 + W (write mode)
3. Slave ACKs
4. Master sends register address high byte (REG_H), slave ACKs
5. Master sends register address low byte (REG_L), slave ACKs

Phase 2 — Read register data:

1. Master generates Repeated START (Sr) without releasing the bus
2. Master sends 0x64 + R (read mode)
3. Slave ACKs
4. Slave sends data high byte (DATA_H), master ACKs
5. Slave sends data low byte (DATA_L), master sends NACK (signals last byte)
6. Master generates STOP condition (P)

Total clock cycles: approximately 54, giving approximately 560 μ s per register read at 100 kHz. In the firmware, the `smoke_sensor_Readreg()` function implements this sequence. The transition from write to read mode is achieved by clearing the UCTR bit and setting UC-TXSTT to generate the Repeated START with the read address.

4.5 I2C Initialization on MSP430FRxxxx

The eUSCI_B0 I2C master is initialised in the `I2C_Init()` function. The complete initialisation sequence is:

```
void I2C_Init(void)
{
```

```

FRCTL0      = FRCTLPW | NWAITS_1; // FRAM wait state
UCB0CTLW0 |= UCSWRST;    // Hold peripheral in reset

UCB0CTLW0 = UCMST          // Master mode
          | UCMODE_3       // I2C mode (mode 3)
          | UCSYNC         // Synchronous mode
          | UCSSEL__SMCLK; // Clock = SMCLK (8 MHz)

UCB0BRW      = 80;        // 8MHz / 80 = 100 kHz
UCB0I2CSA = 0x64;        // Optical Smoke Sensor address

P1SEL1 |= BIT6 | BIT7; // P1.6=SDA, P1.7=SCL
P1SEL0 &= ~(BIT6 | BIT7); // Peripheral function 2

UCB0CTLW0 &= ~UCSWRST; // Release from reset
}

```

The UCSWRST (software reset) bit must be set before configuring the eUSCI module and cleared after configuration to begin operation. While UCSWRST is set, the module is held in a known initialised state and ignores any activity on the SDA and SCL lines.

4.6 Optical Smoke Sensor Sensor Interfacing Summary

The complete I²C connection between the MSP430FRxxxx and the Optical Smoke Sensor is summarised in Table 4.2.

Table 4.2: I²C Connection Between MSP430FRxxxx and Optical Smoke Sensor

Signal	MSP430FRxxxx Pin	Optical Smoke Sensor Pin	Pull-up
SDA	P1.6 (UCB0 SDA)	SDA	4.7 k Ω to 1.8 V
SCL	P1.7 (UCB0 SCL)	SCL	4.7 k Ω to 1.8 V

The I²C clock frequency is 100 kHz (Standard Mode). The Optical Smoke Sensor slave address is 0x64. All register accesses use 16-bit register addresses and 16-bit data values. Read operations use a combined write-then-read transaction with Repeated START. Each register write takes approximately 470 μ s and each register read takes approximately 560 μ s at 100 kHz.

The total I²C sensor read time per polling cycle is:

- Write 0x005F (data hold enable): $\approx 470\ \mu s$
- Read Blue channel low word (0x0070): $\approx 560\ \mu s$
- Read Blue channel high word (0x0074): $\approx 560\ \mu s$
- Write 0x005F (data hold release): $\approx 470\ \mu s$
- Same four operations for IR channel

Total I²C sensor read time per frame: approximately 4.12 ms for both Blue and IR channels combined.

CHAPTER 5

UART PROTOCOL AND FRAME LAYER

5.1 Introduction to UART

Universal Asynchronous Receiver/Transmitter (UART) is one of the oldest and most widely used serial communication protocols in embedded systems. It was first designed by Gordon Bell at DEC for the PDP series of computers and has remained an essential peripheral in virtually every microcontroller produced since. The name itself describes its fundamental nature: it is *universal* (works with many devices), *asynchronous* (requires no shared clock between transmitter and receiver), and provides both *receiver* and *transmitter* functionality in a single hardware block.

In this project, UART serves as the physical communication channel between the MSP430FRxxxx microcontroller and the host Python script running on a PC. The microcontroller transmits sensor data frames containing processed smoke detection results, and receives polling request frames from the PC, all through the UART peripheral (eUSCI_A1) over an RS485 physical layer.

Unlike I²C which uses a clock line shared between master and slave, UART has no clock line between the two communicating devices. Instead, both devices must be independently configured with the same timing parameters before communication begins. This is both the simplicity and the challenge of UART — it needs less wiring but requires careful configuration on both ends.

5.2 UART Fundamentals

5.2.1 Asynchronous Communication

The term *asynchronous* means that there is no shared clock signal between the transmitter and receiver. Each device uses its own internal clock to generate and sample bit timings. Synchronisation is achieved through two mechanisms:

Agreed baud rate: Both devices must be configured with the same baud rate before communication begins. The baud rate defines how many bits per second are transmitted. If the baud rates differ by more than approximately 10%, the receiver will sample bits at incorrect moments and the received data will be corrupted.

Start bit framing: The start bit is the mechanism by which the receiver synchronises to each individual byte. When the transmitter wants to send a byte, it first pulls the line LOW for exactly one bit period. The receiver, which monitors the line continuously, detects this HIGH-to-LOW transition and uses it as the precise moment to start its bit sampling timer. This re-synchronisation at the beginning of every byte means that even if the clocks have drifted slightly between bytes, the start bit corrects the timing for the next byte.

5.2.2 Baud Rate

The baud rate (sometimes called bit rate) is the number of bits transmitted per second. Common baud rates are 9600, 19200, 38400, 57600, 115200, and 921600 bps. In this project, 9600 bps is used.

At 9600 bps, the duration of each bit is:

$$t_{bit} = \frac{1}{9600} \approx 104.17 \mu s \quad (5.1)$$

This means the transmitter holds the TX line at each bit's voltage level for exactly 104.17 μs before changing to the next bit. The receiver samples the line at the same 104.17 μs interval to read each bit.

5.2.3 UART Frame Format (8N1)

A UART transmission is organized as a series of frames. Each frame carries exactly one byte of data. The standard frame format used in this project is 8N1, meaning 8 data bits, No parity bit, and 1 stop bit.

Figure 5.1 shows the complete UART 8N1 frame waveform.

A complete UART 8N1 frame consists of the following elements in order:

Idle state: When no data is being transmitted, the TX line is held HIGH. This is called the *mark* state. The idle line being HIGH is important because it allows the receiver to detect the start bit as a definitive HIGH-to-LOW transition.

Start bit (1 bit, always LOW): The transmitter pulls the TX line LOW for exactly one bit period (104.17 μs at 9600 bps). This transition from HIGH to LOW signals to the receiver that a new byte is about to be transmitted. The receiver detects this falling edge and starts its internal bit-sampling timer from this moment.

Data bits (8 bits): The 8 data bits of the byte are transmitted one by one, LSB (Least

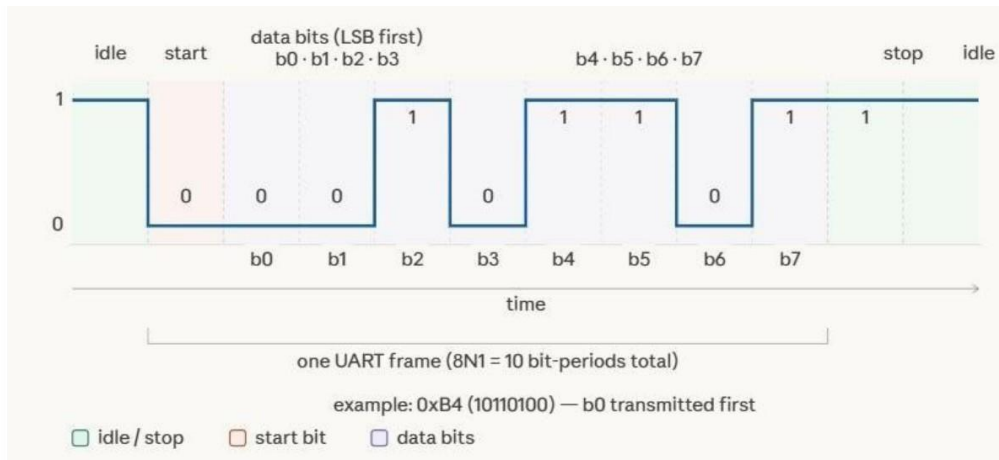


Figure 5.1: UART 8N1 Frame Waveform — Idle, Start Bit, 8 Data Bits (LSB First), Stop Bit, Idle

Figure 5.1: UART 8N1 Frame Waveform — Idle, Start Bit, 8 Data Bits (LSB First), Stop Bit, Idle

Significant Bit) first. Each bit is held on the TX line for exactly one bit period. The TX line is HIGH for a logic 1 bit and LOW for a logic 0 bit. Importantly, the LSB of the byte is sent first, not the MSB. For example, to send the byte 0xC8 (binary 11001000), the bit sequence on the line is: D0 = 0, D1 = 0, D2 = 0, D3 = 1, D4 = 0, D5 = 0, D6 = 1, D7 = 1.

Stop bit (1 bit, always HIGH): After all 8 data bits, the transmitter holds the TX line HIGH for one bit period. This stop bit serves two purposes: it guarantees the line returns to the idle HIGH state after the byte, and it provides a guaranteed HIGH period before any subsequent start bit, ensuring the receiver can always detect the next start bit as a clear HIGH-to-LOW transition.

Total bits per byte: 1 start + 8 data + 1 stop = **10 bits per byte**.

The time to transmit one complete byte at 9600 bps:

$$t_{\text{byte}} = \frac{10}{9600} \approx 1.042 \text{ ms} \quad (5.2)$$

5.2.4 Parity Bit (Not Used in This Project)

A parity bit is an optional 9th bit that can be inserted between the last data bit and the stop bit. It provides a simple single-bit error detection mechanism:

Even parity: The parity bit is set so that the total number of 1-bits in the data byte plus the parity bit is even.

Odd parity: The parity bit is set so that the total number of 1-bits is odd.

If any single bit flips during transmission due to noise, the parity will no longer match and the receiver can detect the error. However, parity cannot detect errors where an even number of bits flip, and it cannot identify which bit was corrupted. In this project, no parity bit is used (N in 8N1) because error detection is handled by the CRC16 field in the custom Frame Layer protocol, which provides far stronger error detection capability.

5.3 RS485 Physical Layer

5.3.1 Why RS485

The UART signal produced by the MSP430FRxxxx is a TTL-level single-ended signal (3.3 V logic HIGH, 0 V logic LOW). While this works for very short connections (a few centimetres on a PCB), it is unsuitable for longer cable runs in industrial environments because:

- Single-ended signals are susceptible to electromagnetic interference (EMI) and ground potential differences between devices.
- The signal voltage drops over long cable runs due to cable resistance.
- Common-mode noise (the same noise appearing on both signal and ground) cannot be rejected.

RS485 (also called TIA-485) solves these problems by using *differential signalling*. Instead of one signal wire referenced to ground, RS485 uses two complementary wires (A and B or D[−] and D⁺). The data is represented by the *voltage difference* between the two wires, not the absolute voltage of either wire relative to ground.

Table 5.1: RS485 vs RS232 vs TTL UART Comparison

Property	TTL UART	RS232	RS485
Signal type	Single-ended	Single-ended	Differential
Logic levels	0 V / 3.3 V	−12 V / +12 V	$\Delta V \geq 200 \text{ mV}$
Max distance	<1 m	15 m	1200 m
Max baud rate	–	1 Mbps	10 Mbps
Nodes on bus	Point-to-point	Point-to-point	Up to 32 (standard)
EMI immunity	Low	Medium	High
Full/Half duplex	Full	Full	Half (typical)

5.3.2 RS485 Differential Signalling

An RS485 driver converts the single-ended TTL UART output from the microcontroller into a differential signal on the A and B lines:

- When UART TX is HIGH (logic 1, idle or stop bit): $B > A$ by at least 200 mV (typically B at +2.5 V, A at -0.5 V)
- When UART TX is LOW (logic 0, start bit or 0 data bit): $A > B$ by at least 200 mV

At the receiver end, an RS485 receiver converts the differential voltage back to a single-ended TTL signal. Because only the *difference* between A and B carries the data, any common-mode noise (which appears equally on both A and B) is cancelled by the differential measurement. This is the fundamental reason for RS485's excellent noise immunity.

5.3.3 Half-Duplex Operation and DE/RE Control

RS485 in this project operates in half-duplex mode using a single twisted pair (A and B). This means only one device can transmit at a time. Direction control is managed by two pins on the RS485 transceiver IC:

DE (Driver Enable): When DE is HIGH, the RS485 driver is enabled and the transceiver transmits the signal on the A/B bus lines. When DE is LOW, the driver output is in high-impedance state (disabled).

RE (Receiver Enable, active LOW): When /RE is LOW, the RS485 receiver is enabled and listens to the A/B bus. When /RE is HIGH, the receiver is disabled.

Figure 5.2 shows the complete DE//RE timing diagram for one polling cycle.

In the firmware, the RS485 direction is controlled using GPIO pins P1.1 (DE) and P1.0 (/RE):

Receive mode (waiting for request):

```
RS485_DE_POUT &= ~RS485_DE_PIN; // DE = LOW
RS485_RE_POUT &= ~RS485_RE_PIN; // /RE = LOW (RX on)
```

Transmit mode (sending response):

```
RS485_DE_POUT |= RS485_DE_PIN; // DE = HIGH (TX on)
RS485_RE_POUT |= RS485_RE_PIN; // /RE = HIGH (RX off)
```

After the transmission is complete (both UCTXIFG and UCBUSY clear), the firmware restores receive mode.

The UART RX buffer is then flushed by clearing the request flags.

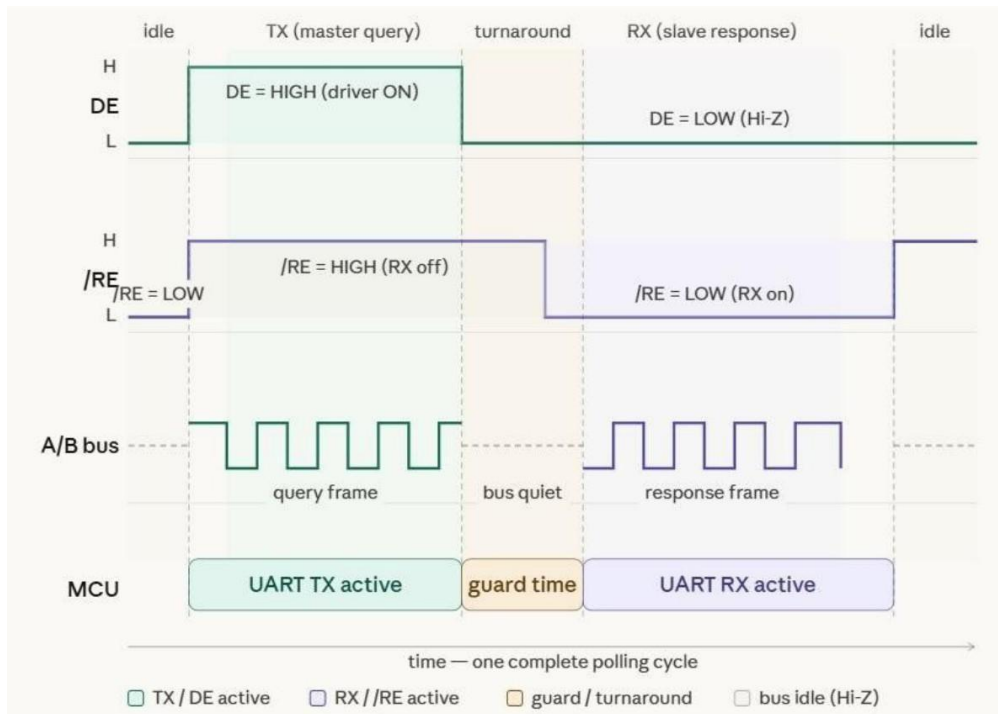


Figure 5.2: RS485 Half-Duplex DE//RE Control Timing for One Complete Polling Cycle

Figure 5.2: RS485 Half-Duplex DE//RE Control Timing for One Complete Polling Cycle

5.4 Frame Layer Protocol Design

5.4.1 What is a Frame Layer?

A Frame Layer (also called a framing protocol or application-level protocol) is a structured format for organising raw bytes into meaningful messages. Without a frame layer, a UART connection simply passes bytes — there is no way for the receiver to know where one message ends and the next begins, or whether the data it received is intended for it, or whether any errors occurred during transmission.

A well-designed frame layer solves four problems:

1. **Addressing:** Which device should respond to this message?
2. **Message delimitation:** How does the receiver know how many bytes form a complete message?
3. **Data integrity:** How does the receiver verify the message was not corrupted during transmission?
4. **Command identification:** What action should the receiver take upon receiving the mes-

sage?

The Frame Layer protocol designed in this project addresses all four of these requirements through the structure of the request frame and response frame.

5.4.2 Request Frame — 4 Bytes

The request frame is sent from the Python host script to the MSP430FRxxxx to request a sensor data reading. It contains exactly 4 bytes:

Table 5.2: Request Frame Field Description

Byte	Field	Value	Purpose
0	Device ID	0x64	Identifies the target device. The MSP430FRxxxx only responds if this matches the Optical Smoke Sensor I2C address 0x64, ensuring the request is addressed to the correct sensor node.
1	Request Command	0xFF	Specifies the command type. 0xFF means “send me the latest sensor data frame”. Only this specific value triggers a response.
2	CRC Low Byte	Computed	The low byte (LSB) of the CRC16 computed over bytes 0 and 1 only.
3	CRC High Byte	Computed	The high byte (MSB) of the CRC16.

Why the CRC is little-endian in the request frame: The CRC in the request frame is stored in little-endian order (low byte first) because this is the native byte order of the MSP430 hardware CRC module output. The Python script constructs the CRC as:

```
crc_low = request_crc & 0xFF
crc_high = (request_crc >> 8) & 0xFF
frame = bytes([DEVICE_ID, REQUEST_CMD,
               crc_low, crc_high])
```

Request frame validation in firmware: The MSP430FRxxxx validates the request by checking all three conditions:

```
if ((device_id == SENSOR_DEVICE_ID) && // 0x64
    (request == SENSOR_REQUEST) && // 0xFF
```

```

{
}

else

{
} (received_crc == calculated_crc)) // CRC match

// Process and respond
// Silent discard    no responseA request that fails any of these three checks is silently discarded. The host will
time out waiting for a response and can retry.

```

5.4.3 Response Frame — 21 Bytes

The response frame is transmitted by the MSP430FRxxxx upon receiving a valid request. It carries the complete sensor measurement result for that polling cycle. The response frame uses Big Endian byte order (MSB first) for all multi-byte fields.

Why Big Endian for the response frame: The response frame uses Big Endian byte order because the MSP430FRxxxx firmware explicitly constructs it using the `put_u32_be()` and `put_u16_be()` utility functions that place the MSB first. This is the conventional network byte order and makes manual inspection of raw HEX output intuitive since the most significant byte of each value appears first.

Frame count: The rolling frame counter provides a simple mechanism for the host to detect if any frames were missed. If the received frame count is not exactly one more than the previous frame count, the host knows a frame was dropped. In this project the host polls every 1 second so missed frames would indicate a communication issue.

Obscuration scaling: The obscuration values are multiplied by 100 before packing into the 16-bit fields. This allows two decimal places of precision (e.g., obscuration 2.35 is stored as 235) to be stored as an integer without floating-point in the frame, keeping the response compact.

5.5 CRC16 Error Detection

5.5.1 Why CRC16

The CRC (Cyclic Redundancy Check) is the error detection mechanism of the frame layer. It provides a mathematical fingerprint of the data payload that allows the receiver to verify the data was not corrupted during transmission.

A simple approach to error detection would be a checksum (sum all bytes, keep only the low byte). However, a checksum cannot detect many common error patterns such as transposed bytes or multiple bits changing in ways that cancel each other out. CRC16 uses polynomial division mathematics that detects:

- All single-bit errors
- All double-bit errors (in messages up to 32,767 bits)
- All odd numbers of bit errors
- All burst errors of length 16 or fewer
- 99.997% of all 17-bit burst errors

5.5.2 CRC-CCITT Polynomial

The CRC-CCITT uses the generator polynomial:

$$G(x) = x^{16} + x^{12} + x^5 + 1 = 0x1021 \quad (5.3)$$

The initial value is 0xFFFF. The CRC is computed by treating the data bytes as the coefficients of a polynomial and performing modulo-2 division by $G(x)$. The remainder of this division is the 16-bit CRC.

5.5.3 Hardware CRC on MSP430FRxxxx

The MSP430FRxxxx hardware CRC module implements this algorithm in hardware. The firmware function:

```
uint16_t crc16_hw_bytes(uint8_t *data, uint16_t len)
{
    uint16_t i;
    CRCINIRES = 0xFFFF;           // Init with 0xFFFF
```

```

    for (i = 0; i < len; i++)
        CRCDIRB_L = data[i];        // Feed each byte
    return CRCINIRES;                // Read result
}

```

For the 19-byte response frame payload, this takes 19 hardware clock cycles ($2.375\ \mu\text{s}$ at 8 MHz).

5.5.4 CRC Verification in Python

The Python host script implements the identical CRC-CCITT algorithm in software:

```

def crc16_msp430(data):
    crc = 0xFFFF
    for byte in data:
        crc ^= (byte << 8)
        for _ in range(8):
            if crc & 0x8000:
                crc = (crc << 1) ^ 0x1021
            else:
                crc <<= 1
        crc &= 0xFFFF
    return crc

```

Validation:

```

def validate_crc(frame):
    data_part    = frame[:-2]          # Bytes 0 to 18
    received_crc = (frame[-2] << 8) | frame[-1] # BE
    calculated   = crc16_msp430(data_part)
    return received_crc == calculated

```


CHAPTER 6

SIGNAL CALCULATIONS AND EFUSE CALIBRATION

6.1 Introduction

The raw 32-bit ADC output values produced by the Optical Smoke Sensor are not directly usable as smoke indicators. They are integer counts that depend on the specific LED current configured, the TIA gain selected, the number of LED pulses accumulated, and the inherent optical and electrical characteristics of each individual sensor module. To obtain a physically meaningful and device-independent quantity, the firmware performs a multi-stage signal processing pipeline that converts these raw ADC counts into the Power Transfer Ratio (PTR) expressed in nW/mW. The PTR is then used to calculate the obscuration level and make the final alarm decision.

This chapter describes every stage of this pipeline in complete detail, including the sensor initialisation sequence, the raw data acquisition mechanism, the LED peak current calculation, the ADC resolution extraction, the photodetector current calculation, the LED efficiency calculation using eFUSE calibration data, the PTR calculation, and the precomputed scale factor mechanism used to reduce computational load on the MSP430FRxxxx.

The signal processing pipeline for one polling cycle is summarised as follows:

1. Sensor initialisation and register configuration (executed once at startup)
2. Raw 32-bit ADC data acquisition from the Optical Smoke Sensor via I2C using the data-hold mechanism
3. LED peak current calculation from register fields (COARSE, FINE, SCALE)
4. ADC resolution extraction based on TIA gain and pulse width
5. Pulse count extraction from register 0x0031
6. Photodetector current (IPD) calculation
7. LED efficiency calculation with eFUSE calibration
8. Gain calibration factor computation using eFUSE gain and intercept coefficients

9. PTR scale factor precomputation (once at startup)
10. Per-cycle PTR computation using the precomputed scale factor
11. Baseline correction
12. Obscuration calculation and alarm decision

6.2 LED Peak Current Calculation

The peak LED drive current is the first quantity that must be determined from the register settings. It is used in both the LED efficiency calculation and the PTR calculation. The Optical Smoke Sensor controls LED current through three independent register fields: COARSE, FINE, and SCALE.

6.2.1 LED Peak Current Formula

The peak LED current $I_{LED,PK}$ in milliamperes is given by the three-component formula from the Optical Smoke Sensor datasheet:

$$I_{LED,PK} = I_{COARSE} \times I_{FINE} \times I_{SCALE} \quad (6.1)$$

where each component is:

$$I_{COARSE} = 50.3 + 19.8 \times I_{LEDx_COARSE} \quad (6.2)$$

$$I_{FINE} = 0.74 + 0.022 \times I_{LEDx_FINE} \quad (6.3)$$

$$I_{SCALE} = 0.1 + 0.9 \times I_{LEDx_SCALE} \quad (6.4)$$

The COARSE field (bits [3:0] of the LED control register) provides the primary coarse setting of the LED current in steps of approximately 19.8 mA. The FINE field (from register 0x0025) provides a high-precision trim within each coarse step in steps of 0.022 mA. The SCALE bit (bit [13] of the LED control register) either delivers the full configured current (SCALE = 1) or reduces it to approximately 10% (SCALE = 0) for low-power testing.

6.2.2 Register Bit Field Extraction

For the Blue LED (LED1), the register fields are extracted from registers 0x0023 and 0x0025 as follows:

- **ILED1_COARSE** = bits [3:0] of Register 0x0023
- **ILED1_SCALE** = bit [13] of Register 0x0023
- **ILED1_FINE** = bits [4:0] of Register 0x0025

For the IR LED (LED3), the fields come from registers 0x0022 and 0x0025:

- **ILED3_COARSE** = bits [3:0] of Register 0x0022
- **ILED3_SCALE** = bit [13] of Register 0x0022
- **ILED3_FINE** = bits [15:11] of Register 0x0025

The firmware implements both calculations in the `calculate_led1_peak_current()` and `calculate_led3_peak_current()` functions:

6.2.3 Worked Example: Blue LED (LED1) Current

The firmware configures Register 0x0023 = 0x3536 and Register 0x0025 = 0x630C.

Step 1: Extract bit fields from Register 0x0023.

$$0x3536 = 0011\ 0101\ 0011\ 0110_2$$

- **ILED1_COARSE** = bits [3:0] = 0110₂ = 6
- **ILED1_SCALE** = bit [13] = 1 (100% current strength)

Step 2: Extract ILED1_FINE from Register 0x0025.

$$0x630C = 0110\ 0011\ 0000\ 1100_2$$

- **ILED1_FINE** = bits [4:0] = 01100₂ = 12

Step 3: Calculate each component.

$$I_{\text{COARSE}} = 50.3 + 19.8 \times 6 = 50.3 + 118.8 = 169.10 \text{mA} \quad (6.5)$$

$$I_{\text{FINE}} = 0.74 + 0.022 \times 12 = 0.74 + 0.264 = 1.004 \text{mA} \quad (6.6)$$

$$I_{\text{SCALE}} = 0.1 + 0.9 \times 1 = 1.000 \quad (6.7)$$

Step 4: Calculate the peak current.

$$I_{\text{LED1,PK}} = 169.10 \times 1.004 \times 1.000 = 169.78 \text{mA} \approx 169.8 \text{mA} \quad (6.8)$$

6.2.4 Worked Example: IR LED (LED3) Current

The firmware configures Register 0x0022 = 0x3539 and Register 0x0025 = 0x630C.

Step 1: Extract bit fields from Register 0x0022.

$$0x3539 = 0011\ 0101\ 0011\ 1001_2$$

- ILED3_COARSE = bits [3:0] = $1001_2 = 9$
- ILED3_SCALE = bit [13] = 1 (100% current strength)

Step 2: Extract ILED3_FINE from Register 0x0025.

$$0x630C = 0110\ 0011\ 0000\ 1100_2$$

- ILED3_FINE = bits [15:11] = $01100_2 = 12$

Step 3: Calculate each component.

$$I_{\text{COARSE}} = 50.3 + 19.8 \times 9 = 50.3 + 178.2 = 228.50 \text{mA} \quad (6.9)$$

$$I_{\text{FINE}} = 0.74 + 0.022 \times 12 = 0.74 + 0.264 = 1.004 \text{mA} \quad (6.10)$$

$$I_{\text{SCALE}} = 0.1 + 0.9 \times 1 = 1.000 \quad (6.11)$$

Step 4: Calculate the peak current.

$$I_{LED3,PK} = 228.50 \times 1.004 \times 1.000 = 229.41 \text{ mA} \approx 229.4 \text{ mA} \quad (6.12)$$

Table 6.1 summarises the LED current calculation results for both channels.

Table 6.1: LED Peak Current Calculation Summary

Parameter	Blue LED (LED1)	IR LED (LED3)
Control register	0x0023 = 0x3536	0x0022 = 0x3539
COARSE field	6	9
SCALE bit	1 (100%)	1 (100%)
FINE field (from 0x0025)	12	12
I_{COARSE}	169.10 mA	228.50 mA
I_{FINE}	1.004 mA	1.004 mA
I_{SCALE}	1.000	1.000
Peak current	169.8 mA	229.4 mA

The IR LED is driven at a higher current than the Blue LED because the IR LED has lower radiant efficiency at 850 nm compared to the Blue LED at 470 nm, and a higher drive current is required to produce sufficient backscattered signal from the smoke chamber.

6.3 Photodetector Current Calculation

The photodetector current (IPD) represents the average photocurrent flowing from the photodetectors during each LED pulse window. It is derived from the raw 32-bit ADC accumulation code using the ADC resolution in nanoamperes per LSB (Q) and the number of LED pulses accumulated.

6.3.1 ADC Resolution (Q)

The ADC resolution Q in nA/LSB depends on the TIA gain setting and the LED pulse width. For a 3 μ s LED pulse width as configured in this project, the values are taken directly from the Optical Smoke Sensor datasheet:

Table 6.2: ADC Resolution Q (nA/LSB) for 3 μ s Pulse Width

TIA Gain	Register	Q (nA/LSB)
200 k Ω	0x0042/0x0044 = 0x1C34	0.41
100 k Ω		0.82
50 k Ω		1.64
25 k Ω		3.27

In this project, both time slots are configured at 200 k Ω TIA gain (Register 0x0042 and 0x0044 both set to 0x1C34), giving:

$$Q = 0.41 \text{ nA/LSB} \quad (6.13)$$

6.3.2 Pulse Count Extraction

The number of LED pulses per time slot is extracted from Register 0x0031 (Slot A) and Register 0x0036 (Slot B). Both are written as 0x400E, giving bits [15:8] = 0x40:

$$N_{\text{pulses}} = (0x400E \gg 8) = 0x40 = 64 \quad (6.14)$$

6.3.3 IPD Formula

The average photodetector current IPD in nanoamperes is calculated from the accumulated ADC code, the pulse count, and the ADC resolution:

$$I_{\text{PD}} = \frac{\text{ADC_CODE} \times Q}{N_{\text{pulses}}} \quad (6.15)$$

where ADC_CODE is the 32-bit accumulated value read from the sensor, Q is the ADC resolution in nA/LSB, and N_{pulses} is the number of LED pulses per time slot. Division by N_{pulses} normalises the accumulated sum back to a single-pulse equivalent current.

6.3.4 Worked Example: IPD Calculation

For a typical clean-air ADC code of, for example, ADC_CODE = 150,000 counts at 200 k Ω TIA gain with 64 pulses:

$$I_{\text{PD}} = \frac{150,000 \times 0.41}{64} = \frac{61,500}{64} = 960.9 \text{ nA} \quad (6.16)$$

6.4 LED Efficiency Calculation

The LED efficiency η in W/A is the radiant optical output power per unit of drive current. It is the conversion factor that relates how much optical power the LED emits for each milliampere of drive current. The efficiency varies from device to device due to manufacturing tolerances, and also varies with the drive current due to LED droop. The Optical Smoke Sensor provides eFUSE-programmed calibration coefficients to account for both sources of variation.

6.4.1 eFUSE Calibration Overview

Each Optical Smoke Sensor module is individually characterised at the factory and a set of calibration coefficients are stored in its internal eFUSE non-volatile memory. These coefficients are read by the firmware once at startup from registers 0x0070 through 0x0074:

- **Register 0x0070** – Module ID (identifies which calibration equation set to use)
- **Register 0x0071** – eFUSE blue gain coefficient (efuse_blue_gain)
- **Register 0x0072** – eFUSE IR gain coefficient (efuse_ir_gain)
- **Register 0x0073** – eFUSE blue intercept coefficient (efuse_blue_intercept)
- **Register 0x0074** – eFUSE IR intercept coefficient (efuse_ir_intercept)

The Module ID selects between two calibration equation sets: Module ID 33 uses one set of nominal coefficients, while Module IDs 30 and 31 use a different set. This is because different hardware revisions of the Optical Smoke Sensor module have different nominal LED characteristics.

6.4.2 Blue LED Efficiency

The blue LED efficiency η_{blue} in W/A is computed in two stages: first a gain calibration factor C is derived from the eFUSE coefficients, and then a current-dependent droop factor k is applied.

Gain calibration factor for the blue LED (Module ID 33):

$$\text{Gain}_{\text{blue}} = \frac{21 \times (\text{efuse_blue_gain} - 112)}{256} + 21 \quad (6.17)$$

$$\text{INTER}_{\text{blue}} = 8 \times (\text{efuse_blue_intercept} - 80) \quad (6.18)$$

$$\text{NOMINAL_SCALAR}_{\text{blue}} = 21 \times I_{\text{LED1,PK}} + 753 \quad (6.19)$$

$$C_{\text{blue}} = \frac{\text{Gain}_{\text{blue}} \times I_{\text{LED1,PK}} + \text{INTER}_{\text{blue}}}{\text{NOMINAL_SCALAR}_{\text{blue}}} \quad (6.20)$$

Current-dependent droop factor for the blue LED:

The blue LED exhibits an efficiency droop at high drive currents due to carrier recombination mechanisms. This is modelled by a third-order polynomial in the LED current:

$$k = 9.8976 \times 10^{-1} - 5.1448 \times 10^{-3} \cdot I_{\text{LED}} + 2.0287 \times 10^{-5} \cdot I_{\text{LED}}^2 - 2.9645 \times 10^{-8} \cdot I_{\text{LED}}^3 \quad (6.21)$$

where I_{LED} is the blue LED peak current in mA. At lower current levels (below ≈ 100 mA), k is close to 1.0. At the configured 169.8 mA, the droop factor k reduces the nominal efficiency slightly.

Final blue LED efficiency:

$$\eta_{\text{blue}} = 0.38 \times k \times C_{\text{blue}} \quad (6.22)$$

The constant 0.38 W/A is the nominal peak radiant efficiency of the Blue LED at its datasheet reference conditions. The product $k \times C_{\text{blue}}$ scales this nominal value to account for the actual drive current and this specific device's manufacturing variation.

6.4.3 IR LED Efficiency

The IR LED efficiency η_{IR} in W/A uses the same eFUSE calibration approach but without a droop polynomial, because the IR LED at 850 nm does not exhibit the same degree of efficiency droop as the blue LED at the currents used in this design.

For Module ID 33:

$$\text{Gain}_{\text{IR}} = \frac{42 \times (\text{efuse_ir_gain} - 112)}{256} + 42 \quad (6.23)$$

$$\text{INTER}_{\text{IR}} = 5 \times (\text{efuse_ir_intercept} - 80) \quad (6.24)$$

$$\text{NOMINAL_SCALAR}_{\text{IR}} = 42 \times I_{\text{LED3,PK}} + 156 \quad (6.25)$$

$$C_{\text{IR}} = \frac{\text{Gain}_{\text{IR}} \times I_{\text{LED3,PK}} + \text{INTER}_{\text{IR}}}{\text{NOMINAL_SCALAR}_{\text{IR}}} \quad (6.26)$$

Final IR LED efficiency:

$$\eta_{\text{IR}} = 0.22 \times C_{\text{IR}} \quad (6.27)$$

The constant 0.22 W/A is the nominal radiant efficiency of the IR LED at its datasheet reference conditions.

6.5 Power Transfer Ratio Calculation

The Power Transfer Ratio (PTR) is the central figure of merit for smoke detection in the Optical Smoke Sensor system. It is defined as the ratio of optical power received by the photodetector to the optical power emitted by the LED, expressed in units of nW/mW. The PTR is a device-independent quantity because the LED current, the LED efficiency, and the photodetector responsivity are all explicitly accounted for in the calculation.

6.5.1 PTR Formula

The PTR in nW/mW is given by:

$$\text{PTR} = \frac{I_{\text{PD}}}{I_{\text{LED,PK}}} \times \frac{1}{R_{\text{PD}}} \times \frac{1}{\eta_{\text{LED}}} \quad (6.28)$$

where:

- I_{PD} is the photodetector current in nA
- $I_{\text{LED,PK}}$ is the LED peak current in mA
- R_{PD} is the photodetector spectral responsivity in A/W (0.26 A/W for blue at 470 nm, 0.41 A/W for IR at 850 nm)
- η_{LED} is the LED radiant efficiency in W/A (calculated from eFUSE coefficients)

The derivation of this formula can be understood from first principles. The optical power emitted by the LED is: $P_{\text{opt,LED}} = I_{\text{LED,PK}} \times \eta_{\text{LED}}$. The optical power received by the photodetector is: $P_{\text{opt,PD}} = I_{\text{PD}}/R_{\text{PD}}$. The PTR is the ratio $P_{\text{opt,PD}}/P_{\text{opt,LED}}$, which simplifies to Equation 6.28.

6.5.2 Worked Example: PTR Calculation

Continuing the worked example from Section 6.4 with $\text{ADC_CODE} = 150,000$, $Q = 0.41 \text{ nA/LSB}$, $N_{\text{pulses}} = 64$, and assuming example eFUSE values that give $\eta_{\text{blue}} = 0.36 \text{ W/A}$ for a nominal module:

$$I_{PD} = \frac{150,000 \times 0.41}{64} = 960.9 \text{ nA} \quad (6.29)$$

$$\text{PTR}_{\text{blue}} = \frac{960.9}{169.8} \times \frac{1}{0.26} \times \frac{1}{0.36} \quad (6.30)$$

$$= 5.659 \times 3.846 \times 2.778 \quad (6.31)$$

$$= 60.5 \text{ nW/mW} \quad (6.32)$$

This value represents the fraction of the emitted blue light that was scattered back onto the photodetector. In clean air, a typical PTR value lies in the range of 50 to 150 nW/mW depending on the chamber background scattering. When smoke is introduced, the PTR rises above the clean-air baseline value.

6.6 eFUSE Gain Calibration

6.6.1 Purpose of eFUSE Calibration

The eFUSE calibration addresses an important practical problem in mass production of optical sensors. Although every Optical Smoke Sensor module is manufactured to the same specification, there are small device-to-device variations in the LED radiant efficiency and the photodetector sensitivity that cause the raw ADC output to differ between individual modules for the same amount of smoke. Without calibration, a single alarm threshold would need to be set conservatively wide to account for all devices, reducing sensitivity at one end and increasing false alarm rate at the other.

The eFUSE calibration eliminates this variation by measuring the actual optical sensitivity of each module individually during factory testing, and storing a pair of gain and intercept coefficients in the internal eFUSE memory. The firmware reads these coefficients at startup and uses them to normalise the ADC output to a canonical reference value that is independent of the specific device's optical characteristics. This allows a single precise alarm threshold to be used across all units.

6.6.2 Module ID 33

For a module with Module ID = 33, the device-specific gain and intercept for the blue channel are computed from the eFUSE values as:

$$\text{Gain}_{\text{blue}} = \frac{21}{256} \times (\text{efuse_blue_gain} - 112) + 21 \quad (6.33)$$

$$\text{INTER}_{\text{blue}} = 8 \times (\text{efuse_blue_intercept} - 80) \quad (6.34)$$

The nominal scalar for Module ID 33 is:

$$\text{Nominal} = 21 \times I_{\text{LED1,PK}} + 753$$

The eFUSE values are centred around 112 for gain and 80 for intercept. A device with exactly nominal optical characteristics will have $\text{efuse_blue_gain} = 112$ and $\text{efuse_blue_intercept} = 80$, giving $G_{\text{cal}} = 1.000$, which means no correction is applied. Devices with above-nominal sensitivity will have eFUSE gain values above 112, and the correction factor will be greater than 1.0, causing the ADC code to be divided by a larger number and scaling the result down to the nominal reference. Devices with below-nominal sensitivity will have eFUSE values below 112 and the correction scales the result up.

6.6.3 Module IDs 30 and 31

For modules with Module ID 30 or 31, the same structure applies but with different nominal coefficients reflecting the different hardware revision:

$$\text{Gain}_{\text{blue}} = \frac{17}{256} \times (\text{efuse_blue_gain} - 112) + 17 \quad (6.35)$$

$$\text{INTER}_{\text{blue}} = 8 \times (\text{efuse_blue_intercept} - 128) \quad (6.36)$$

The nominal scalar for Module IDs 30 and 31 is:

$$\text{Nominal} = 17 \times I_{\text{LED1,PK}} + 622$$

Note that the intercept reference changes from 80 (Module ID 33) to 128 (Module IDs 30 and 31), reflecting the different centering of the eFUSE values for the two hardware revisions. For the IR channel, the structure is the same with different scaling constants as shown in Table ???. The same module-ID selection logic applies to both channels — the firmware selects

the full set of blue and IR coefficients together based on the single Module ID value read from register 0x0070.

Table 6.3 summarises the complete eFUSE calibration equation structure for all supported module variants.

Table 6.3: eFUSE Calibration Equation Parameters by Channel and Module ID

Module ID	Channel	Gain slope	INTER slope	eFUSE centre
33	Blue	21/256	8	112 / 80
33	IR	42/256	5	112 / 80
30, 31	Blue	17/256	8	112 / 128
30, 31	IR	34/256	5	112 / 128

CHAPTER 7

ALGORITHMS

7.1 Introduction

The PTR values computed from the raw ADC codes by the signal processing pipeline of Chapter 6 are not immediately used for alarm decisions. Before a physically meaningful obscuration level can be computed, two important post-processing steps must be applied: a moving average filter to reduce sample-to-sample noise, and a baseline correction to remove the clean-air background signal. Only after these two steps does the corrected PTR represent the smoke-only contribution to the backscattered light.

This chapter describes the complete post-processing chain, from raw PTR values through noise filtering, baseline subtraction, obscuration calculation, adaptive baseline tracking, and the final alarm decision. The entire pipeline is summarised in Figure 7.1 and each stage is explained in detail in the sections that follow.

explained in detail in the sections that follow.

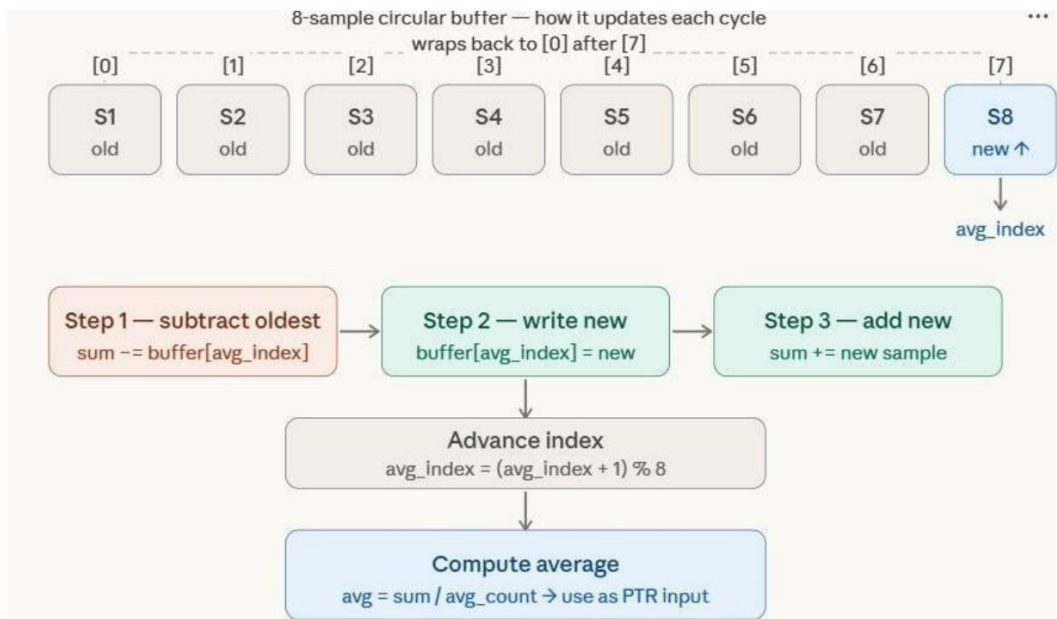


Figure 7.1: Post-Processing Pipeline: PTR (raw) to Alarm Decision

Figure 7.1: Post-Processing Pipeline: PTR (raw) to Alarm Decision

7.2 Moving Average Filter

7.2.1 Why Filtering is Necessary

The ADC output of the Optical Smoke Sensor, even after the internal 128-sample block averaging performed by the sensor hardware, still contains a residual sample-to-sample variation. This variation arises from several sources: thermal noise in the TIA amplifier, quantisation noise from the ADC conversion, small variations in the timing of the LED pulses, and mechanical vibrations in the smoke chamber that momentarily change the backscatter geometry. In a smoke detection application, this random noise can cause the computed PTR to fluctuate above and below its true mean value, which in turn causes the computed obscuration to fluctuate. Without additional filtering, a noise spike in the PTR could briefly exceed the alarm threshold and trigger a false alarm even in perfectly clean air.

A low-pass filter applied in firmware after the hardware averaging further reduces this noise. The firmware implements an 8-sample moving average (also called a sliding window average or boxcar filter) to smooth the raw ADC output before PTR computation.

7.2.2 The Moving Average Concept

A moving average filter maintains a window of the N most recent samples and outputs the arithmetic mean of those N samples at each step. As a new sample arrives, it enters the window and the oldest sample is discarded. The output at step n is:

$$\bar{x}[n] = \frac{1}{N} \sum_{k=0}^{N-1} x[n-k] \quad (7.1)$$

where N is the window size (8 in this project) and $x[n-k]$ are the N most recent samples.

Figure 7.2 illustrates how the moving average window slides through a noisy signal. At each step, the oldest sample leaves the window from the left and the newest sample enters from the right. The output is the average of all samples currently inside the window.

7.2.3 Noise Reduction Property

The moving average filter is a finite impulse response (FIR) low-pass filter. For a window of size N , the SNR improvement compared to using raw samples is:

$$\text{SNR improvement} = \sqrt{N} \quad (7.2)$$

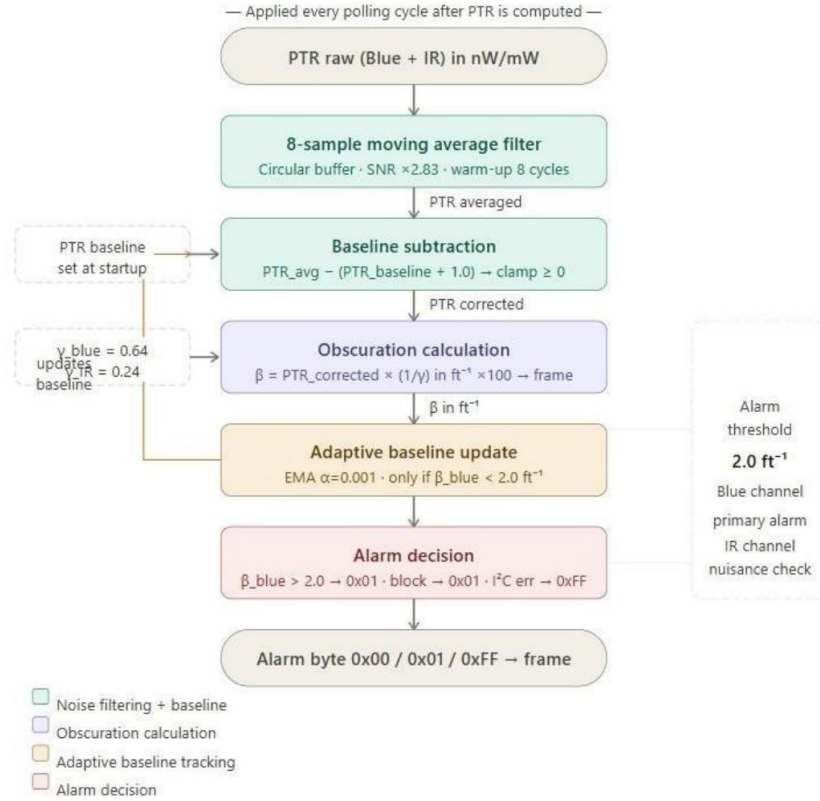


Figure 7.2: Moving Average Window (N=8): New sample enters from right, oldest leaves from left, output is the mean of the window contents

Figure 7.2: Moving Average Window (N=8): New sample enters from right, oldest leaves from left, output is the mean of the window contents

With $N = 8$:

$$\text{SNR improvement} = \frac{\sqrt{N}}{1} \approx 2.83 \times \quad (7.3)$$

This means the noise standard deviation of the averaged output is $\frac{1}{\sqrt{N}} \approx 0.35$ times smaller than the noise of individual samples. Combined with the 128-sample internal hardware averaging already performed by the Optical Smoke Sensor, the total averaging depth is $8 \times 128 = 1024$ samples per output, giving a total SNR improvement of $\frac{\sqrt{1024}}{1} = 32 \times$ relative to a single ADC sample.

7.2.4 Circular Buffer Implementation

The 8-sample moving average is implemented in firmware using a circular buffer — a fixed-size array in which the oldest entry is overwritten by the newest sample at each step. The circular buffer avoids shifting all elements in the array on every update, keeping the implementation to a constant number of operations regardless of the window size.

Figure 7.3 shows the circular buffer structure for the Blue channel. An identical buffer exists for the IR channel.

exists for the IR channel.

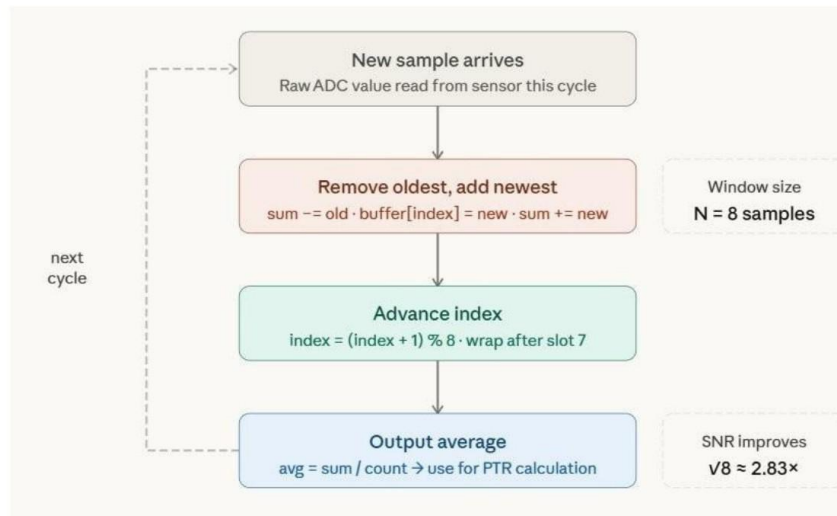


Figure 7.3: Circular Buffer Structure for 8-Sample Moving Average: avg_index advances one position each cycle, wrapping from position 7 back to position 0

Figure 7.3: Circular Buffer Structure for 8-Sample Moving Average: avg_index advances one position each cycle, wrapping from position 7 back to position 0

The firmware maintains four variables for the moving average: the circular buffer array `blue_buffer[8]`, the running sum `blue_sum`, a write index `avg_index`, and a fill counter `avg_count` that tracks how many valid samples are in the buffer (relevant only during the first 8 cycles after startup).

The update sequence at each polling cycle is:

1. Subtract the oldest sample (the entry at `avg_index`) from the running sum: `blue_sum -= blue_buffer[avg_index]`
2. Write the new sample into the buffer at the same position: `blue_buffer[avg_index] = blue_raw`
3. Add the new sample to the running sum: `blue_sum += blue_raw`
4. Advance the index, wrapping around to 0 after position 7: `avg_index = (avg_index + 1) % 8`
5. Increment the fill counter up to a maximum of 8: `avg_count = min(avg_count+1, 8)`

6. Compute the average: `blue_avg = blue_sum / avg_count`

The division by `avg_count` rather than always by 8 handles the start-up transient correctly. During the first 8 polling cycles, there are fewer than 8 valid samples in the buffer, so dividing by the actual number of valid samples gives the correct mean rather than a value biased toward zero by the empty initial buffer entries.

7.2.5 Effect of Moving Average on the PTR Signal

Figure 7.4 shows the effect of the 8-sample moving average on a typical PTR time series. The raw PTR (computed directly from individual ADC readings) exhibits noticeable sample-to-sample variation even in clean air. The 8-sample average produces a much smoother signal that accurately tracks the slow-varying true PTR while suppressing the high-frequency noise.

Figure 7.4: Effect of 8-Sample Moving Average on PTR: Raw PTR (grey) shows sample-to-sample noise; filtered output (blue) tracks the true mean while suppressing noise peaks

The trade-off introduced by the moving average is a response lag of approximately $N/2 = 4$ samples. When smoke suddenly enters the chamber, the output of the filter takes approximately 4 sample periods to rise to half the step change, and approximately $N = 8$ sample periods to fully reflect the new steady-state PTR. At the sensor's output data rate of 1.5625 Hz (one new averaged output every 640 ms), an 8-sample lag represents approximately 5 seconds. This is acceptable for smoke detection, where the rate of change of obscuration from a developing fire is much slower than 5 seconds.

7.3 Baseline Calibration

7.3.1 What is the Baseline?

When the Optical Smoke Sensor is mounted inside a smoke chamber in clean air (no smoke present), the photodetectors still receive a measurable amount of light. This background light has two sources:

- **Chamber wall scattering:** The LED light reflects off the inner walls of the smoke chamber and a fraction reaches the photodetectors. This is a constant background that depends on the chamber geometry and the surface reflectivity of the chamber walls.
- **PCB surface scattering:** The LED light scatters off the PCB surface and other components near the sensor package before entering the photodetector optical path.

As a result, the PTR measured in clean air is not zero — it has a non-zero *baseline* value PTR_{baseline} that represents the clean-air background. The total measured PTR at any time is:

$$PTR_{\text{total}} = PTR_{\text{baseline}} + PTR_{\text{smoke}} \quad (7.4)$$

The smoke-only PTR that is needed for obscuration calculation is:

$$PTR_{\text{smoke}} = PTR_{\text{total}} - PTR_{\text{baseline}} \quad (7.5)$$

Figure 7.5 illustrates the relationship between the clean-air baseline, the smoke contribution, and the total measured PTR over time.

tion, and the total measured PTR over time.

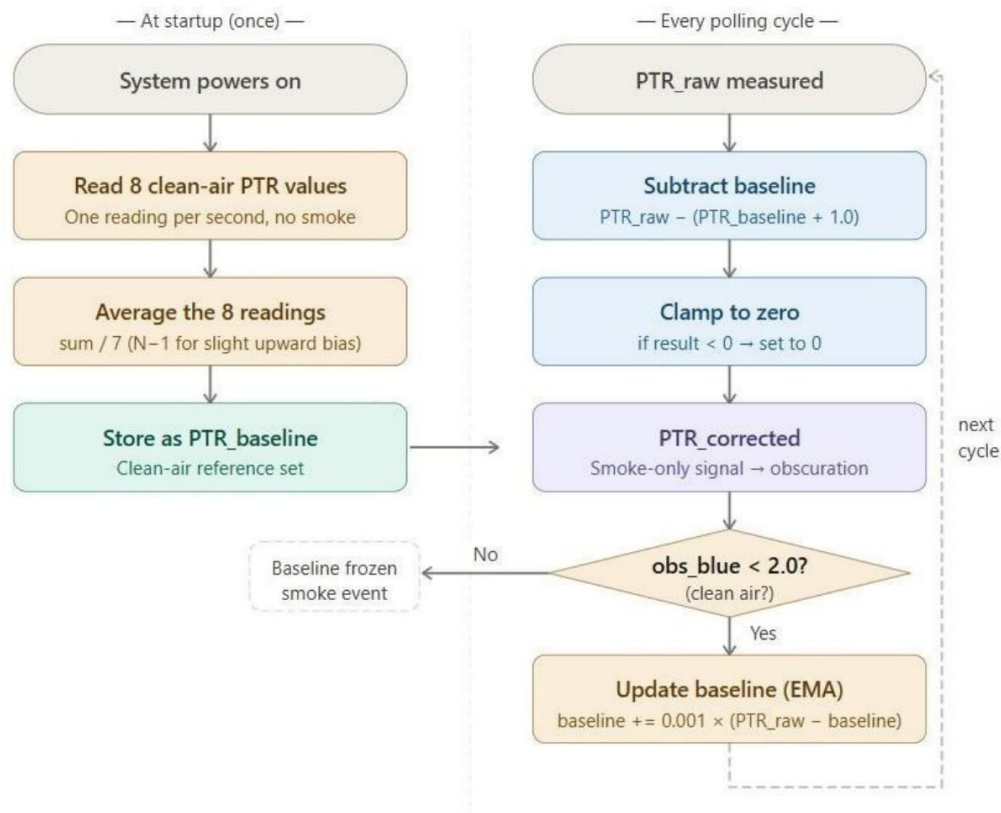


Figure 7.5: Baseline Concept: PTR_{total} is the sum of the chamber background (PTR_{baseline}) and the smoke signal (PTR_{smoke}). Only PTR_{smoke} is used for obscuration calculation.

Figure 7.5: Baseline Concept: PTR_{total} is the sum of the chamber background (PTR_{baseline}) and the smoke signal (PTR_{smoke}). Only PTR_{smoke} is used for obscuration calculation.

7.3.2 Startup Baseline Measurement

At system startup, before the main polling loop begins, the firmware executes a dedicated baseline measurement procedure by calling `ptr_baseline()`. This function performs the complete signal processing calculation (LED current, IPD, LED efficiency, gain calibration, PTR) for each of 8 consecutive clean-air readings taken with approximately 1-second intervals between them.

The baseline is computed as the average of these 8 readings, divided by `BASELINE_DIVIDER = 7` (one less than the number of samples). This deliberate use of $N - 1$ in the denominator rather than N gives the baseline a slight upward bias compared to a true mean. The effect is that the baseline sits slightly below the measured clean-air PTR rather than at its exact centre, ensuring that in clean air the corrected PTR ($\text{PTR}_{\text{raw}} - \text{PTR}_{\text{baseline}}$) remains a small positive value rather than fluctuating around zero. This prevents the corrected PTR from going negative due to noise, which would require the firmware to clamp it to zero on every cycle.

The resulting baseline values are stored in the global floating-point variables `ptr_blue_baseline` and `ptr_ir_baseline`, where they remain until updated by the adaptive baseline tracker (Section 7.4).

7.3.3 The +1 Guard in Baseline Subtraction

The firmware applies the baseline correction with an additional +1.0 guard offset:

$$\text{PTR}_{\text{corrected}} = \text{PTR}_{\text{raw}} - (\text{PTR}_{\text{baseline}} + 1.0) \quad (7.6)$$

The +1.0 is a deliberate engineering margin. It ensures that the corrected PTR is guaranteed to be negative (and therefore clamped to zero) when the raw PTR equals the baseline exactly. Without this guard, random noise could cause the corrected PTR to occasionally produce a small positive value even when no smoke is present, which would result in a small non-zero obscuration reading in perfectly clean air. The +1.0 absorbs this residual noise floor, so the displayed obscuration in clean air is reliably zero rather than noisy near zero. If the corrected PTR is negative after the subtraction, the firmware clamps it to zero:

$$\text{PTR}_{\text{corrected}} = \max(\text{PTR}_{\text{corrected}}, 0) \quad (7.7)$$

7.4 Adaptive Baseline Tracking

7.4.1 Why the Baseline Must Track Over Time

The clean-air baseline PTR is not a fixed constant. It drifts slowly over the lifetime of the sensor due to several mechanisms:

- **Temperature variation:** The optical efficiency of the LED and the responsivity of the photodetector both vary with temperature. Variations in ambient temperature cause the clean-air PTR to shift slowly over hours and days.
- **Gradual contamination:** Dust particles and other non-fire aerosols can slowly accumulate on the chamber walls and on the sensor package surface, gradually increasing the background scattering signal.
- **LED aging:** The LED radiant efficiency decreases very slowly over its operational lifetime (tens of thousands of hours), which causes the clean-air PTR to decrease gradually over months.

If the baseline were fixed at its startup value and never updated, a gradual upward drift in the clean-air PTR would cause the corrected PTR to slowly increase over time even with no smoke present, eventually triggering a false alarm. Adaptive baseline tracking prevents this by continuously adjusting the stored baseline to follow the long-term clean-air level.

7.4.2 Exponential Moving Average Tracker

The adaptive baseline tracker uses a first-order exponential moving average (EMA), also known as an exponential smoothing filter. The update equation is:

$$\text{PTR}_{\text{baseline}}[n] = \text{PTR}_{\text{baseline}}[n - 1] + \alpha (\text{PTR}_{\text{raw}}[n] - \text{PTR}_{\text{baseline}}[n - 1]) \quad (7.8)$$

where α is the smoothing factor (BASELINE_ALPHA = 0.001 in this firmware).

This equation has an elegant interpretation. The term $(\text{PTR}_{\text{raw}}[n] - \text{PTR}_{\text{baseline}}[n - 1])$ is the error between the current measurement and the current baseline estimate. The baseline is nudged toward the current measurement by a fraction α of this error at each step. With $\alpha = 0.001$, only 0.1% of the error is corrected per cycle. This makes the tracker extremely slow to respond — it takes approximately $1/\alpha = 1000$ cycles to close 63% of a step change

— which is exactly the desired behaviour. The tracker needs to be slow enough that it tracks long-term drift but does not follow a smoke event.

it shows the difference in time response between a fast tracker ($\alpha = 0.1$) and the slow tracker ($\alpha = 0.001$) used in this firmware. The fast tracker would follow a smoke event and raise the baseline during the alarm, completely masking the smoke signal. The slow tracker ignores the smoke event entirely because the event duration (tens of seconds to minutes) is far shorter than the tracker time constant (approximately 640 seconds at 1.5625 Hz output data rate).

7.4.3 Conditional Update Logic

A critical design decision in the adaptive tracker is the condition under which the baseline is updated. If the baseline is updated unconditionally on every cycle, then during a smoke event the baseline would rise toward the smoke-elevated PTR, reducing the corrected PTR and potentially cancelling the alarm. To prevent this, the firmware only updates the baseline when the measured obscuration is below the alarm threshold:

$$\text{Update baseline only if: } \text{obscuratio}_{\text{blue}} < 2.0 \text{ ft}^{-1} \quad (7.9)$$

When smoke raises the obscuration above 2.0 ft^{-1} , the baseline update is suspended. The baseline remains frozen at its last clean-air value. This ensures that the smoke signal continues to be measured relative to the pre-smoke clean-air baseline for the entire duration of the alarm event, maintaining full alarm sensitivity.

Figure 7.6 illustrates this gated update mechanism. During the smoke event, the gate is closed and the baseline is held constant. When the smoke clears and the obscuration falls back below threshold, the gate reopens and the baseline resumes tracking.

7.4.4 Tracker Time Constant

The time constant τ of the exponential moving average tracker can be expressed in terms of the number of samples required to close 63% of a step change:

$$\tau_{\text{samples}} = \frac{1}{\alpha} = \frac{1}{0.001} = 1000 \text{ samples} \quad (7.10)$$

At the output data rate of 1.5625 Hz:

$$\tau_{\text{time}} = \frac{1000}{1.5625} = 640 \text{ seconds} \approx 10.7 \text{ minutes} \quad (7.11)$$

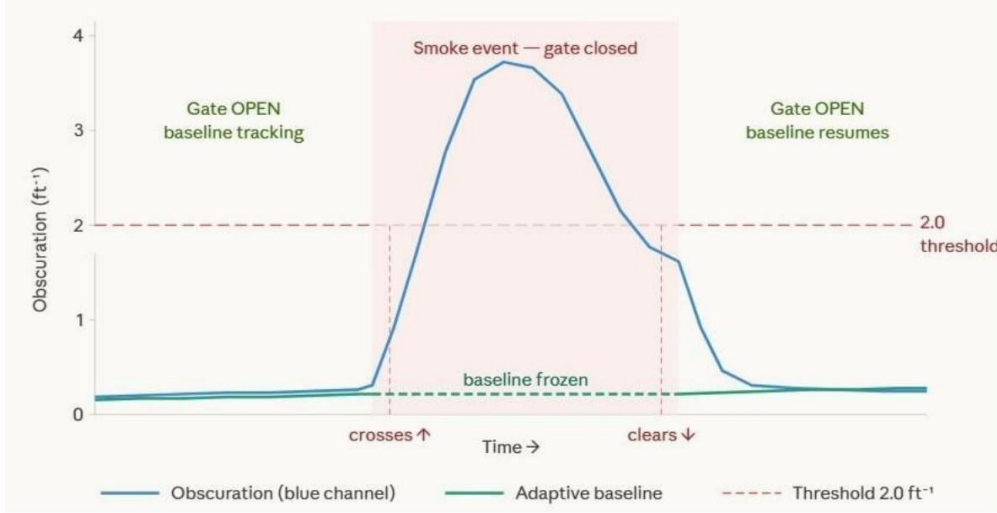


Figure 7.6: Conditional Baseline Update Gate: Baseline tracking is suspended when obscuration exceeds the threshold (shaded region), preventing the baseline from rising during a smoke event

Figure 7.6: Conditional Baseline Update Gate: Baseline tracking is suspended when obscuration exceeds the threshold (shaded region), preventing the baseline from rising during a smoke event

This means the baseline tracker takes approximately 11 minutes to follow 63% of a step change in the clean-air PTR. For the types of slow drift described in Section 7.4.1 (temperature, contamination, LED aging), this time constant is appropriate because these changes occur over hours, days, and months — far slower than the tracker time constant.

7.5 Obscuration Calculation

7.5.1 From PTR to Obscuration

The corrected PTR $PTR_{corrected}$ represents only the smoke contribution to the backscattered optical signal. This is converted to an obscuration level β in ft^{-1} by dividing by the wavelength-dependent sensitivity coefficient γ :

$$\beta = \frac{PTR_{corrected}}{\gamma} \quad (7.12)$$

The γ coefficients are defined by the Optical Smoke Sensor application note and reflect how sensitive each wavelength channel is to the type of smoke particles present in the chamber:

- $\gamma_{blue} = 0.64 \text{ nW/mW per } ft^{-1}$ for the 470 nm Blue LED channel
- $\gamma_{IR} = 0.24 \text{ nW/mW per } ft^{-1}$ for the 850 nm IR LED channel

The firmware avoids the division by computing the reciprocals once as preprocessor constants:

$$1/\gamma_{\text{blue}} = 1/0.64 = 1.5625 \quad (7.13)$$

$$1/\gamma_{\text{IR}} = 1/0.24 \approx 4.1667 \quad (7.14)$$

The obscuration is then computed with a single multiply:

$$\beta_{\text{blue}} = \text{PTR}_{\text{corrected,blue}} \times 1.5625 \quad (7.15)$$

$$\beta_{\text{IR}} = \text{PTR}_{\text{corrected,IR}} \times 4.1667 \quad (7.16)$$

7.5.2 Physical Meaning of Obscuration

Obscuration in ft^{-1} quantifies the attenuation of a light beam per unit path length through the smoke-laden air. A value of 1 ft^{-1} means that light intensity is reduced by a factor of e^{-1} ($\approx 36.8\%$) per foot of path length. A value of 5 ft^{-1} means a 99.3% intensity reduction per foot. In the context of this system, obscuration is computed from backscattering rather than from transmission, so the absolute numerical values are calibrated specifically for the Optical Smoke Sensor chamber geometry and the γ coefficients provided by the application note.

7.5.3 Obscuration Scaling for Transmission

The computed obscuration values are multiplied by 100 before storing them in the response frame as 16-bit unsigned integers:

$$\text{obs_blue} = \lfloor \beta_{\text{blue}} \times 100 \rfloor \quad (7.17)$$

This scaling preserves two decimal places of precision in the integer representation. For example, an obscuration of 1.56 ft^{-1} is stored as the integer 156. The Python host script divides by 100 when displaying the value:

$$\text{Displayed} = \frac{\text{obs_blue}}{100} = \frac{156}{100} = 1.56 \text{ ft}^{-1} \quad (7.18)$$

This approach keeps the frame compact (2 bytes per channel) while providing sufficient

resolution for the alarm threshold comparison.

7.6 Alarm Decision Logic

7.6.1 Primary Threshold: Obscuration

The primary alarm condition is based on the blue channel obscuration value. The alarm is triggered when the corrected blue obscuration exceeds the defined threshold:

$$\text{alarm} = \begin{cases} 0x01 & \text{if } \beta_{\text{blue}} > 2.0 \text{ ft}^{-1} \\ 0x00 & \text{otherwise} \end{cases} \quad (7.19)$$

The blue channel is used as the primary alarm indicator rather than the IR channel because the 470 nm blue LED is more sensitive to the fine sub-micron soot particles characteristic of early-stage fire events. The IR channel at 850 nm is more sensitive to larger nuisance particles such as dust and steam. Using the blue channel for the alarm threshold therefore provides better true alarm sensitivity while maintaining nuisance rejection.

7.6.2 Secondary Threshold: Total Optical Blocking

In addition to the obscuration-based alarm, the firmware implements a secondary hard-signal-loss alarm condition. If the raw ADC codes for both channels simultaneously fall below the minimum signal threshold of 5000 counts, the firmware sets the alarm immediately without computing any PTR or obscuration:

$$\text{alarm} = 0x01 \quad \text{if } \text{blue_raw} < 5000 \text{ AND } \text{ir_raw} < 5000 \quad (7.20)$$

This condition represents total optical blocking of the smoke chamber, which occurs in the case of extremely dense smoke that completely attenuates the LED signal. In this scenario the backscattering calculation would produce a misleadingly low PTR (because no scattered light is reaching the detector) while the chamber is actually in an extreme smoke condition. The hard-signal-loss check catches this edge case and ensures the alarm is triggered even when the signal is too low for the backscattering calculation to be meaningful.

7.6.3 I2C Error Alarm

A third alarm condition exists for I2C communication failures. When the I2C bus recovery mechanism triggers during a polling cycle (Section 4.8 of Chapter 4), the alarm byte in the

response frame is set to 0xFF to signal to the host that the data in the frame is unreliable:

$$\text{alarm} = 0xFF \text{ if I}^2\text{C recovery was triggered this cycle} \quad (7.21)$$

The Python host script treats 0xFF differently from 0x01 and displays a communication fault indicator rather than a smoke alarm.

7.6.4 Complete Alarm Decision Tree

Figure 7.7 shows the complete alarm decision tree implemented in the firmware for each polling cycle. The three conditions are evaluated in priority order: I²C recovery takes the highest priority, followed by the total-blocking check, and finally the obscuration threshold comparison.

comparison.

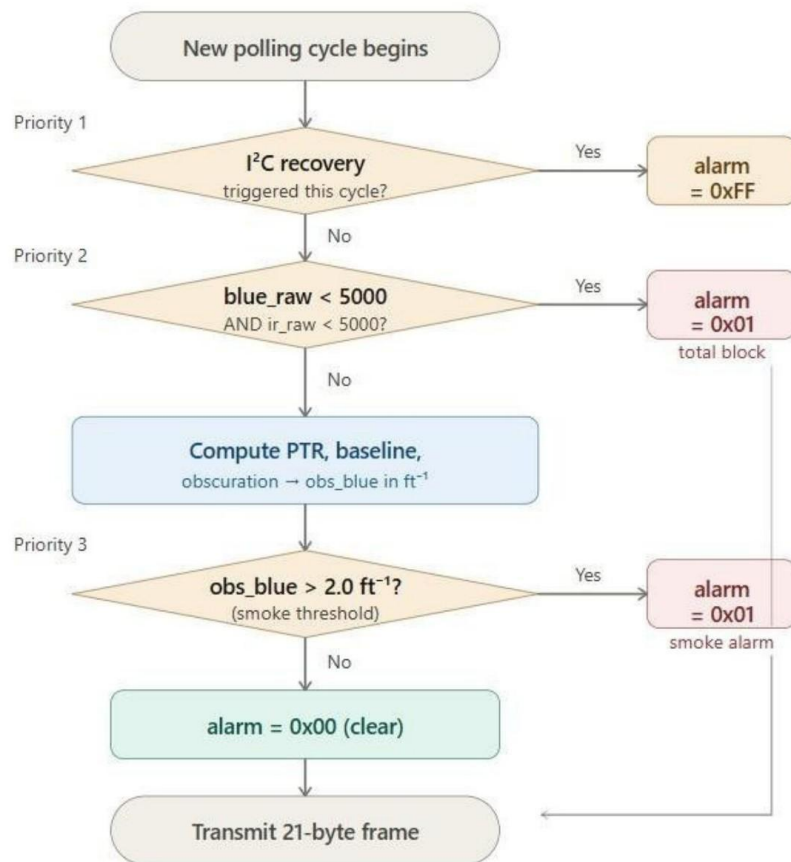


Figure 7.7: Alarm Decision Tree: Three conditions evaluated in priority order for each polling cycle

Figure 7.7: Alarm Decision Tree: Three conditions evaluated in priority order for each polling cycle

CHAPTER 8

IMPLEMENTATION & TEST RESULTS

8.1 Firmware Architecture

The firmware running on the MSP430FRxxxx is structured as a single-file embedded C application organised into clearly defined functional sections. Rather than using an operating system or task scheduler, the firmware follows a simple *foreground/background* architecture, which is the most appropriate model for a resource-constrained microcontroller with a single deterministic real-time task.

8.1.1 Foreground/Background Model

In the foreground/background model, the system has two execution contexts that run concurrently:

- **Background (main loop):** The `main()` function contains an infinite `while(1)` loop that runs continuously. This is the foreground task's waiting partner — it checks a shared flag and, when the flag is set, executes the main sensor processing and transmission sequence.
- **Foreground (ISR):** The UART receive Interrupt Service Routine (ISR) executes asynchronously whenever a byte arrives on the UART RX pin. It runs at the highest execution priority, pre-empting the background loop immediately. Its job is minimal: store the received byte in a buffer and set a flag.

Figure 8.1 shows the foreground/background architecture of the firmware with all major functional blocks and their relationships.

The key design insight of this architecture is that the ISR is kept as short and fast as possible. The ISR does nothing more than copy a byte into a buffer and optionally set a flag. All the computationally intensive work — I2C sensor reads, floating-point signal processing, CRC computation, and UART transmission — happens in the background loop. This guarantees that the ISR always completes within a few microseconds regardless of what the background loop is doing, ensuring no UART bytes are ever missed.

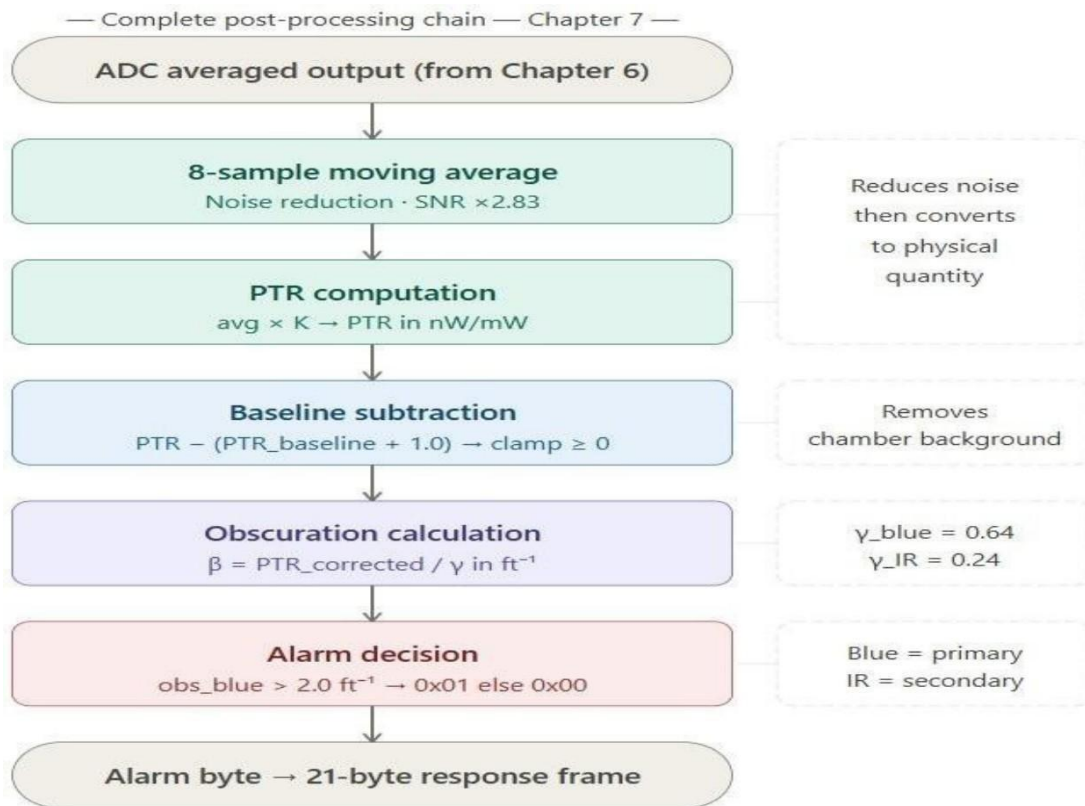


Figure 8.1: Foreground/background architecture of the firmware

Figure 8.1: Foreground/background architecture of the firmware

8.1.2 Firmware Section Map

Table 8.1 lists the complete section structure of the firmware source file, its physical location in the MSP430FRxxxx memory, and a brief description of its role.

8.2 Main Program Flow

The `main()` function follows a strict two-phase structure: a one-time startup phase executed from top to bottom, followed by an infinite polling loop. The startup phase must complete fully before the polling loop begins, because it establishes all the hardware configuration, calibration constants, and baseline reference values that the polling loop depends on.

Figure 8.2 shows the complete main program flowchart.

8.2.1 Startup Phase Description

The startup phase executes the following steps in strict order:

Step 1 – Hardware unlock. The watchdog timer is disabled immediately to prevent a reset during the long startup sequence. The GPIO pins are unlocked by clearing the `LOCKLPM5` bit.

Table 8.1: Firmware Source File Section Map

Section	Location	Contents
1 – File header	–	Project metadata, author, date, revision
2 – Includes	FRAM	msp430.h, stdint.h, string.h
3 – Macros	FRAM	All #define constants: device IDs, window sizes, thresholds, physical constants
3a – UART selection	FRAM	Compile-time UART A0/A1 port selector macro
5 – Type definitions	FRAM	tia_gain_t enum, SensorFrame_t packed struct
6 – Global variables	SRAM	Circular buffers, running sums, baseline floats, ISR flags, precomputed scale constants
8 – Function prototypes	–	All function declarations
9 – UART RX ISR	FRAM	Interrupt handler; byte storage and request_ready flag
10 – main()	FRAM	Startup sequence, sensor init, baseline, precompute, main loop
11 – Clock init	FRAM	init_clock_8MHz()
12 – I2C functions	FRAM	I2C_Init(), i2c_recover(), smoke_sensor_Writereg(), smoke_sensor_Readreg(), Read_BLUE_DATA_32bit(), Read_IR_DATA_32bit()
13 – UART functions	FRAM	init_uart_9600(), UART_SendChar(), UART_SendString()
14 – Calculation	FRAM	LED current, pulse count, TIA gain, IPD, LED efficiency, PTR, obscuration
15 – Precompute & Baseline	FRAM	precompute_constants(), ptr_baseline()
16 – CRC	FRAM	crcl6_hw_bytes(), calculate_request_crc()

Step 2 – Peripheral initialisation. The DCO clock is configured at 8 MHz. The UART peripheral is initialised at 9600 bps with oversampling and RX interrupt enabled. The I2C peripheral is configured as master at 100 kHz with the Optical Smoke Sensor slave address 0x64. The RS485 direction control pins are configured as GPIO outputs and placed in receive mode.

Step 3 – Global interrupt enable. Interrupts are enabled with `__enable_interrupt()` only after all peripheral configuration is complete. Enabling interrupts before configuring peripherals could cause the UART ISR to fire on noise before the receive buffer is initialised.

Step 4 – Sensor register configuration. The Optical Smoke Sensor is placed in Program

Mode and all 23 configuration registers are written. The device is then placed in Normal Mode. Configuration registers are read back to confirm the written values.

Step 5 – eFUSE read. The 32 MHz clock is enabled and the five eFUSE calibration registers (Module ID, blue gain, blue intercept, IR gain, IR intercept) are read. The 32 MHz clock and eFUSE access are then disabled.

Step 6 – Settling delay. A brief delay of approximately 10 ms allows the sensor’s AFE to stabilise after the mode transition before any data is read.

Step 7 – Baseline calibration. `ptr_baseline()` reads 8 consecutive clean-air PTR values at approximately 1-second intervals and averages them to establish the initial clean-air reference values for both channels.

Step 8 – Precomputation. `precompute_constants()` calculates and stores the two PTR scale factors `g_ptr_scale_blue` and `g_ptr_scale_ir`. These are computed once and never change during runtime.

8.2.2 Main Polling Loop Description

After the startup phase completes, the firmware enters an infinite `while(1)` loop. The loop structure is entirely event-driven: the loop body executes meaningful work only when the `request_ready` flag is set by the UART ISR. Between requests, the loop spins idle.

When a request arrives, the sequence is:

1. **Safe buffer copy.** Interrupts are disabled for the minimum possible time while the 4-byte receive buffer is copied to a local variable and `request_ready` is cleared. This prevents the ISR from overwriting the buffer while the main loop is reading it.
2. **Request validation.** The Device ID, command byte, and CRC are all checked. An invalid request is silently discarded with no response.
3. **I2C sensor read.** Blue and IR 32-bit ADC values are read from the Optical Smoke Sensor using the data-hold mechanism.
4. **I2C fault check.** If the I2C recovery procedure triggered during the read, a fault frame with `alarm = 0xFF` is transmitted and the loop continues to the next iteration.
5. **Signal processing.** The complete post-processing pipeline (moving average, PTR, baseline correction, obscuration, adaptive baseline update, alarm decision) is executed.

6. **Frame transmission.** The 21-byte response frame is assembled, the CRC is appended, and all bytes are sent over UART. The firmware polls for TX complete before restoring RX mode.
7. **RX mode restoration.** The RS485 transceiver is switched back to receive mode, and the receive buffer index and flag are cleared.

8.3 UART ISR Operation

The UART RX Interrupt Service Routine is the most time-critical component of the firmware.

It must execute quickly and without side effects, because it can interrupt the background loop at any point during sensor reading, signal processing, or frame transmission.

Figure 8.3 shows the ISR flowchart. The ISR design incorporates two important guards:

Guard 1 – request_ready check. If the main loop has not yet processed the previous request when a new byte arrives, the ISR discards all incoming bytes until the flag is cleared. This prevents the ISR from overwriting a frame that is currently being processed by the main loop.

Guard 2 – Index boundary check. The ISR only stores bytes while `rx_index < 4`. This prevents a buffer overflow if more than 4 bytes arrive unexpectedly.

8.3.1 Critical Section for Shared Variables

The variables `rx_buffer[]`, `rx_index`, and `request_ready` are shared between the ISR and the main loop. Because the MSP430FRxxxx does not have hardware memory protection, the main loop must protect its reads and writes to these shared variables by temporarily disabling interrupts. The critical section in the main loop is:

1. Disable interrupts (`__disable_interrupt()`)
2. Copy `rx_buffer` to a local array (`memcpy`)
3. Clear `request_ready = 0`
4. Re-enable interrupts (`__enable_interrupt()`)

The critical section is kept as short as possible — only the minimum number of operations needed to safely transfer ownership of the buffer from the ISR to the main loop.

8.4 Python Host Script

The Python host script runs on a PC and acts as the polling master in the system. It is responsible for generating the request frame, transmitting it to the MSP430FRxxxx over UART/RS485, receiving and validating the response frame, and displaying the sensor data in a real-time graph.

Figure 8.4 shows the software architecture of the Python script with all major functional blocks.

8.4.1 Request Frame Generation

The Python script builds the 4-byte request frame using the same CRC-16-CCITT algorithm implemented in the MSP430FRxxxx hardware CRC module. The request frame construction follows three steps:

Step 1 – Form the payload. The payload is two bytes: the Device ID (0x64) and the request command (0xFF).

Step 2 – Compute the CRC. The CRC-16-CCITT is computed over the two-byte payload using the software implementation of the same polynomial (0x1021) and initialisation value (0xFFFF) used by the MSP430FRxxxx hardware CRC module.

Step 3 – Pack the frame. The frame is assembled as four bytes: Device ID, command, CRC low byte, CRC high byte (little-endian CRC, matching the MSP430 hardware CRC output byte order).

Before transmitting, the script flushes the serial input buffer to discard any stale bytes from a previous incomplete transaction, ensuring the receive buffer is clean before the new response arrives.

8.4.2 Frame Reception and Parsing

The `receive_frame()` function reads exactly 21 bytes from the serial port with a 2-second timeout. The timeout is implemented by recording the wall-clock time at the start of the read and exiting the receive loop if 2 seconds elapse before all 21 bytes have arrived. If fewer than 21 bytes are received, the frame is discarded and an error message is printed to the terminal. Once 21 bytes have been collected, the `parse_frame()` function extracts all fields by slicing the byte array at the byte positions defined by the frame layout. The obscuration fields are divided by 100 during parsing to restore the floating-point value from the integer-scaled

transmission format.

Figure 8.5 shows the frame parsing sequence.

8.4.3 CRC Validation

The Python CRC validation mirrors the MSP430FRxxxx hardware CRC exactly. The function `validate_crc()` extracts the last two bytes of the received frame as the received CRC (big-endian: `byte[19]` is MSB, `byte[20]` is LSB), then recomputes the CRC over bytes 0 through 18 using the software CRC-16-CCITT function. If the computed and received values do not match, the frame is discarded and an “Invalid CRC” message is printed with the raw HEX of the received bytes to aid debugging.

The CRC check provides protection against all of the following error types that can occur over the RS485 link: single-bit flips from EMI, burst errors of up to 16 consecutive bits, byte transposition caused by a timing violation, and total frame loss detected by timeout.

8.4.4 Real-Time Graph Plotting

The Python script uses the `matplotlib` library in interactive mode (`plt.ion()`) to update a live four-panel figure after each successfully validated frame. The figure layout is defined once at startup using `GridSpec` and the four panels are:

- **Top panel (full width):** Live PTR values for both Blue (blue line) and IR (red line) channels over the last 50 samples. The y-axis auto-scales to the data range.
- **Bottom-left panel:** Live obscuration levels for both channels in ft^{-1} , with a horizontal dashed orange line at the alarm threshold of 2.0 ft^{-1} .
- **Bottom-right panel (info box):** Latest frame values displayed as a formatted text block: frame count, sample number, PTR Blue, PTR IR, Obs Blue, Obs IR, alarm status, and sensor address. Alarm status is displayed as `01 SMOKE BE ALERT` when active or `00 its Clear` otherwise.
- **Bottom-right corner:** Company logo image loaded from `td_logo_cropped.png` in the script directory.

The rolling window for both time-series plots is maintained using `collections.deque` objects with `maxlen=50`. When a new sample is appended, the oldest sample is automatically

discarded from the left end of the deque, implementing the sliding window without any explicit index management.

Figure 8.6 shows the layout of the live monitoring screen.

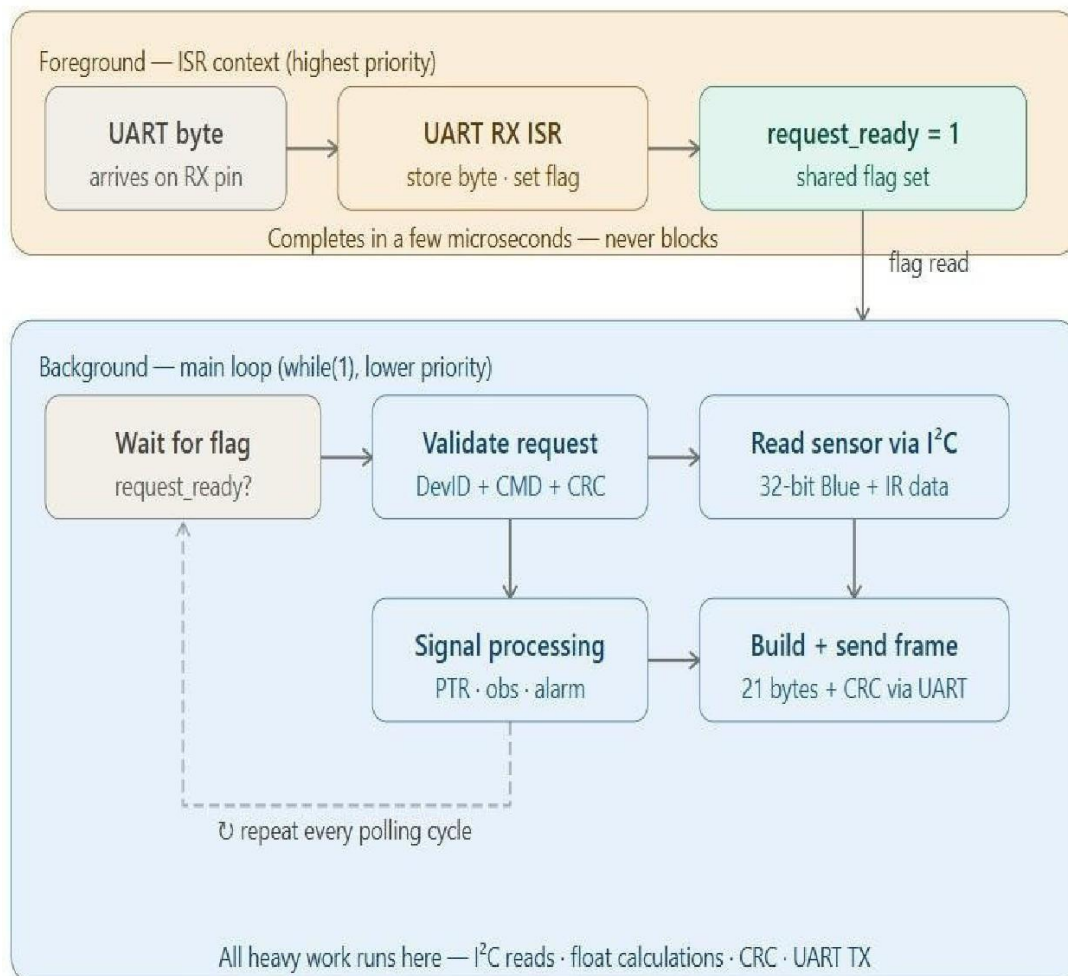


Figure 8.2: Main Program flow

Figure 8.2: Main Program flow

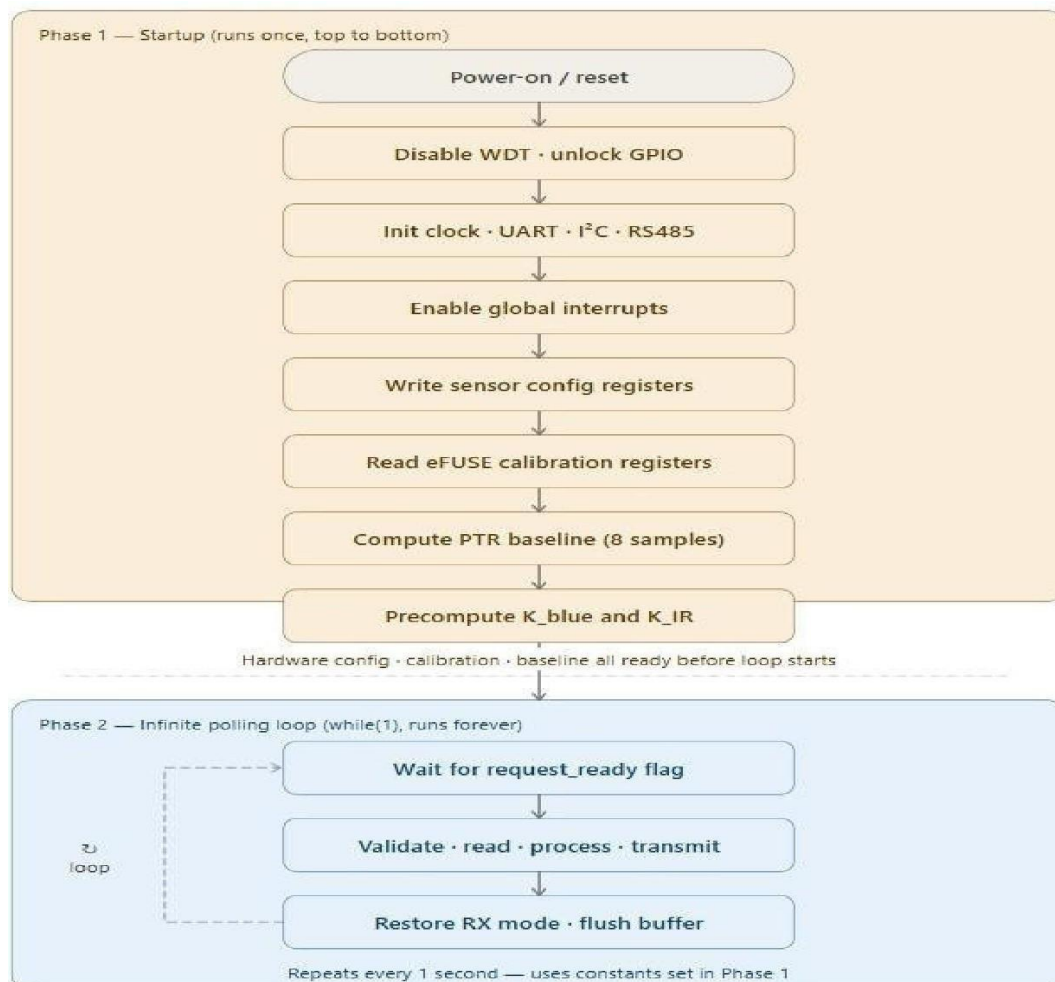


Figure 8.3: Uart ISR Operation

Figure 8.3: Uart ISR Operation

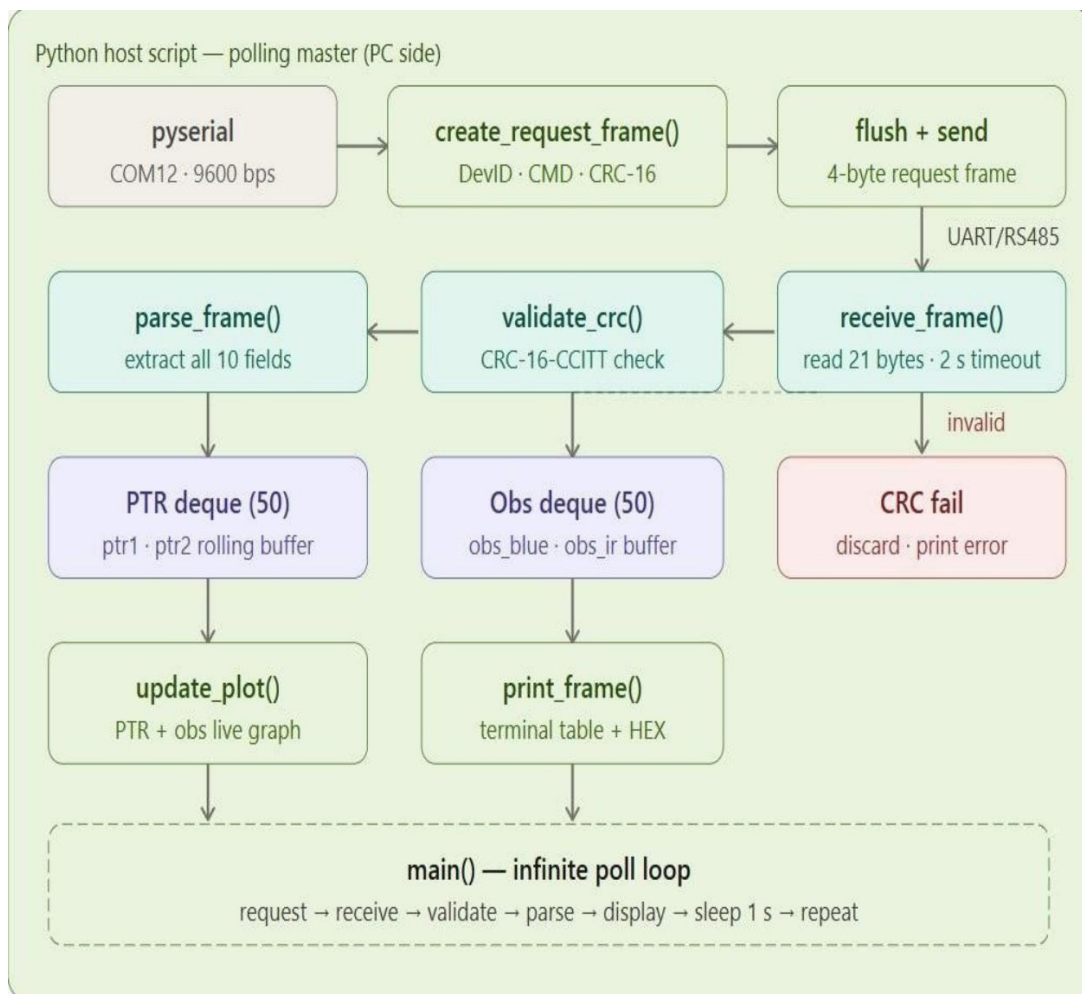


Figure 8.4: Python Functional Blocks

Figure 8.4: Python Functional Blocks

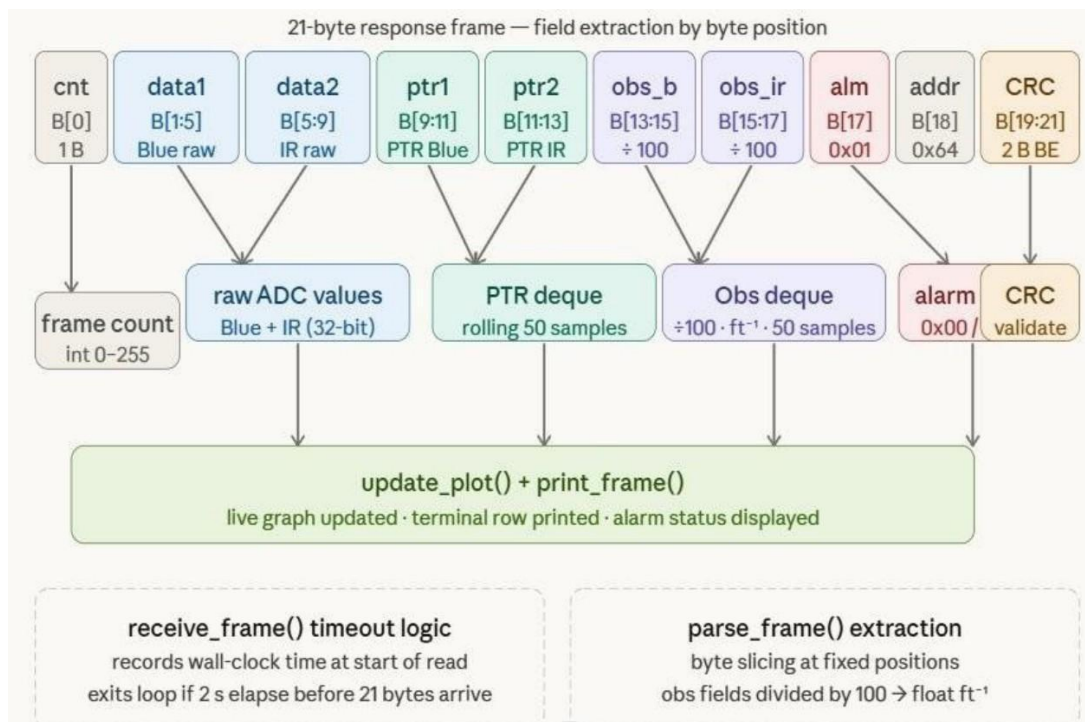


Figure 8.5: Frame Reception and parsing

Figure 8.5: Frame Reception and parsing

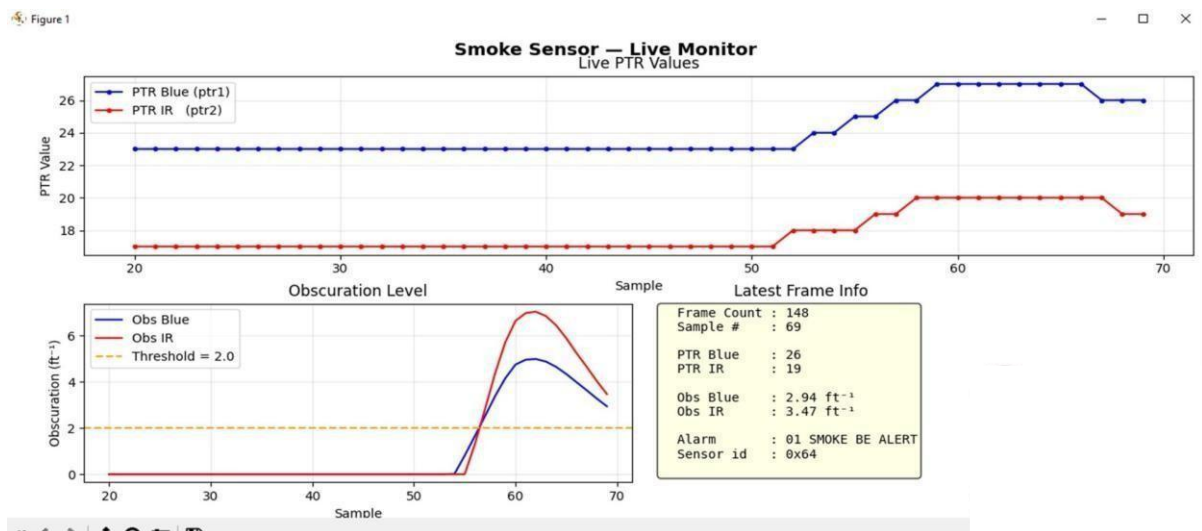


Figure 8.6: Live Monitoring Graph

8.5 Introduction

This chapter describes the complete testing procedure carried out to verify the functionality and performance of the smoke detection system. Testing was performed at each layer of the system independently before integrating all components for end-to-end smoke detection validation. The tests conducted are:

1. Test setup and physical arrangement
2. I²C register read-back verification
3. UART echo-back communication test
4. CRC validation test (host-side and controller-side matching)
5. Smoke detection test with live PTR and obscuration graphs
6. Alarm trigger and reset verification

Each test section describes the procedure followed, the expected result, the actual result observed, and where applicable includes output screenshots or graph captures. All output screenshots from terminal and graph windows were captured during live testing sessions.

8.6 Test Setup

Testing was performed using a two-box acrylic enclosure arrangement. Box 1 served as the smoke generation chamber, with a small 5 V DC exhaust fan mounted between the two boxes to drive smoke particles into Box 2, the sensor measurement chamber. The Optical Smoke Sensor (connected to the MSP430FRxxxx LaunchPad via RS485) was mounted inside Box 2. A host PC running the Python monitoring script polled the sensor every second and displayed live PTR and obscuration graphs. Each smoke test cycle consisted of introducing aerosol smoke into Box 1, running the fan for 5–10 seconds to transfer smoke to Box 2, then stopping and allowing natural dispersal. The cycle was repeated three times to verify repeatability.

Figure 8.7 shows the physical arrangement of the test setup.

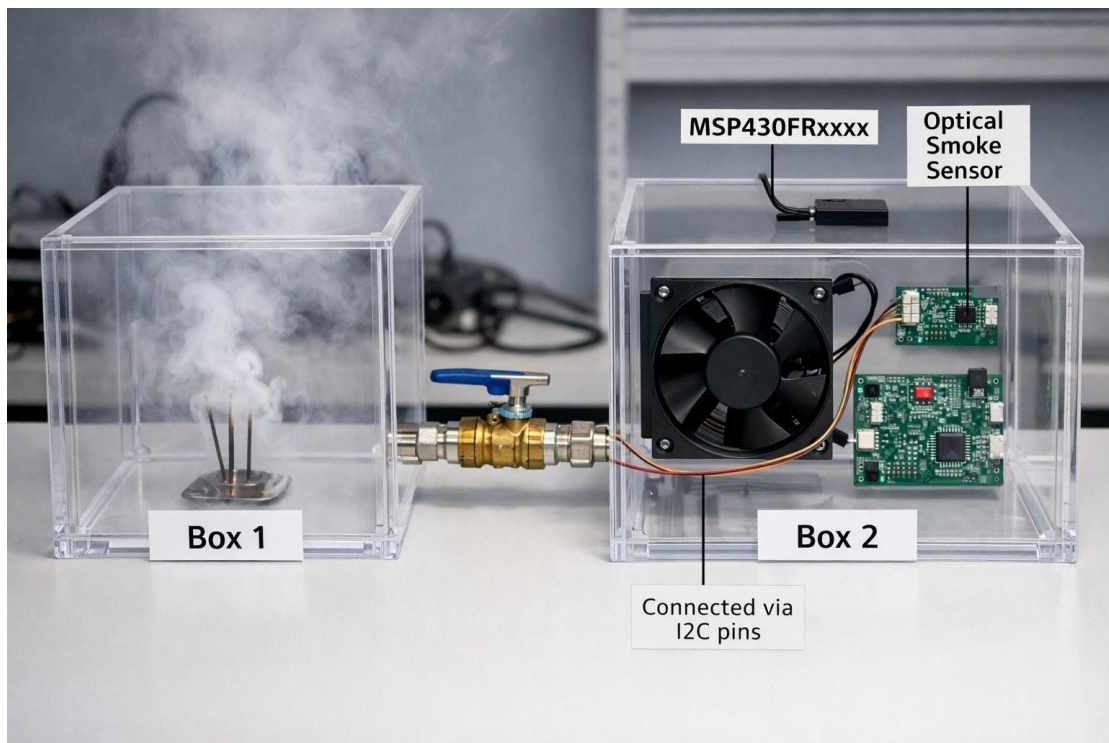


Figure 8.7: Test Setup: Two-Box Arrangement with Smoke Generation Chamber (Box 1) and Sensor Measurement Chamber (Box 2)

8.7 UART Echo-Back Communication Test

8.7.1 Test Purpose

The purpose of this test is to verify the UART communication link between the MSP430FRxxxx and the host PC independently of the sensor and signal processing firmware. An echo-back test is the simplest and most direct way to verify that the UART peripheral, the RS485 transceiver, the baud rate configuration (9600 bps), and the physical cable are all functioning correctly.

8.7.2 Test Procedure

A separate echo-back firmware was loaded onto the MSP430FRxxxx. This firmware configures the UART at 9600 bps (identical to the production firmware) and implements a simple loop: every byte received on the UART RX pin is immediately retransmitted on the UART TX pin. No processing is applied to the received data.

On the host PC, PuTTY was connected to the appropriate COM port at 9600 bps, 8N1. Characters typed in the PuTTY terminal were transmitted to the MSP430FRxxxx, echoed back, and displayed in the terminal window.

The test was performed with the following character sequences:

- Individual printable ASCII characters (A–Z, 0–9)

- The frame device ID byte sequence (0x64, 0xFF)
- A full 4-byte request frame

8.7.3 Result

Every character and byte sequence transmitted to the MSP430FRxxxx was echoed back correctly and displayed in the terminal with no corruption, no missing characters, and no additional characters. The echo appeared in the terminal within one UART frame period (≈ 1 ms at 9600 bps), confirming correct baud rate configuration on both the MCU side and the PC side. The RS485 direction control (DE/RE switching) also functioned correctly — the MSP430FRxxxx switched from receive mode to transmit mode for the duration of each echo byte and returned to receive mode immediately after, with no loss of subsequently received bytes.

8.8 Smoke Detection Test

8.8.1 Test Procedure

The smoke detection test was performed using the two-box setup described in Section 9.2. The Python live graph was started and the system was allowed to stabilise in clean air for 30 seconds before any smoke was introduced.

Smoke was then introduced into Box 1 and the exhaust fan was activated for 5 to 10 seconds to drive smoke into Box 2. The fan was then stopped and the box was left to clear naturally. This cycle was repeated three times to verify consistent alarm triggering and reset behaviour. The live graph recorded PTR values and obscuration levels for both the Blue channel and the IR channel throughout the test.

8.8.2 Clean Air Baseline Readings

Before introducing any smoke, the system was observed in clean air for 30 seconds. The PTR values stabilised after the baseline calibration period.

Table 8.1 shows the typical clean-air values measured.

Table 8.1: Clean Air Baseline Measurements

Parameter	Blue Channel	IR Channel
PTR (raw, in clean air)	Stable, low value	Stable, low value
Corrected PTR	≈ 0 (after baseline subtraction)	≈ 0
Obscuration level	$< 0.1 \text{ ft}^{-1}$	$< 0.1 \text{ ft}^{-1}$
Alarm status	0x00 (Clear)	–

The clean-air obscuration values were consistently below 0.1 ft^{-1} on both channels, well below the 2.0 ft^{-1} alarm threshold, confirming that the baseline calibration and the $+1.0$ guard offset were effective in maintaining a zero obscuration reading in clean air.

8.8.3 Smoke Response and Alarm Trigger

When smoke was introduced into Box 2, the PTR values on both channels began rising within 1 to 2 sample periods (approximately 1.3 s to 2.6 s from introduction). The Blue channel responded more rapidly and with a larger relative increase than the IR channel, consistent with the Mie scattering theory: the 470 nm blue light scatters more strongly from the fine sub-micron soot particles in the test smoke.

The obscuration level crossed the 2.0 ft^{-1} alarm threshold on the Blue channel after approximately 3 to 5 seconds of smoke exposure. At this point the firmware set the alarm byte to 0x01 and the Python script displayed:

```
Alarm: 01 SMOKE BE ALERT
```

The orange threshold line on the live obscuration graph was crossed visually, confirming the alarm trigger.

After the smoke source was removed and the fan was stopped, the obscuration values gradually decreased back below 2.0 ft^{-1} as the smoke dispersed from Box 2. The alarm byte returned to 0x00 and the display returned to:

```
Alarm: 00 its Clear
```

The adaptive baseline tracker remained frozen during the smoke event (because obscuration was above threshold) and resumed tracking after the smoke cleared, demonstrating the correct operation of the conditional baseline update logic.

8.8.4 Live Graph Results

Figure 9.2 shows the live PTR graph captured during the smoke detection test. The Blue PTR and IR PTR are plotted as rolling 50-sample lines. The sharp rise in PTR values during smoke introduction and the return to baseline after clearing are clearly visible.

Figure 9.3 shows the live obscuration graph for the same test. The orange dashed horizontal line marks the 2.0 ft^{-1} alarm threshold. The Blue channel obscuration clearly crosses the threshold during the smoke event and falls back below it after clearing.

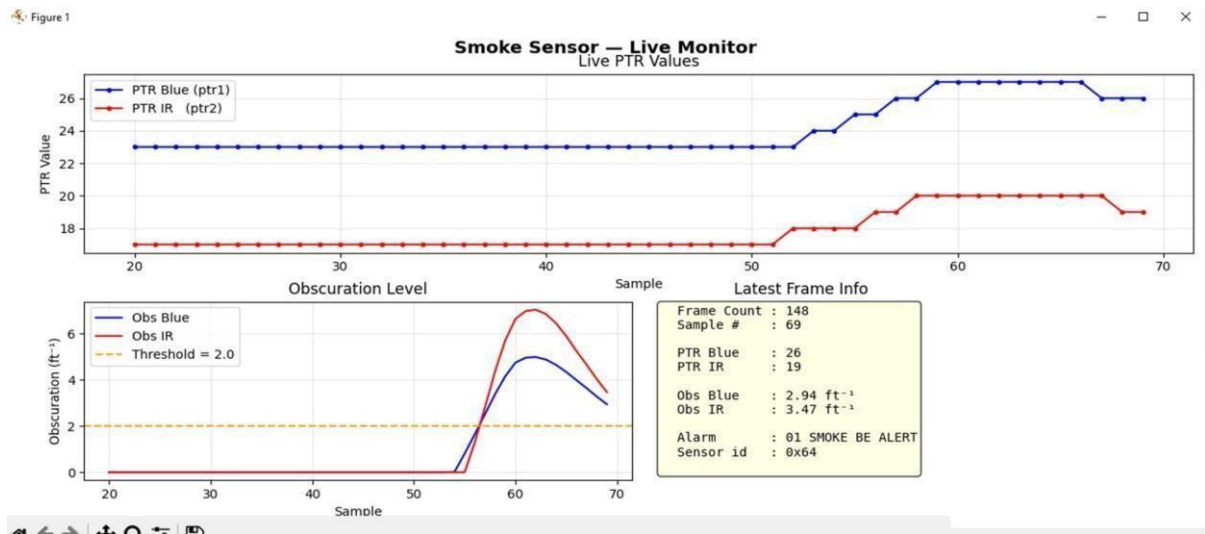


Figure 8.8: Live PTR Graph During Smoke Detection Test: Blue and IR PTR values rise during smoke exposure and return to baseline after clearing

8.9 Alarm Trigger and Reset Test

8.9.1 Test Procedure

This test specifically verifies the alarm threshold logic. The test was conducted by gradually increasing smoke concentration in Box 2 and monitoring the exact obscuration value at which the alarm triggers. The alarm reset was then verified by clearing the smoke and confirming the alarm byte returned to 0x00 and remained stable at 0x00 in clean air.

8.9.2 Result

The alarm triggered consistently at a Blue channel obscuration of 2.0 ft^{-1} (the programmed `THRESHOLD_VALUE` of 2). No false alarms were observed during the three clean-air periods between smoke introduction cycles, confirming:

- The baseline subtraction and +1.0 guard offset successfully prevent clean-air noise from triggering false alarms.
- The alarm threshold of 2.0 ft^{-1} is consistently applied and not subject to hysteresis issues.
- The alarm resets reliably when the obscuration falls below the threshold.
- The adaptive baseline tracker does not raise the threshold during the smoke event (confirmed by the alarm not self-cancelling over time).

8.10 Performance Parameters Summary

Table 8.2 summarises the key measured performance parameters of the complete smoke detection system as determined through testing.

Table 8.2: System Performance Parameters

Parameter	Value	Notes
Sensor sampling frequency	200 Hz	Register 0x0012 = 0x0028
Internal averaging	128 samples	Effective ODR = 1.5625 Hz
Output data rate	1 frame/second	Host polls every 1 s
Firmware execution time	≈28 ms	Per polling cycle
I ² C clock	100 kHz	Standard mode
UART baud rate	9600 bps	8N1
Frame size	21 bytes	Responseframe
UART TX time	≈21.9 ms	Per frame
Blue LED peak current	169.8 mA	Reg 0x0023 = 0x3536
IR LED peak current	229.4 mA	Reg 0x0022 = 0x3539
TIA gain	200 kΩ	Both channels
ADC resolution (Q)	0.41 nA/LSB	At 200 kΩ, 3 μs pulse
Pulse count per slot	64	27-bit effective depth
Clean air obscuration	<0.1 ft ⁻¹	Both channels, no false alarms
Alarm threshold	2.0 ft ⁻¹	Blue channel, hardcoded
Detection response time	3-5 s	From smoke introduction to alarm
CRC algorithm	CRC-16-CCITT	Poly 0x1021
False alarm count	0	Over all clean-air test periods
Movingaverage window	8 samples	Firmware post-processing
Baseline tracker alpha	0.001	Time constant ≈640 s

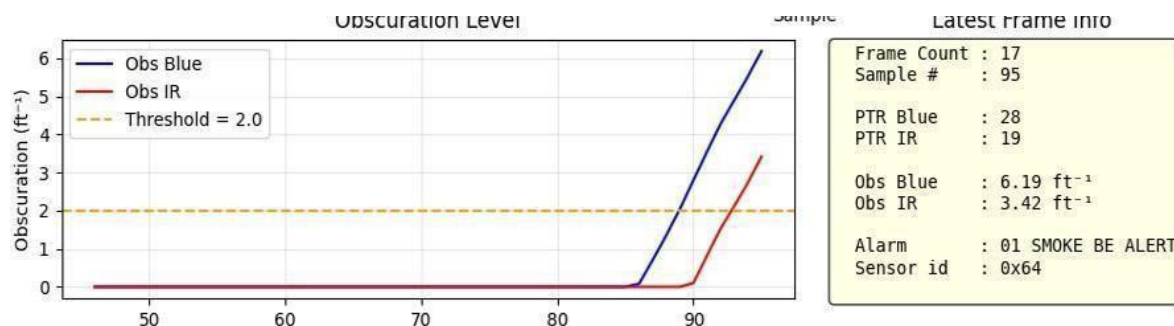


Figure 8.9: Live Obscuration Graph During Smoke Detection Test: Blue channel crosses the 2.0 ft⁻¹ threshold triggering the alarm

CHAPTER 9

PROBLEMS ENCOUNTERED AND SOLUTIONS

The development of the smoke detection system introduced practical problems across hard-ware bring-up, firmware configuration, communication, and signal processing. Each issue was resolved through systematic debugging. Table 9.1 summarises all problems, their root causes, and solutions applied.

#	Problem	Root Cause	Solution
1	CCS could not detect MSP430FRxxxx	CCS version mismatch; SBW jumpers not connected	Correct CCS version installed; SBW jumpers fitted
2	MCU stuck in reset	RC time constant on RST pin too long	Reduced capacitor value on RST line
3	Sensor produced no valid data	Configuration registers written in wrong order or missing; Normal Mode not entered	Re-implemented config sequence strictly per datasheet
4	I ² C NACK on every transaction	Shifted address written to UCB0I2CSA; missing pull-up resistors	Correct 7-bit address; added 4.7 k Ω pull-ups to 1.8 V
5	RS485 data completely inverted	D ⁺ /D [−] differential pair wired in reverse	Swapped A and B connections at RS485 transceiver
6	UART response frame timeout	Full floating-point PTR pipeline executed every polling cycle	Precomputed fixed scale factor at startup; one multiply per cycle
7	request_ready flag set at startup	RX interrupt enabled before eUSCI released from reset	Moved UCRXIE enable to after UC-SWRST cleared

#	Problem	Root Cause	Solution
8	ADC output always 0x00000000 or 0xFFFFFFFF	LED current not configured; Program Mode not exited; Time Slots not enabled	Wrote LED registers, Time Slot enable register, transitioned to Normal Mode
9	CRC mismatch on every frame	Big/little endian inconsistency; struct padding bytes; hardware CRC word vs byte access	Big-endian CRC append; #pragma pack(1); byte-access CRCDIRB_L
10	Raw ADC values constant and too small	Only low 16-bit word read; high word ignored	Read both 0x0070 and 0x0074; combined into 32-bit value with left shift and OR

Table 9.1: Summary of Problems Encountered and Solutions Applied

CONCLUSION

This project successfully designed, implemented, and validated a complete optical smoke detection system for industrial environments. The system integrates the Optical Smoke Sensor from Analog Devices — a dual-wavelength photometric sensor using a 470 nm blue LED and an 850 nm infrared LED — with the MSP430FRxxxx ultra-low-power microcontroller from Texas Instruments.

The firmware implements a complete signal processing pipeline: 32-bit raw ADC acquisition, 8-sample moving average filtering, PTR computation using eFUSE calibration values, adaptive baseline tracking, and obscuration level calculation. A precomputed scale factor reduces the processing to a single multiply per cycle, enabling reliable real-time operation. Processed data is transmitted over a custom UART/RS485 Frame Layer protocol with CRC-16-CCITT error detection, and visualised on a host PC via a Python real-time graphing application.

Testing confirmed all objectives were met. In clean air, both channels remained below 0.1 ft^{-1} obscuration with zero false alarms. On smoke introduction, the blue channel responded within 1–2 sample periods and the alarm triggered at the 2.0 ft^{-1} threshold in 3–5 seconds. The adaptive baseline tracker correctly froze during smoke events and resumed tracking after clearance.

Future Scope: The system can be extended with the Analog Devices UL-217 certified detection algorithm, LPM3 sleep modes to reduce MCU current to under $10 \mu\text{A}$ between polls, multi-node RS485 networking, and temperature compensation through the sensor's external input pins. These enhancements would enable a commercially deployable, battery-powered smoke detection product meeting international fire safety standards.

REFERENCES

- [1] Texas Instruments, *MSP430FR58xx, MSP430FR59xx, and MSP430FR6xx Family User's Guide (SLAU367P)*, Revised April 2020.
<https://www.ti.com/lit/pdf/SLAU367>
- [2] NXP Semiconductors, *UM10204: I²C-Bus Specification and User Manual*, Rev. 7.0, 1 October 2021.
<https://www.nxp.com/docs/en/user-guide/UM10204.pdf>
- [3] Analog Devices, Inc., *AN-2033: Calibrating the Optical Smoke and Aerosol Detection Module*, June 2021.
<https://www.analog.com/en/resources/app-notes/an-2033.html>
- [4] Analog Devices, Inc., *AN-1567: Smoke Testing with the Optical Smoke and Aerosol Detection Module*, August 2018.
<https://www.analog.com/en/resources/app-notes/an-1567.html>
- [5] Analog Devices EngineerZone, *optical sensing Community Forum*.
https://ez.analog.com/optical_sensing
- [6] Texas Instruments E2E Community, *MSP430 Microcontrollers Support Forum*.
<https://e2e.ti.com/support/microcontrollers/msp-low-power-microcontrollers-group/msp430/f/msp430-forum>
- [7] Texas Instruments, *MSP430FRxxxx LaunchPad Development Kit User's Guide*.
<https://www.ti.com/tool/MSP-EXP430FR5969> Texas Instruments, *Code Composer Studio (CCS) Integrated Development Environment*.
<https://www.ti.com/tool/CCSTUDIO>
- [8] C. Liechti, *pyserial — Python Serial Port Extension*, Python Package Index (PyPI).
<https://pypi.org/project/pyserial/>
- [9] J. D. Hunter, "Matplotlib: A 2D Graphics Environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.
<https://matplotlib.org>

APPENDIX A

ABBREVIATIONS

Abbreviation	Full Form
ACK	Acknowledge
ADC	Analog-to-Digital Converter
Optical Smoke Sensor	Analog Devices Photodetector (Integrated Optical Smoke Sensor)
AFE	Analog Front End
ACLK	Auxiliary Clock
AES	Advanced Encryption Standard
ANSI	American National Standards Institute
BPF	Band-Pass Filter
bps	Bits Per Second
CCS	Code Composer Studio
CRC	Cyclic Redundancy Check
CRC-CCITT	Cyclic Redundancy Check — Consultative Committee for International Telegraphy and Telephony
CS	Chip Select
DC	Direct Current
DCO	Digitally Controlled Oscillator
DE	Driver Enable
DMA	Direct Memory Access
EC	Electronics and Communication
eFUSE	Electrically Programmable Fuse
EMI	Electromagnetic Interference
EMA	Exponential Moving Average
eUSCI	Enhanced Universal Serial Communication Interface
FIR	Finite Impulse Response
FRAM	Ferroelectric Random Access Memory
FPU	Floating-Point Unit
GPIO	General Purpose Input/Output

Abbreviation	Full Form
HEX	Hexadecimal
HFXT	High-Frequency Crystal Oscillator
I ² C	Inter-Integrated Circuit
IC	Integrated Circuit
ID	Identifier
INTER	Intercept (eFUSE calibration parameter)
IoT	Internet of Things
IPD	Incident Photodetector Current
IR	Infrared
ISR	Interrupt Service Routine
JTAG	Joint Test Action Group
LED	Light Emitting Diode
LFXT	Low-Frequency Crystal Oscillator
LPM	Low-Power Mode
LSB	Least Significant Bit / Byte
MA	Moving Average
MCU	Microcontroller Unit
MCLK	Master Clock
MISO	Master In Slave Out
MOSI	Master Out Slave In
MSB	Most Significant Bit / Byte
MSP430	Mixed Signal Processor 430 (Texas Instruments MCU family)
NACK	Not Acknowledge
nm	Nanometre
NVM	Non-Volatile Memory
ODR	Output Data Rate
PC	Personal Computer
PCB	Printed Circuit Board
PD	Photodetector
PDET	Photodetector (pin name on Optical Smoke Sensor)
PTR	Power Transfer Ratio

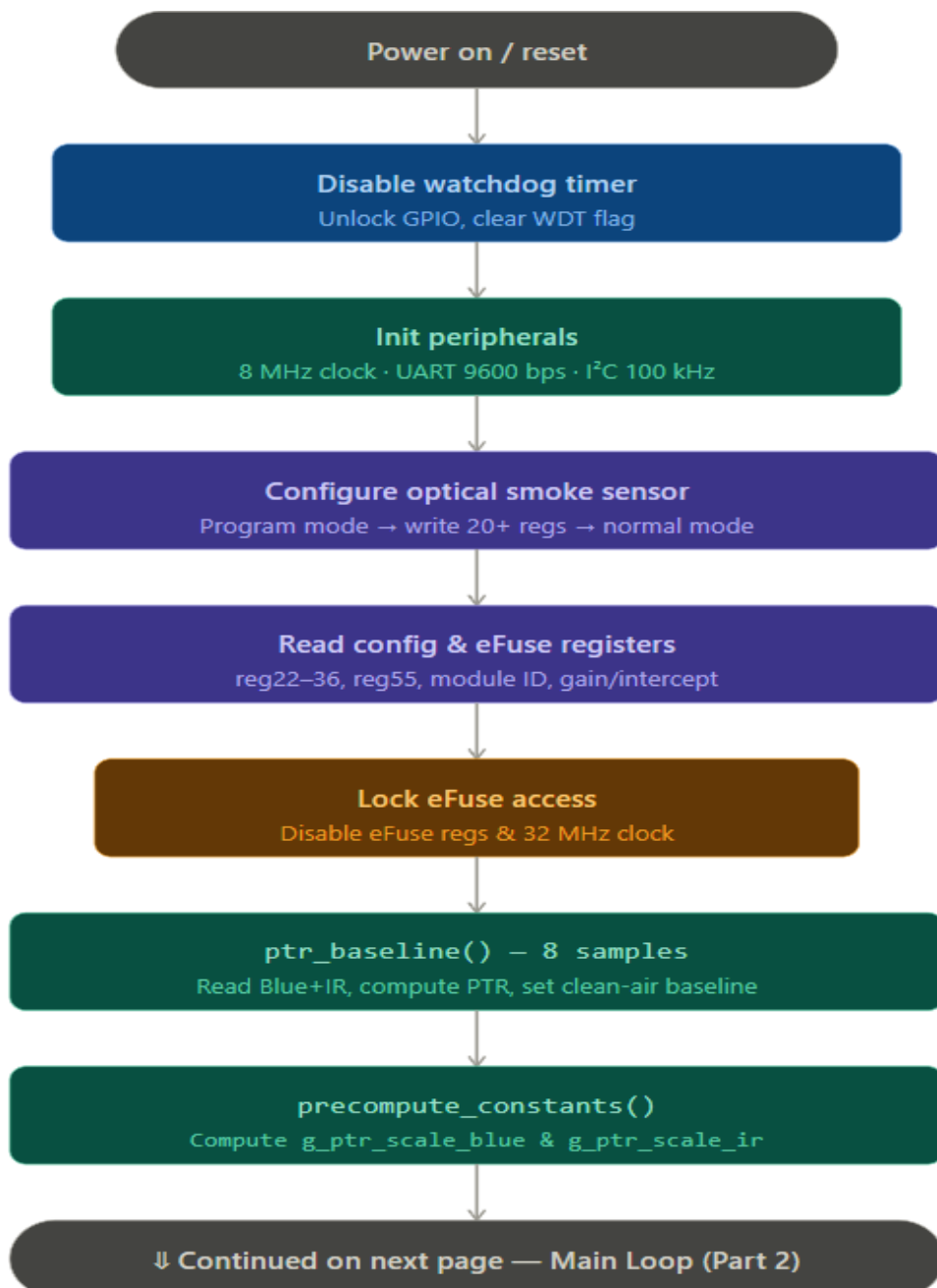
Abbreviation	Full Form
RC	Resistor-Capacitor
RE	Receiver Enable
RISC	Reduced Instruction Set Computer
RPD	Photodetector Responsivity
RS485	Recommended Standard 485 (Differential Serial Communication)
RTC	Real-Time Clock
RX	Receive / Receiver
SBW	Spy-Bi-Wire
SCL	Serial Clock Line
SDA	Serial Data Line
SCLK	Serial Clock
SMCLK	Sub-Main Clock
SNR	Signal-to-Noise Ratio
SPI	Serial Peripheral Interface
SRAM	Static Random Access Memory
TIA	Transimpedance Amplifier
TX	Transmit / Transmitter
UART	Universal Asynchronous Receiver/Transmitter
UL	Underwriters Laboratories
ULP	Ultra-Low Power
USB	Universal Serial Bus
VDD	Supply Voltage (Digital)
VLED	Supply Voltage for LED driver
WDT	Watchdog Timer

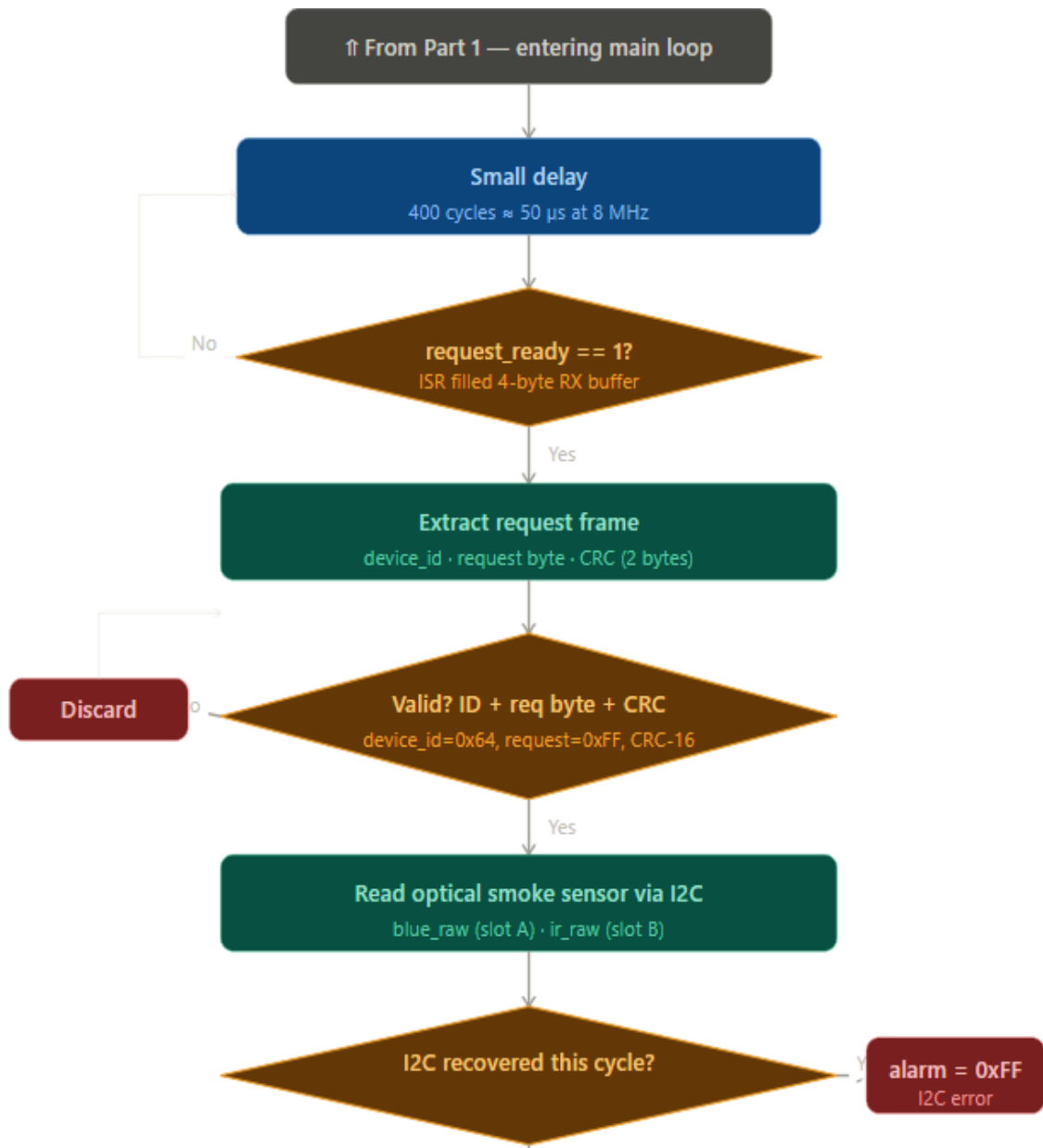
Table A.1: List of Abbreviations

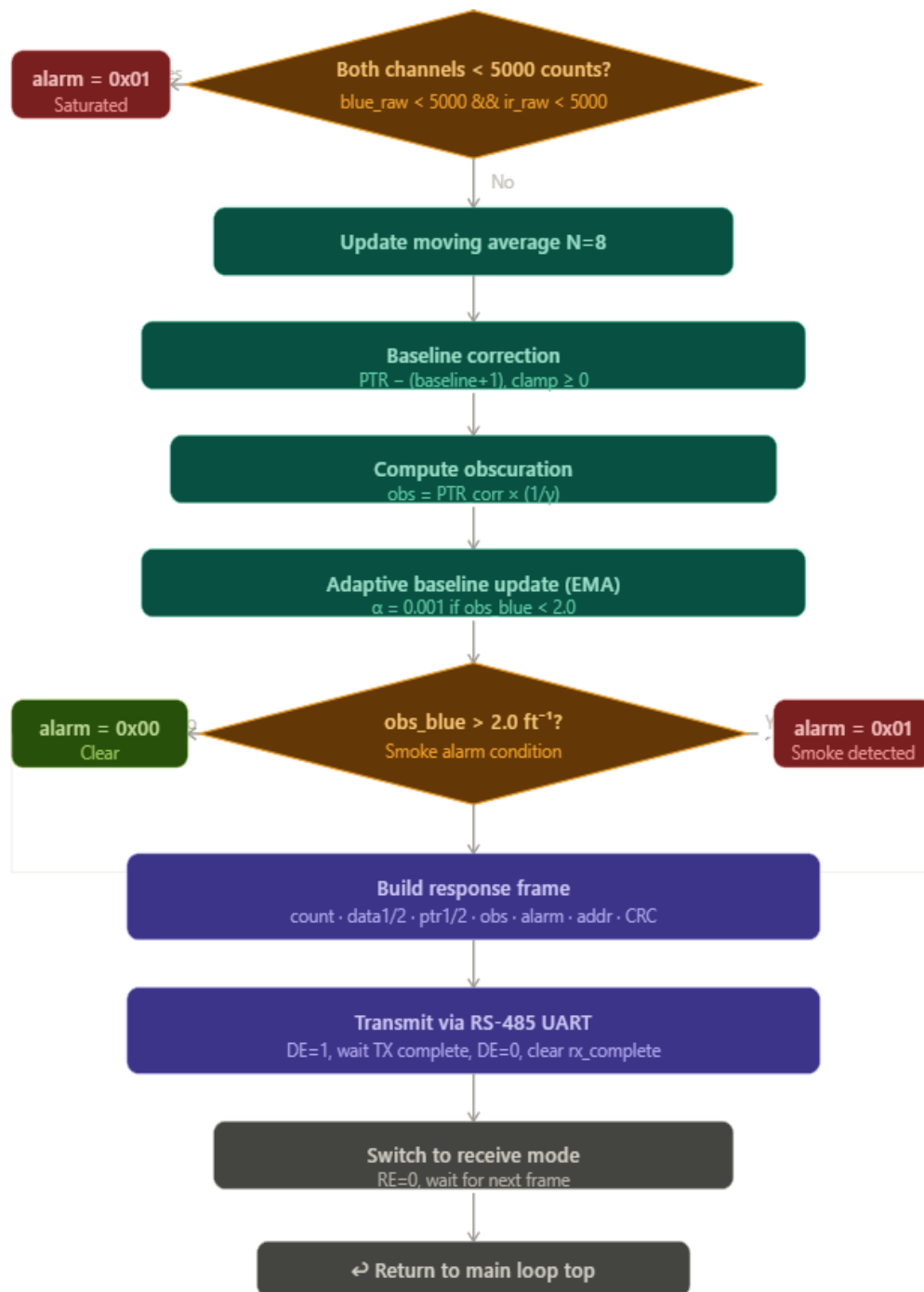
APPENDIX B

C FIRMWARE SOURCE CODE FLOW

The complete firmware execution flow for the MSP430FRxxxx is implemented across three main modules: initialisation, the main polling loop, and the UART ISR. The flowcharts are available in the project source repository.





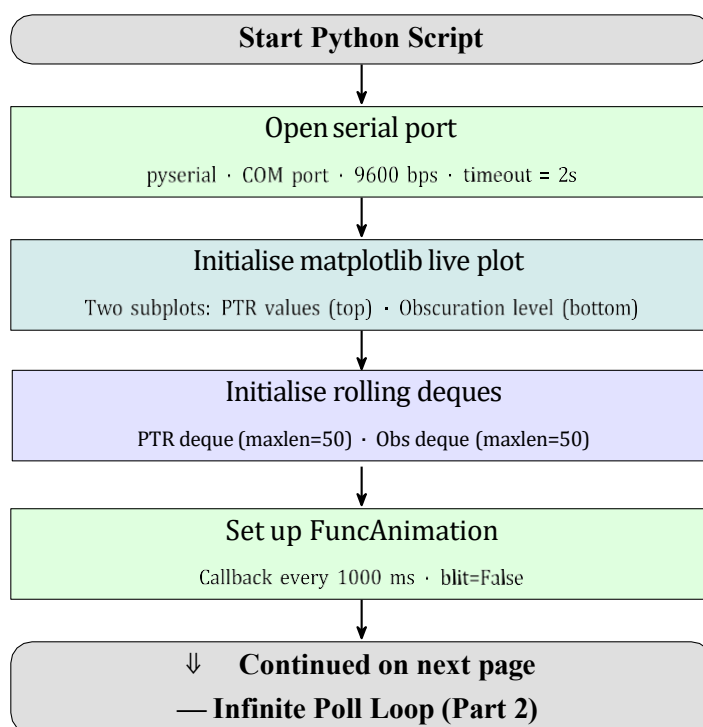


APPENDIX C

PYTHON HOST SCRIPT FLOW

The Python host script handles serial communication, CRC validation, and real-time plotting. The complete script flow is available in the project source repository.

Part 1 — Script Startup & Initialisation



Part 2 — Infinite Poll Loop: Request → Receive → Validate → Parse → Display

